150 questions

d'entretien technique

expliquées simplement, avec un schéma — du niveau Junior au niveau Senior

150 interview questions, explained simply · Junior to Senior · FR / EN

- Junior · 56
- Confirmé · 67
- Senior · 27

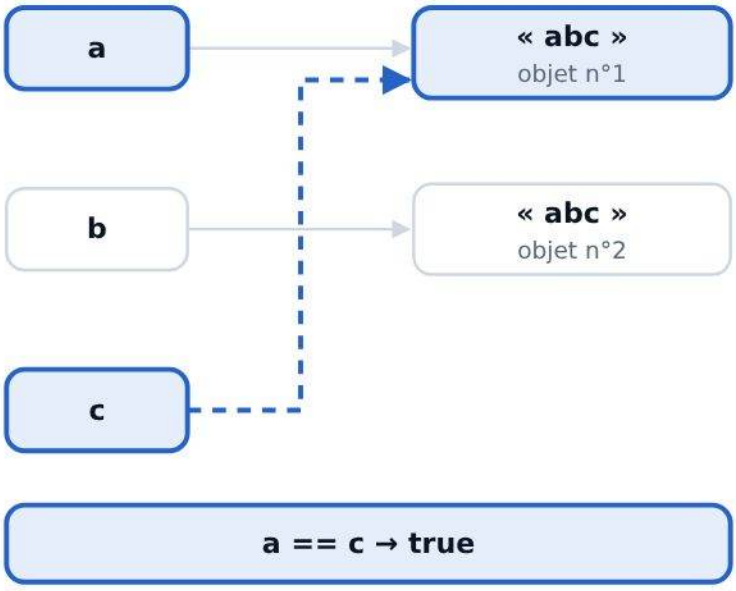
Java	29 questions
Spring Boot	28 questions
JPA / Hibernate	17 questions
SQL / PostgreSQL	19 questions
Angular	25 questions
RxJS	16 questions
REST, Git, Docker	16 questions

Rached Chakchouk

Ingénieur Full Stack Java / Angular · rached-chakchouk.vercel.app

== ou equals

== or equals

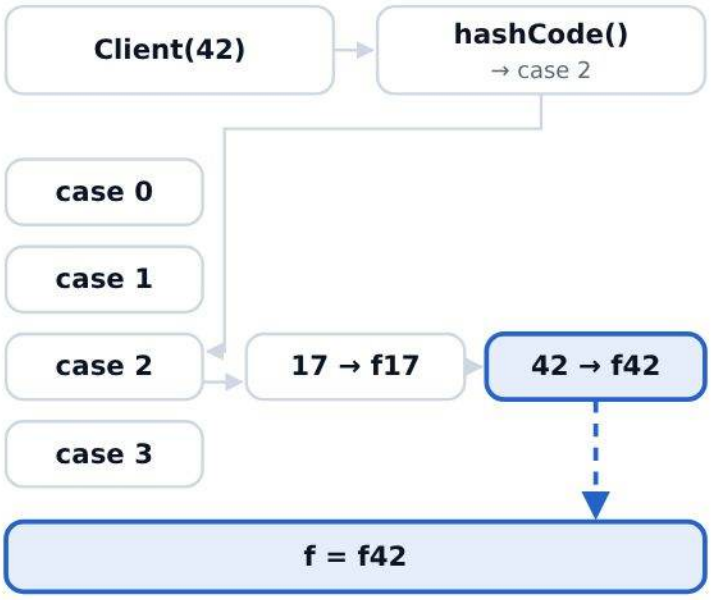


La réponse, étape par étape · *Step-by-step answer*

1. a pointe vers un premier objet « abc » en mémoire.
   a points to a first "abc" object in memory.
2. new crée un deuxième objet, même s'il contient le même texte.
   new creates a second object, even if it contains the same text.
3. == compare les adresses : ce sont deux objets différents, donc false.
   == compares addresses: these are two different objects, so false.
4. equals compare le contenu : « abc » = « abc », donc true.
   equals compares content: "abc" = "abc", so true.
5. c pointe vers le même objet que a : cette fois, == renvoie true.
   c points to the same object as a: this time, == returns true.

hashCode et equals

hashCode and equals

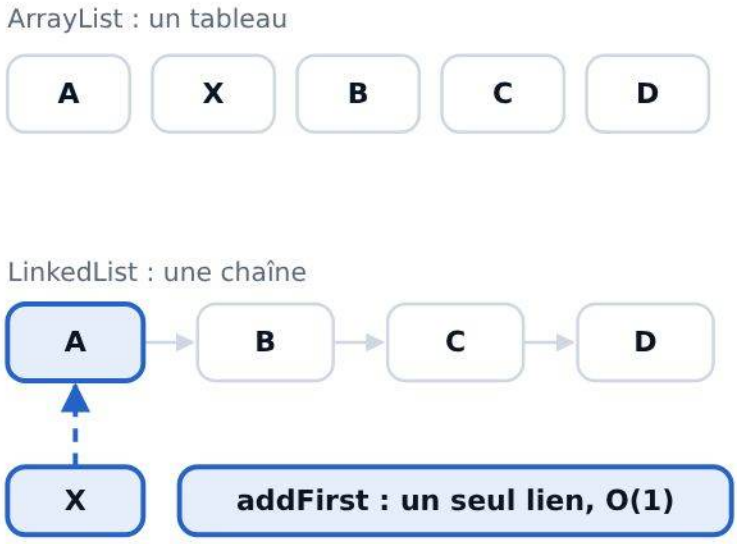


La réponse, étape par étape · *Step-by-step answer*

1. Une HashMap, c'est comme une armoire avec des tiroirs (les « cases »). Ici, 4 tiroirs vides.
   A HashMap is like a cabinet with drawers (the "buckets"). Here, 4 empty drawers.
2. On range le client 17. hashCode() donne le numéro du tiroir : le 2. On y met sa facture.
   We store customer 17. hashCode() gives the drawer number: 2. We put their invoice there.
3. On range le client 42. Lui aussi va dans le tiroir 2. Pas grave : un tiroir peut contenir plusieurs éléments.
   We store customer 42. It also goes into drawer 2. No problem: a drawer can hold several items.
4. On cherche le client 42. hashCode() dit « tiroir 2 ». Java ouvre directement ce tiroir, sans regarder les autres.
   We look for customer 42. hashCode() says "drawer 2". Java opens that drawer directly, without checking the others.
5. Dans le tiroir, equals() compare : est-ce le client 42 ? Le 17 ? Non.
   Inside the drawer, equals() compares: is it customer 42? The 17? No.
6. Le suivant, le 42 ? Oui, on l'a trouvé.
   The next one, the 42? Yes, we found it.
7. Résultat : f42. À retenir : deux objets égaux avec equals() doivent avoir le même hashCode(). Sinon, Java ouvre le mauvais tiroir.
   Result: f42. Remember: two objects that are equal with equals() must have the same hashCode(). Otherwise, Java opens the wrong drawer.

ArrayList ou LinkedList

ArrayList or LinkedList

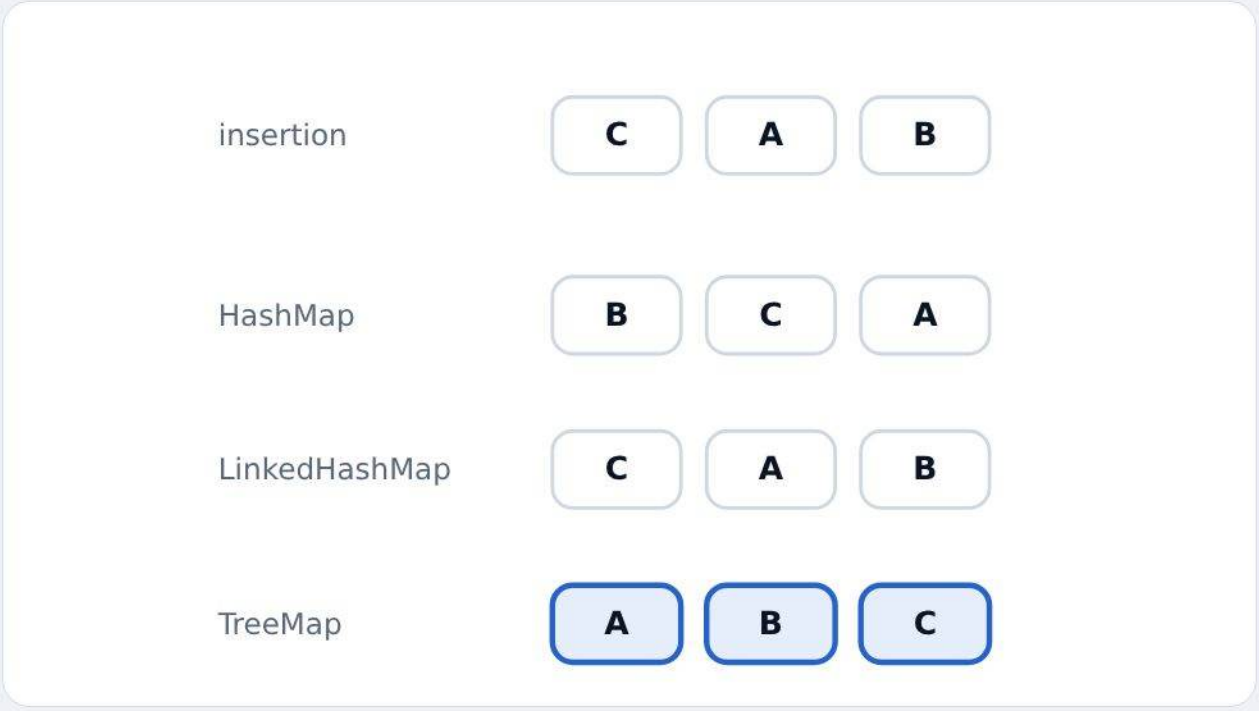


La réponse, étape par étape · *Step-by-step answer*

1. ArrayList range les éléments côte à côte, dans un tableau.
   ArrayList stores elements side by side, in an array.
2. Pour lire la case 2, Java calcule directement sa position : très rapide.
   To read slot 2, Java calculates its position directly: very fast.
3. Insérer au milieu oblige à décaler tous les éléments suivants : lent sur une grande liste.
   Inserting in the middle means shifting all the following elements: slow on a large list.
4. LinkedList : chaque élément connaît le suivant, comme les maillons d'une chaîne.
   LinkedList: each element knows the next one, like the links of a chain.
5. Pour lire l'élément 2, il faut suivre la chaîne depuis le début.
   To read element 2, you have to follow the chain from the start.
6. Ajouter en tête : on crée un maillon et on change un lien. Rien à décaler.
   Adding at the front: we create a link and change one pointer. Nothing to shift.

Les 3 Map

The 3 Maps

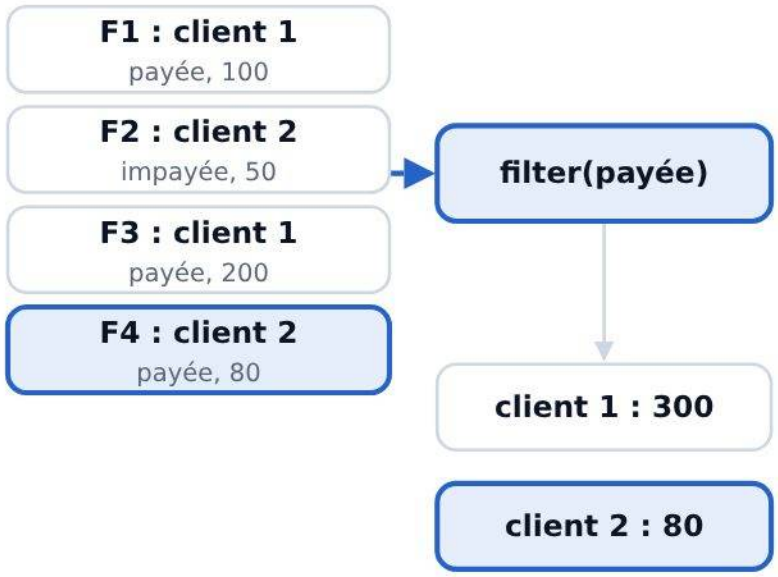


La réponse, étape par étape · *Step-by-step answer*

1. On ajoute C, puis A, puis B.
   We add C, then A, then B.
2. HashMap range selon le hashCode : l'ordre de parcours n'est pas garanti. C'est la plus rapide, O(1).
   HashMap stores by hashCode: iteration order is not guaranteed. It is the fastest, O(1).
3. LinkedHashMap garde l'ordre d'ajout : C, A, B.
   LinkedHashMap keeps insertion order: C, A, B.
4. TreeMap trie les clés : A, B, C. Un peu plus lente, O(log n).
   TreeMap sorts the keys: A, B, C. A bit slower, O(log n).

Streams

Streams

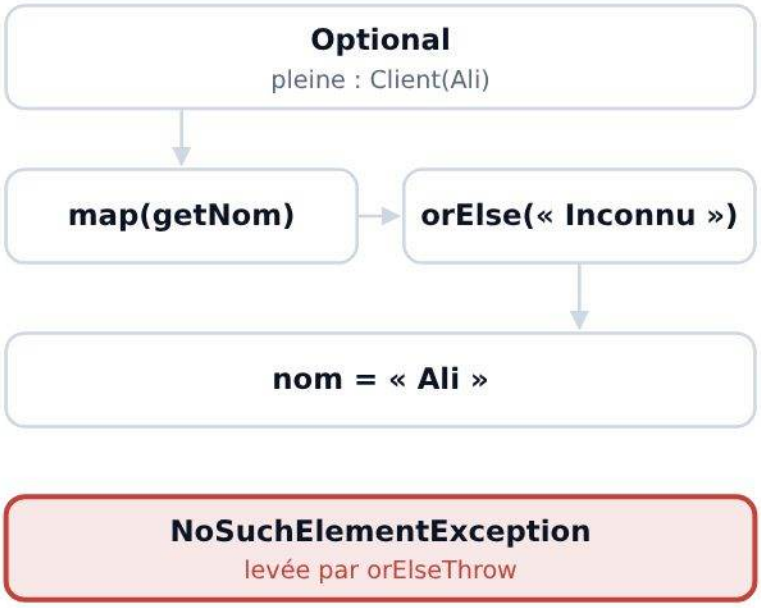


La réponse, étape par étape · *Step-by-step answer*

1. On prépare le flux des 4 factures. Rien ne s'exécute encore.
   We prepare the stream of 4 invoices. Nothing runs yet.
2. filter est déclaré, mais toujours rien : les opérations intermédiaires attendent.
   filter is declared, but still nothing: intermediate operations wait.
3. collect lance tout : F1 est payée, elle passe et s'ajoute au total du client 1.
   collect starts everything: F1 is paid, it passes and is added to customer 1's total.
4. F2 est impayée : le filtre la bloque.
   F2 is unpaid: the filter blocks it.
5. F3 est payée : le client 1 passe à 300.
   F3 is paid: customer 1 goes up to 300.
6. F4 est payée : client 2 = 80. Résultat final : une Map client → total.
   F4 is paid: customer 2 = 80. Final result: a Map customer → total.

Optional

Optional

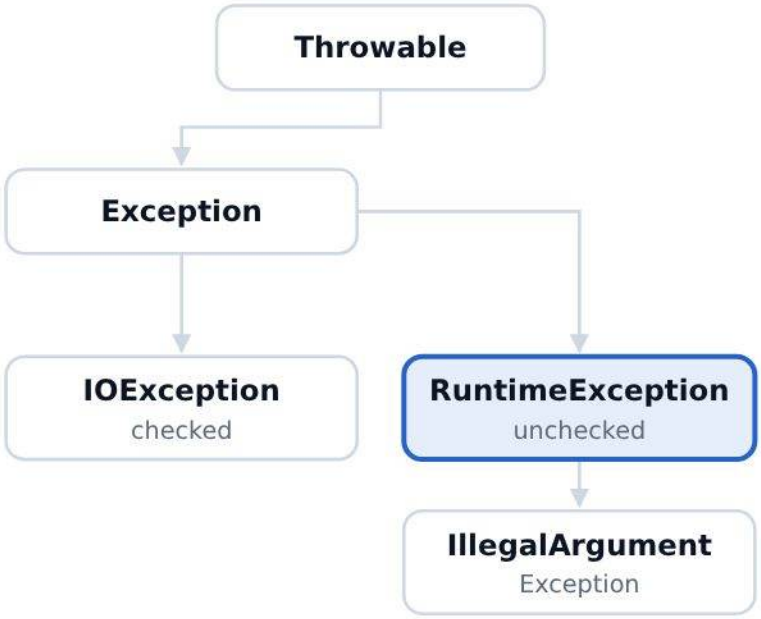


La réponse, étape par étape · *Step-by-step answer*

1. findById renvoie une boîte : pleine si le client existe, vide sinon. Jamais null.
   findById returns a box: full if the customer exists, empty otherwise. Never null.
2. La boîte est vide : map ne fait rien et transmet la boîte vide.
   The box is empty: map does nothing and passes the empty box along.
3. orElse donne une valeur par défaut quand la boîte est vide. Pas de NullPointerException.
   orElse gives a default value when the box is empty. No NullPointerException.
4. Si le client existait, map aurait sorti son nom : « Ali ».
   If the customer had existed, map would have taken out their name: "Ali".
5. orElseThrow lève une exception si la boîte est vide : utile quand l'absence est une vraie erreur.
   orElseThrow throws an exception if the box is empty: useful when absence is a real error.

Exceptions

Exceptions



La réponse, étape par étape · *Step-by-step answer*

1. IOException est checked : le compilateur t'oblige à la déclarer (throws) ou à la capturer (try/catch).
   IOException is checked: the compiler forces you to declare it (throws) or catch it (try/catch).
2. IllegalArgumentException hérite de RuntimeException : unchecked, aucune obligation. Elle signale une erreur de programmation ou de données.
   IllegalArgumentException extends RuntimeException: unchecked, no obligation. It signals a programming or data error.
3. À retenir pour Spring : @Transactional annule automatiquement (rollback) seulement sur les unchecked.
   Remember for Spring: @Transactional automatically rolls back only on unchecked exceptions.

BigDecimal

BigDecimal

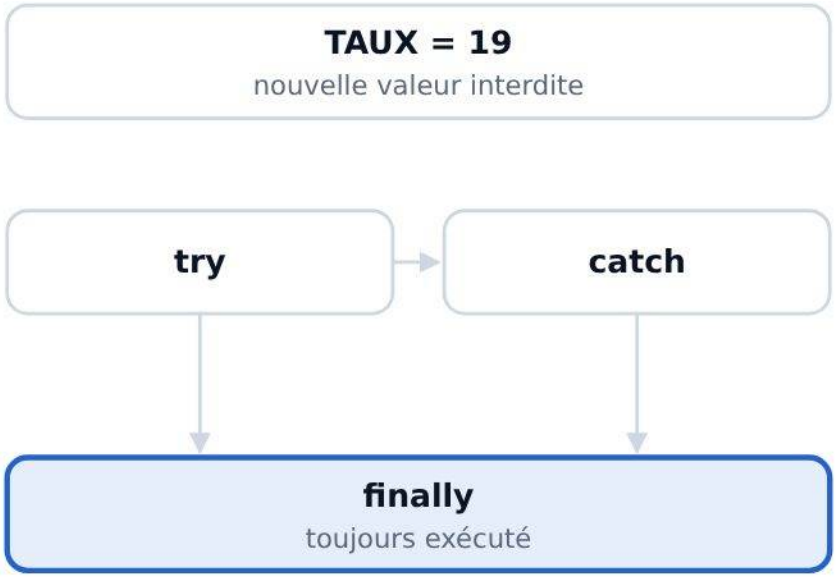


La réponse, étape par étape · *Step-by-step answer*

1. double stocke les nombres en binaire : 0,1 n’est pas exact. Sur des montants, ces petites erreurs s’accumulent.
   double stores numbers in binary: 0.1 is not exact. On amounts, these small errors add up.
2. BigDecimal calcule exactement. Astuce : le créer à partir d’un texte ("0.1"), pas d’un double.
   BigDecimal calculates exactly. Tip: create it from a string ("0.1"), not from a double.
3. On calcule la TVA à 19 %.
   We calculate VAT at 19%.
4. setScale(3, HALF_UP) arrondit au millime, comme le dinar tunisien.
   setScale(3, HALF_UP) rounds to the millime, like the Tunisian dinar.

final, finally, finalize

final, finally, finalize

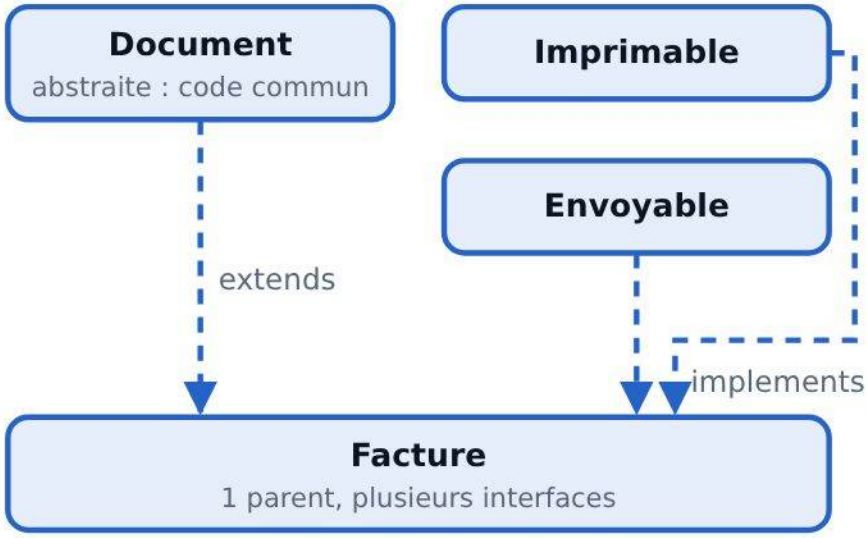


La réponse, étape par étape · *Step-by-step answer*

1. final : la variable ne peut recevoir une valeur qu'une seule fois.
   final: the variable can be assigned a value only once.
2. Essayer de la changer donne une erreur dès la compilation.
   Trying to change it gives an error at compile time.
3. Cas 1 : pas d'erreur. Le bloc finally s'exécute ensuite.
   Case 1: no error. The finally block runs afterwards.
4. Cas 2 : une exception. catch la traite, puis finally s'exécute quand même : la connexion est toujours fermée.
   Case 2: an exception. catch handles it, then finally runs anyway: the connection is always closed.
5. finalize() : vieille méthode appelée par le garbage collector, dépréciée. Aujourd'hui, on utilise try-with-resources.
   finalize(): an old method called by the garbage collector, deprecated. Today, we use try-with-resources.

Abstraite ou interface

Abstract class or interface

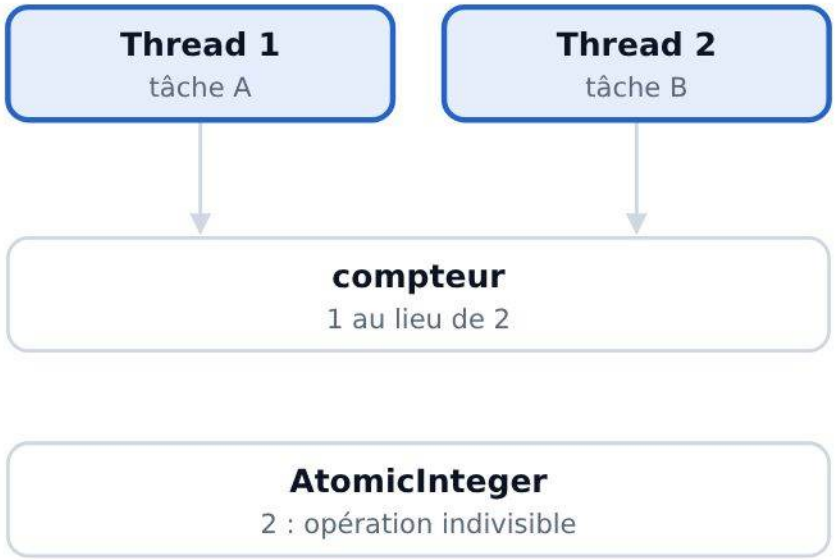


La réponse, étape par étape · *Step-by-step answer*

1. La classe abstraite partage des données (numero) et du code (totalTtc). On ne peut pas créer un Document directement.
   The abstract class shares data (numero) and code (totalTtc). You cannot create a Document directly.
2. Les interfaces décrivent des capacités : savoir s'imprimer, savoir s'envoyer.
   Interfaces describe abilities: being able to print, being able to send.
3. Facture hérite d'une seule classe, mais peut implémenter plusieurs interfaces.
   Facture extends only one class, but can implement several interfaces.

Concurrence

Concurrency

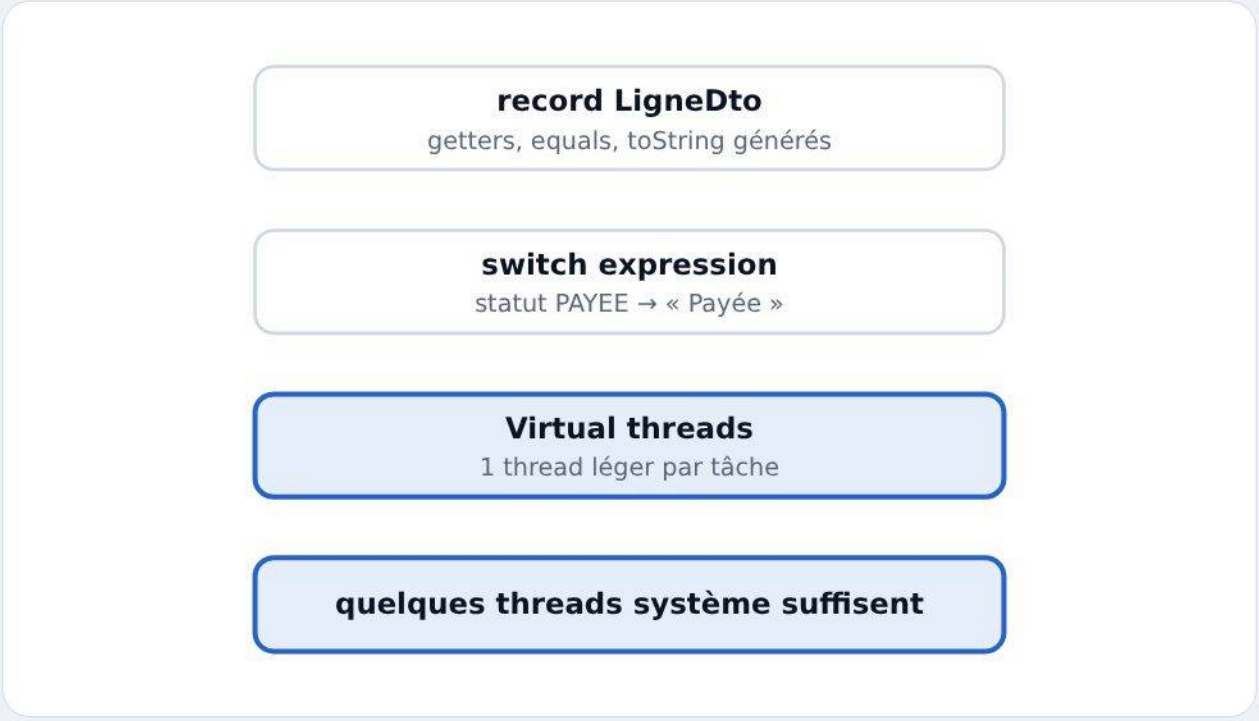


La réponse, étape par étape · *Step-by-step answer*

1. Thread 1 lit compteur : 0.
   Thread 1 reads compteur: 0.
2. Au même moment, Thread 2 lit aussi 0.
   At the same moment, Thread 2 also reads 0.
3. Les deux écrivent 1 : un ajout est perdu. C'est une « race condition ».
   Both write 1: one increment is lost. This is a "race condition".
4. AtomicInteger (ou synchronized) rend l'opération indivisible : le résultat est bien 2.
   AtomicInteger (or synchronized) makes the operation indivisible: the result is correctly 2.
5. ExecutorService réutilise un groupe de threads au lieu d'en créer un nouveau à chaque tâche.
   ExecutorService reuses a pool of threads instead of creating a new one for each task.

Java 17 et 21

Java 17 and 21

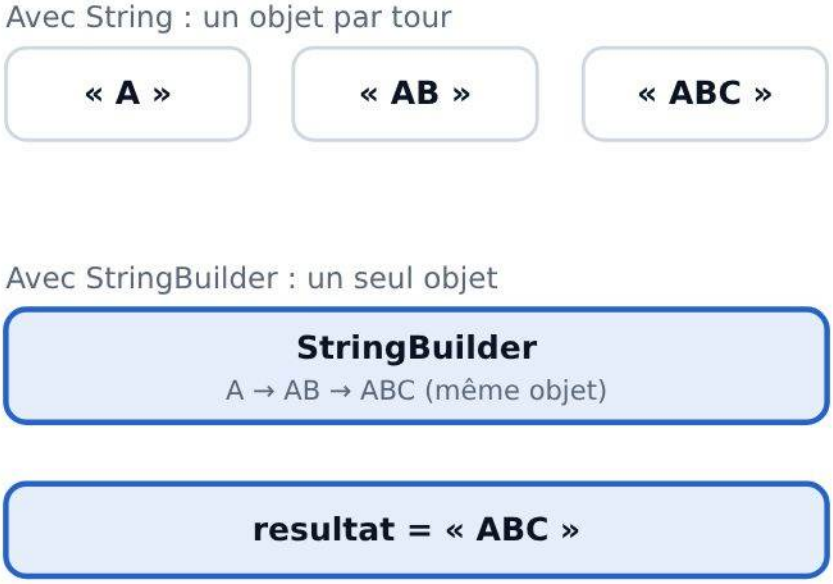


La réponse, étape par étape · *Step-by-step answer*

1. Java 17 : record écrit en une ligne une classe de données qui ne change pas.
   Java 17: record writes an immutable data class in one line.
2. switch devient une expression : il renvoie directement une valeur, sans break.
   switch becomes an expression: it returns a value directly, without break.
3. Java 21 : des milliers de threads virtuels très légers. Idéal pour beaucoup d'attentes (mails, appels HTTP).
   Java 21: thousands of very lightweight virtual threads. Ideal for lots of waiting (emails, HTTP calls).

StringBuilder

StringBuilder

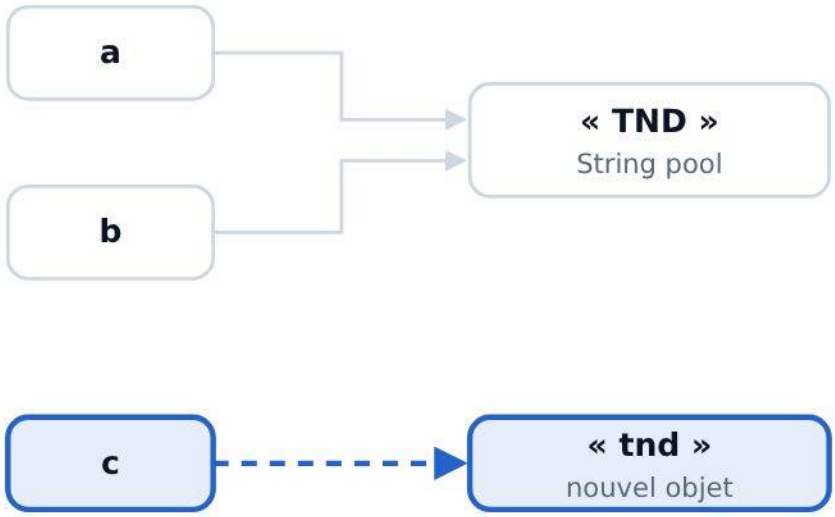


La réponse, étape par étape · *Step-by-step answer*

1. Tour 1 : la chaîne vaut « A ».
   Round 1: the string is "A".
2. Tour 2 : un nouvel objet « AB » est créé, l'ancien est jeté.
   Round 2: a new object "AB" is created, the old one is thrown away.
3. Tour 3 : encore un nouvel objet. Avec 10 000 lignes : 10 000 objets jetés.
   Round 3: yet another new object. With 10,000 lines: 10,000 objects thrown away.
4. StringBuilder modifie toujours le même objet.
   StringBuilder always modifies the same object.
5. toString() crée la chaîne finale une seule fois.
   toString() creates the final string only once.

String immuable

Immutable String

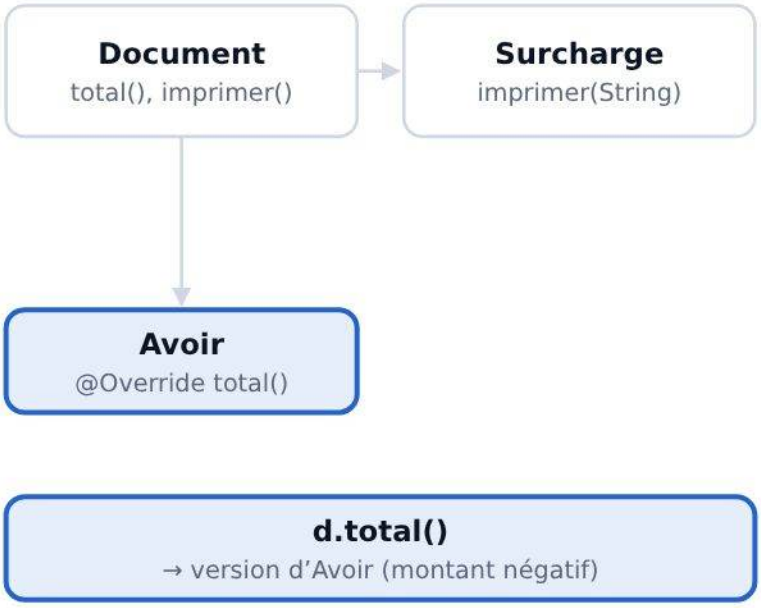


La réponse, étape par étape · *Step-by-step answer*

1. « TND » est rangé dans le String pool, une zone de chaînes partagées.
   "TND" is stored in the String pool, an area of shared strings.
2. b réutilise le même objet. C'est possible uniquement parce qu'une String ne peut pas changer.
   b reuses the same object. This is only possible because a String cannot change.
3. toLowerCase ne modifie pas « TND » : il crée un nouvel objet. a et b ne sont jamais touchés.
   toLowerCase does not modify "TND": it creates a new object. a and b are never touched.

Surcharge ou redéfinition

Overloading or overriding



La réponse, étape par étape · *Step-by-step answer*

1. Surcharge : deux méthodes imprimer avec des paramètres différents. Java choisit à la compilation selon les arguments.
   Overloading: two imprimer methods with different parameters. Java chooses at compile time based on the arguments.
2. Redéfinition : Avoir remplace total() de Document, avec la même signature.
   Overriding: Avoir replaces total() from Document, with the same signature.
3. Le choix se fait à l'exécution, selon l'objet réel : c'est le polymorphisme.
   The choice is made at runtime, based on the real object: this is polymorphism.

4 piliers POO

4 pillars of OOP

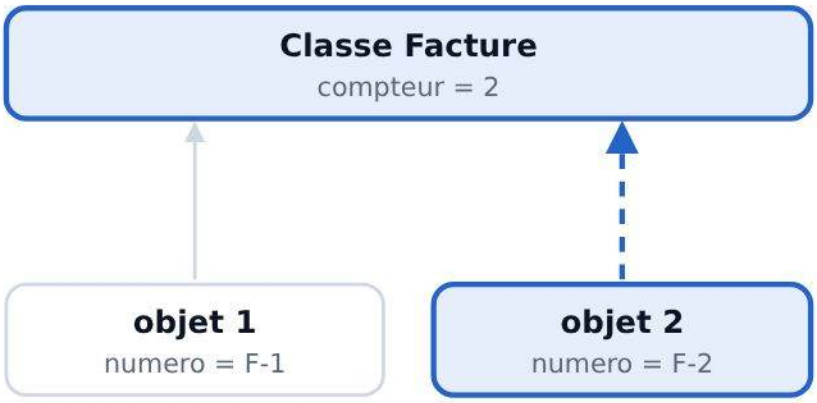


La réponse, étape par étape · *Step-by-step answer*

1. Abstraction : Document dit ce qu'il faut faire (total), sans dire comment.
   Abstraction: Document says what to do (total), without saying how.
2. Encapsulation : ht est caché (private). On y accède par getHt().
   Encapsulation: ht is hidden (private). We access it through getHt().
3. Héritage : Facture reprend tout ce que Document possède.
   Inheritance: Facture takes everything Document has.
4. Polymorphisme : d est déclaré Document, mais c'est la version de Facture qui s'exécute.
   Polymorphism: d is declared as Document, but it is Facture's version that runs.

static

static



La réponse, étape par étape · *Step-by-step answer*

1. compteur est static : il existe une seule fois, dans la classe.
   compteur is static: it exists only once, in the class.
2. Premier objet : le compteur passe à 1, l'objet reçoit F-1.
   First object: the counter goes to 1, the object gets F-1.
3. Chaque objet a son propre numero, mais tous partagent le même compteur.
   Each object has its own numero, but they all share the same counter.

Comparable / Comparator

Comparable / Comparator



La réponse, étape par étape · *Step-by-step answer*

1. Au départ, la liste n’est pas triée.
   At the start, the list is not sorted.
2. Comparable : l’ordre « naturel » est écrit dans la classe. Ici, par numéro.
   Comparable: the "natural" order is written in the class. Here, by number.
3. Comparator : un autre ordre, défini à l’extérieur. Ici, du total le plus grand au plus petit.
   Comparator: another order, defined outside. Here, from the largest total to the smallest.

Pile et tas

Stack and heap



La réponse, étape par étape · *Step-by-step answer*

1. Appel de creer() : un cadre est ajouté sur la pile.
   Call to creer(): a frame is added to the stack.
2. quantite est une variable simple : elle est rangée directement dans la pile.
   quantite is a simple variable: it is stored directly on the stack.
3. new crée l'objet dans le tas. La pile garde seulement une référence vers lui.
   new creates the object on the heap. The stack only keeps a reference to it.
4. Fin de la méthode : le cadre disparaît de la pile.
   End of the method: the frame disappears from the stack.
5. Plus personne ne pointe vers la Facture : le garbage collector libère la mémoire.
   Nothing points to the Facture anymore: the garbage collector frees the memory.

SOLID

SOLID

S : une seule responsabilité
FactureService valide, PdfService génère

O : ouvert à l'extension
nouvelle taxe = nouvelle classe

L : remplaçable par l'enfant
un Avoir peut remplacer un Document

I : petites interfaces
chacune son rôle

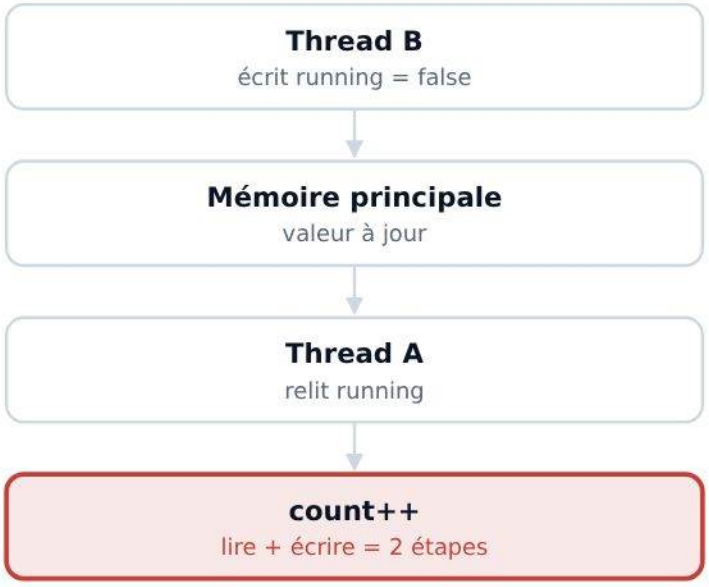
D : dépendre d'interfaces
CalculTaxe, pas TaxeTva

La réponse, étape par étape · *Step-by-step answer*

1. S : chaque classe fait une seule chose. Valider et générer un PDF sont séparés.
   S: each class does only one thing. Validating and generating a PDF are separate.
2. O : pour une nouvelle taxe, on ajoute une classe. On ne modifie pas le code qui marche.
   O: for a new tax, we add a class. We do not modify code that works.
3. L : partout où on attend un Document, un Avoir doit fonctionner aussi.
   L: wherever a Document is expected, an Avoir must work too.
4. I : plusieurs petites interfaces plutôt qu'une grosse qui oblige à tout implémenter.
   I: several small interfaces rather than one big one that forces you to implement everything.
5. D : le service reçoit une interface. Spring injecte l'implémentation : facile à changer et à tester.
   D: the service receives an interface. Spring injects the implementation: easy to change and to test.

volatile

volatile

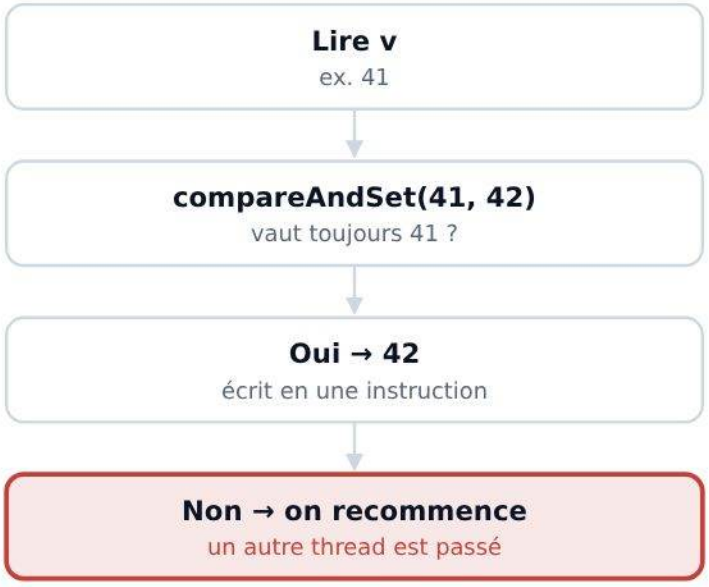


La réponse, étape par étape · *Step-by-step answer*

1. Sans volatile, chaque thread peut garder une copie de running dans le cache de son cœur.
   Without volatile, each thread may keep a copy of running in its core's cache.
2. Avec volatile, l'écriture du thread B part directement en mémoire principale.
   With volatile, thread B's write goes straight to main memory.
3. Le thread A relit la vraie valeur à chaque tour : la boucle s'arrête aussitôt.
   Thread A reads the real value on every loop: it stops right away.
4. Attention : volatile garantit la visibilité, pas l'atomicité. count++ peut perdre des incréments ; il faut AtomicInteger ou un verrou.
   Careful: volatile guarantees visibility, not atomicity. count++ can lose increments; use AtomicInteger or a lock.

AtomicInteger

AtomicInteger

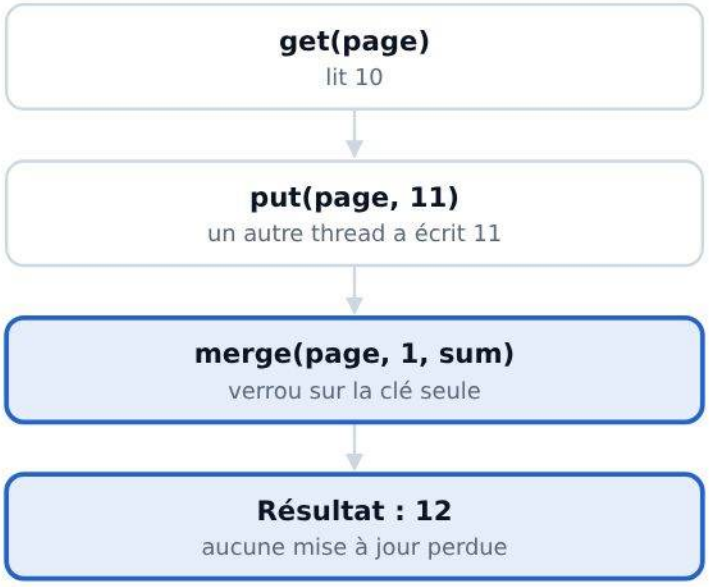


La réponse, étape par étape · *Step-by-step answer*

1. Un int partagé incrémenté par plusieurs threads perd des valeurs : deux threads lisent 41 et écrivent tous les deux 42.
   A shared int incremented by several threads loses values: two threads read 41 and both write 42.
2. AtomicInteger lit la valeur puis demande au processeur : « remplace 41 par 42 seulement si c'est encore 41 ».
   AtomicInteger reads the value then asks the CPU: "replace 41 with 42 only if it is still 41".
3. Si personne n'a touché la valeur, l'écriture réussit : c'est le compare-and-swap (CAS), sans verrou.
   If nobody touched it, the write succeeds: this is compare-and-swap (CAS), with no lock.
4. Sinon on relit et on réessaie. Résultat : 100 incréments = +100, sans synchronized.
   Otherwise it re-reads and retries. Result: 100 increments = +100, without synchronized.

ConcurrentHashMap

ConcurrentHashMap

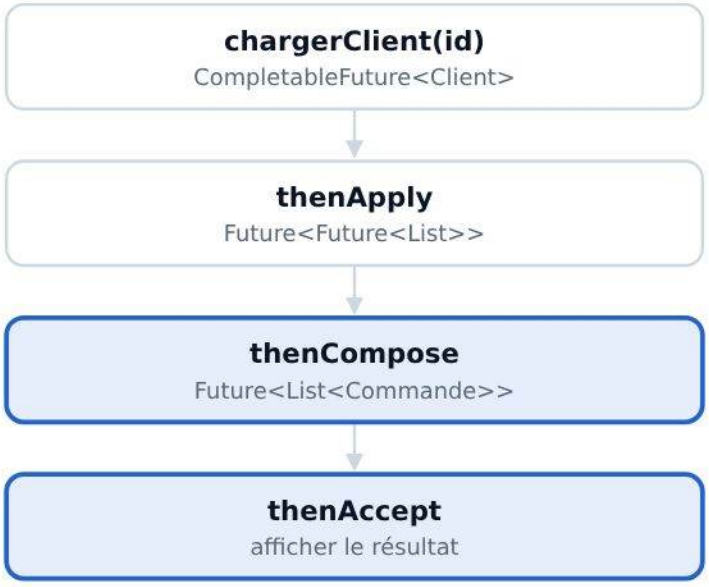


La réponse, étape par étape · *Step-by-step answer*

1. ConcurrentHashMap protège chaque opération, mais pas l'enchaînement de deux opérations.
   ConcurrentHashMap protects each operation, but not a sequence of two operations.
2. get puis put : deux threads lisent 10 et écrivent 11. Une vue est perdue.
   get then put: two threads read 10 and write 11. One view is lost.
3. merge (ou compute) fait lecture et écriture d'un seul bloc, en verrouillant seulement cette clé.
   merge (or compute) reads and writes in one block, locking only that key.
4. Les deux incréments sont comptés, et les autres clés restent accessibles en parallèle.
   Both increments are counted, and other keys stay accessible in parallel.

CompletableFuture

CompletableFuture



La réponse, étape par étape · *Step-by-step answer*

1. chargerClient renvoie un CompletableFuture : le client arrivera plus tard, sans bloquer.
   loadClient returns a CompletableFuture: the client will arrive later, without blocking.
2. chargerCommandes renvoie lui aussi un future. Avec thenApply, on obtient un future imbriqué, pénible à utiliser.
   loadOrders also returns a future. With thenApply you get a nested future, awkward to use.
3. thenCompose enchaîne le deuxième appel asynchrone et aplatit le résultat, comme flatMap.
   thenCompose chains the second async call and flattens the result, like flatMap.
4. On continue la chaîne normalement. Règle : la fonction renvoie un future → thenCompose ; une valeur simple → thenApply.
   The chain continues normally. Rule: the function returns a future → thenCompose; a plain value → thenApply.

Garbage collector

Garbage collector

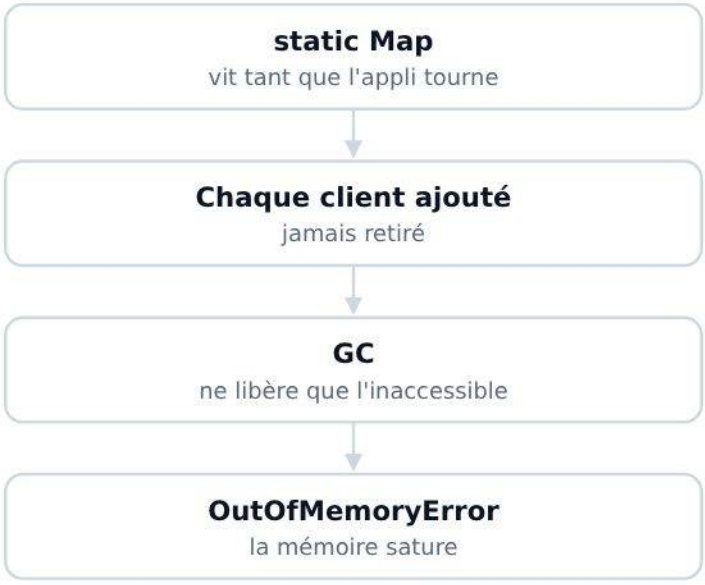


La réponse, étape par étape · *Step-by-step answer*

1. G1 découpe le tas en petites régions au lieu de grandes zones fixes.
   G1 splits the heap into small regions instead of large fixed areas.
2. La plupart des objets meurent jeunes : G1 nettoie souvent l'Eden, c'est rapide.
   Most objects die young: G1 often cleans Eden, which is fast.
3. Pour la vieille génération, il commence par les régions qui contiennent le plus de déchets (d'où « Garbage First »).
   For the old generation it starts with the regions holding the most garbage (hence "Garbage First").
4. Objectif : des pauses courtes et prévisibles. C'est le GC par défaut depuis Java 9 ; ZGC existe pour des pauses encore plus courtes.
   Goal: short, predictable pauses. It is the default since Java 9; ZGC exists for even shorter pauses.

Fuite mémoire

Memory leak

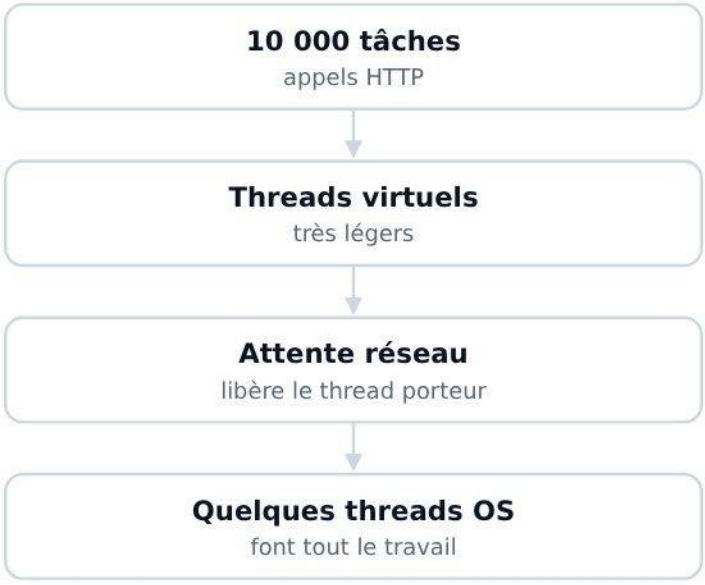


La réponse, étape par étape · *Step-by-step answer*

1. Une collection static reste accessible pendant toute la vie de l'application.
   A static collection stays reachable for the application's whole life.
2. Chaque appel ajoute un client, et rien ne les retire : la map grossit sans fin.
   Each call adds a client and nothing removes them: the map grows forever.
3. Le GC ne libère que les objets inaccessibles. Ici, tous restent référencés : la mémoire monte jusqu'au OutOfMemoryError.
   The GC only frees unreachable objects. Here all stay referenced: memory climbs until OutOfMemoryError.
4. Solutions : un cache borné avec expiration (Caffeine), retirer les listeners, et analyser un heap dump pour trouver le coupable.
   Fixes: a bounded cache with expiry (Caffeine), remove listeners, and analyse a heap dump to find the culprit.

Threads virtuels

Virtual threads

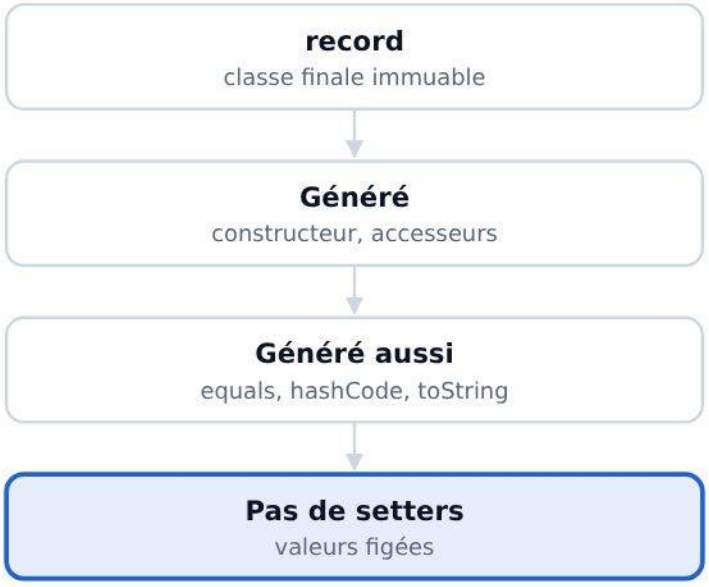


La réponse, étape par étape · *Step-by-step answer*

1. Un thread classique coûte environ 1 Mo : 10 000 appels HTTP en parallèle épuisent la machine.
   A classic thread costs about 1 MB: 10,000 parallel HTTP calls exhaust the machine.
2. Java 21 crée un thread virtuel par tâche : quelques Ko chacun, on peut en avoir des millions.
   Java 21 creates one virtual thread per task: a few KB each, you can have millions.
3. Quand une tâche attend le réseau, son thread virtuel se met de côté et libère le thread système pour une autre.
   When a task waits on the network, its virtual thread steps aside and frees the OS thread for another one.
4. Gros gain pour les E/S (HTTP, base de données). Pour du calcul pur, aucun gain : le nombre de cœurs reste la limite.
   Big win for I/O (HTTP, database). For pure computation, no gain: the number of cores is still the limit.

Records

Records

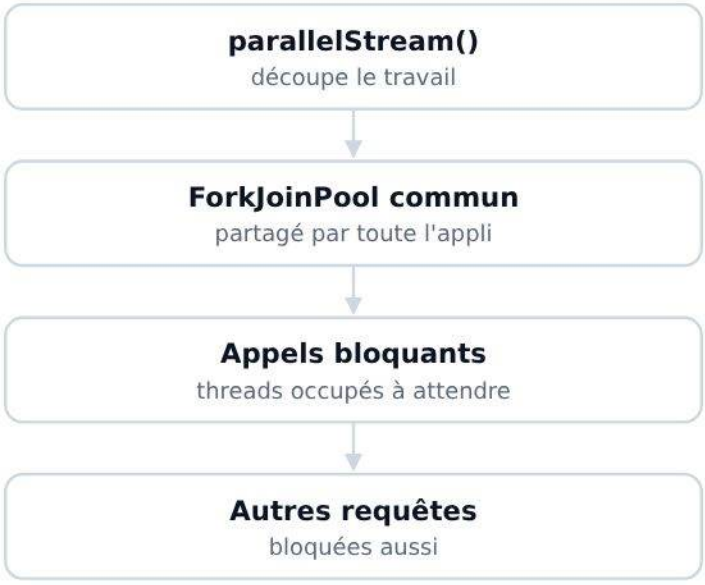


La réponse, étape par étape · *Step-by-step answer*

1. Un record déclare ses champs dans l'en-tête : ils deviennent private final.
   A record declares its fields in the header: they become private final.
2. Java génère le constructeur et les accesseurs valeur() et devise().
   Java generates the constructor and the accessors valeur() and devise().
3. equals, hashCode et toString sont générés à partir des champs : idéal pour des DTO et des valeurs.
   equals, hashCode and toString are generated from the fields: ideal for DTOs and value objects.
4. Pas de setters : pour valider, on utilise le constructeur compact. Un record ne convient pas comme entité JPA (il faut un objet modifiable).
   No setters: validate in the compact constructor. A record is not suited as a JPA entity (which must be mutable).

Streams parallèles

Parallel streams

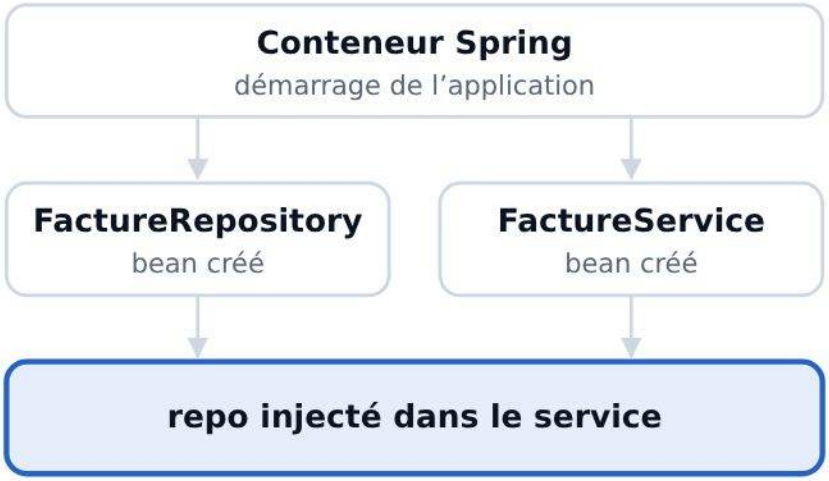


La réponse, étape par étape · *Step-by-step answer*

1. parallelStream découpe la liste et traite les morceaux en parallèle.
   parallelStream splits the list and processes the chunks in parallel.
2. Mais il utilise le ForkJoinPool commun : peu de threads, partagés par tout le code de l'application.
   But it uses the common ForkJoinPool: few threads, shared by all the application's code.
3. Des appels base de données bloquent ces threads : les autres parallelStream de l'appli attendent aussi.
   Database calls block those threads: every other parallel stream in the app waits too.
4. À réserver au calcul pur sur de gros volumes. Pour des E/S : stream() simple, un executor dédié ou des threads virtuels.
   Keep it for pure computation on large data. For I/O: plain stream(), a dedicated executor or virtual threads.

Injection

Injection

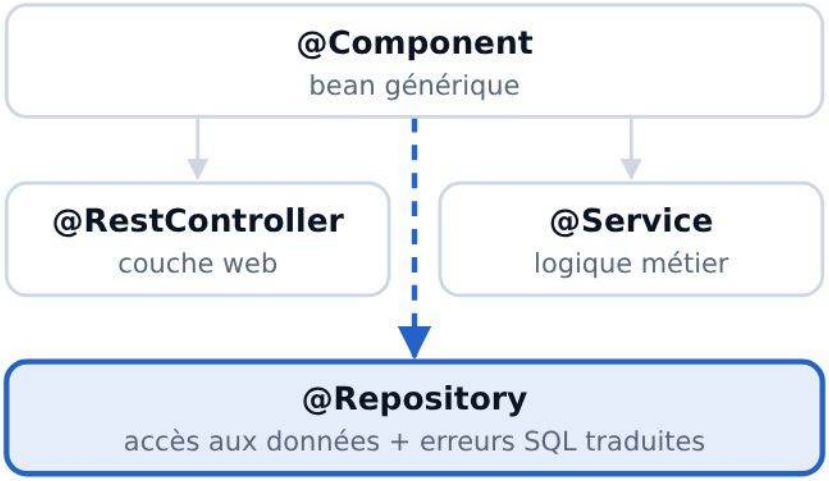


La réponse, étape par étape · *Step-by-step answer*

1. Au démarrage, Spring cherche les classes annotées (@Service, @Repository…).
   At startup, Spring looks for annotated classes (@Service, @Repository…).
2. Il crée d'abord le repository, car le service en a besoin.
   It creates the repository first, because the service needs it.
3. Puis il crée le service.
   Then it creates the service.
4. Il passe le repository au constructeur du service : c'est l'injection. La classe ne fait jamais new elle-même.
   It passes the repository to the service's constructor: this is injection. The class never calls new itself.
5. Avantage : dans un test, on passe un faux repository au constructeur, sans base de données.
   Advantage: in a test, we pass a fake repository to the constructor, with no database.

Les annotations

Annotations



La réponse, étape par étape · *Step-by-step answer*

1. @Component dit à Spring : « crée et gère cet objet ». Les autres annotations en sont des variantes.
   @Component tells Spring: "create and manage this object". The other annotations are variants of it.
2. @RestController : reçoit les requêtes HTTP et renvoie du JSON.
   @RestController: receives HTTP requests and returns JSON.
3. @Service : la logique métier (calculer, valider). Même effet que @Component, mais le rôle est clair.
   @Service: business logic (calculating, validating). Same effect as @Component, but the role is clear.
4. @Repository : l'accès à la base. Bonus : les erreurs SQL deviennent des exceptions Spring.
   @Repository: database access. Bonus: SQL errors become Spring exceptions.

Scopes

Scopes

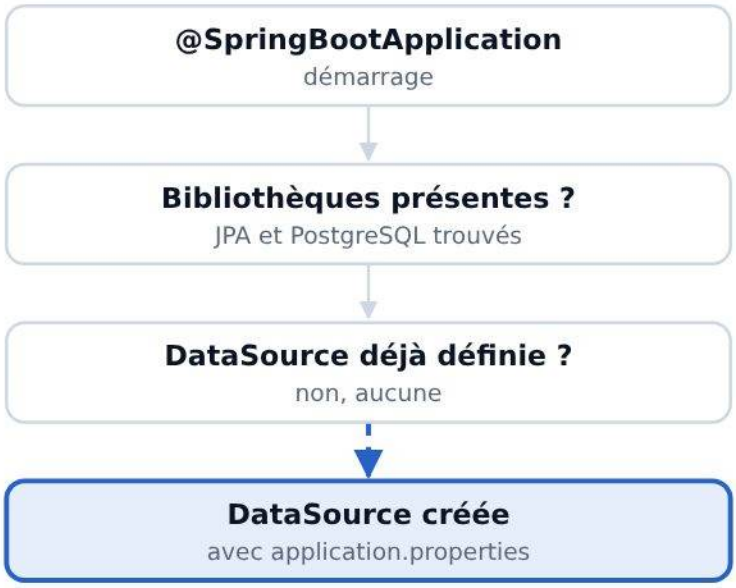


La réponse, étape par étape · *Step-by-step answer*

1. Singleton : Spring crée un seul TauxService et le donne à tout le monde.
   Singleton: Spring creates a single TauxService and gives it to everyone.
2. Prototype : chaque demande reçoit un objet tout neuf.
   Prototype: each request gets a brand-new object.
3. Piège : un singleton est partagé. Ne jamais y garder une donnée propre à une requête ou à un utilisateur.
   Trap: a singleton is shared. Never store data specific to a request or a user in it.

Auto-configuration

Auto-configuration

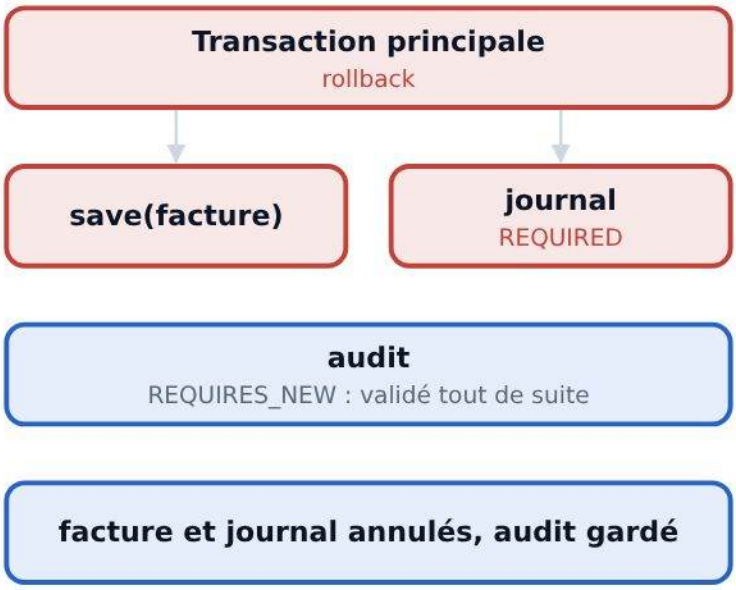


La réponse, étape par étape · *Step-by-step answer*

1. @SpringBootApplication active l'auto-configuration.
   @SpringBootApplication enables auto-configuration.
2. Spring Boot regarde les bibliothèques ajoutées : JPA et le driver PostgreSQL sont là.
   Spring Boot looks at the libraries you added: JPA and the PostgreSQL driver are there.
3. Il vérifie que tu n'as pas déjà configuré ta propre connexion.
   It checks that you have not already configured your own connection.
4. Alors il crée la connexion avec l'URL du fichier de config. Zéro code à écrire.
   So it creates the connection with the URL from the config file. Zero code to write.

Propagation

Propagation

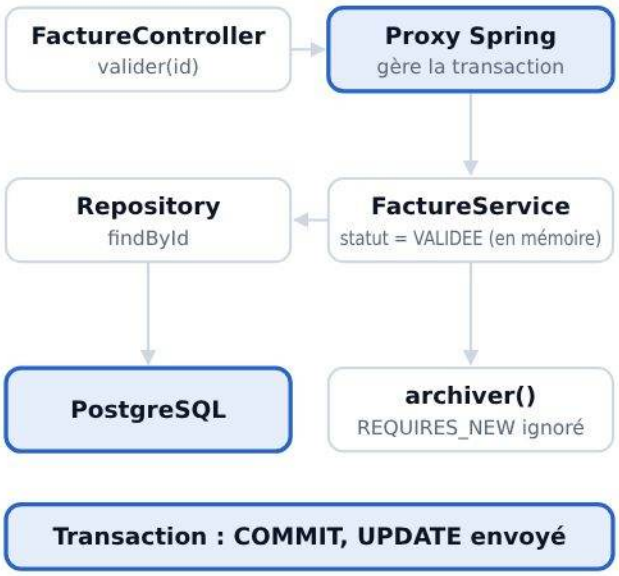


La réponse, étape par étape · *Step-by-step answer*

1. Le proxy ouvre une transaction avant la méthode.
   The proxy opens a transaction before the method.
2. La facture est enregistrée dans cette transaction.
   The invoice is saved in this transaction.
3. REQUIRED (le choix par défaut) : le journal rejoint la même transaction.
   REQUIRED (the default): the log joins the same transaction.
4. REQUIRES_NEW : l'audit ouvre sa propre transaction, validée à part.
   REQUIRES_NEW: the audit opens its own transaction, committed separately.
5. Une erreur arrive : tout ce qui est dans la transaction principale est annulé. L'audit, séparé, reste.
   An error occurs: everything in the main transaction is cancelled. The audit, being separate, stays.

@Transactional

@Transactional

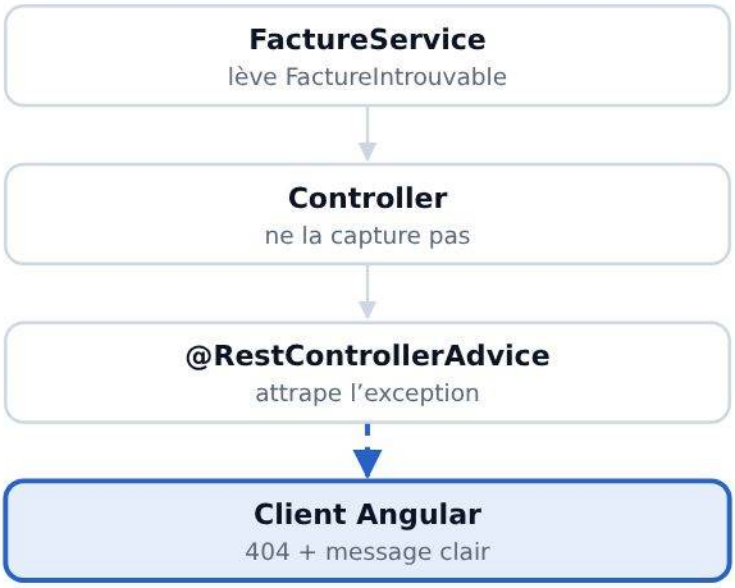


La réponse, étape par étape · *Step-by-step answer*

1. Le contrôleur pense appeler le service. En vrai, il appelle un intermédiaire créé par Spring : le proxy.
   The controller thinks it is calling the service. In reality, it calls a middleman created by Spring: the proxy.
2. Le proxy voit @Transactional. Il ouvre une transaction : tout ce qui suit sera validé ou annulé en un seul bloc.
   The proxy sees @Transactional. It opens a transaction: everything that follows will be committed or cancelled as one block.
3. Le proxy passe ensuite la main au vrai service.
   The proxy then hands over to the real service.
4. Le service lit la facture dans la base de données.
   The service reads the invoice from the database.
5. On change le statut. Rien n'est encore écrit en base : Hibernate note juste le changement.
   We change the status. Nothing is written to the database yet: Hibernate just notes the change.
6. Piège : this.archiver() appelle la méthode directement, sans passer par le proxy. Son @Transactional est donc ignoré.
   Trap: this.archiver() calls the method directly, without going through the proxy. So its @Transactional is ignored.
7. Tout s'est bien passé : le proxy valide (commit) et la modification est écrite en base. En cas d'erreur, tout aurait été annulé (rollback).
   Everything went well: the proxy commits and the change is written to the database. If there had been an error, everything would have been cancelled (rollback).

Erreurs globales

Global error handling

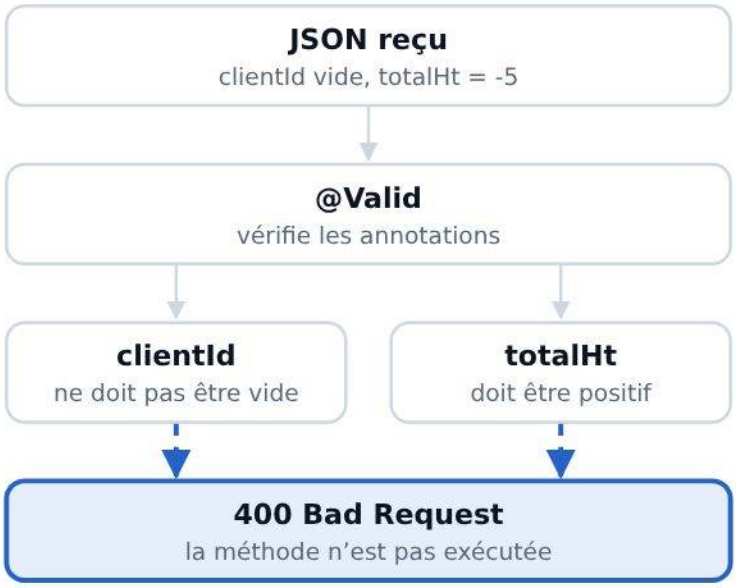


La réponse, étape par étape · *Step-by-step answer*

1. Le service ne trouve pas la facture 42 : il lève une exception.
   The service cannot find invoice 42: it throws an exception.
2. L'exception remonte à travers le contrôleur. Pas besoin de try/catch partout.
   The exception travels up through the controller. No need for try/catch everywhere.
3. Le @RestControllerAdvice l'attrape, comme un filet de sécurité commun à toute l'API.
   The @RestControllerAdvice catches it, like a safety net shared by the whole API.
4. Il renvoie un code 404 et un message JSON propre, toujours au même format.
   It returns a 404 code and a clean JSON message, always in the same format.

Validation

Validation

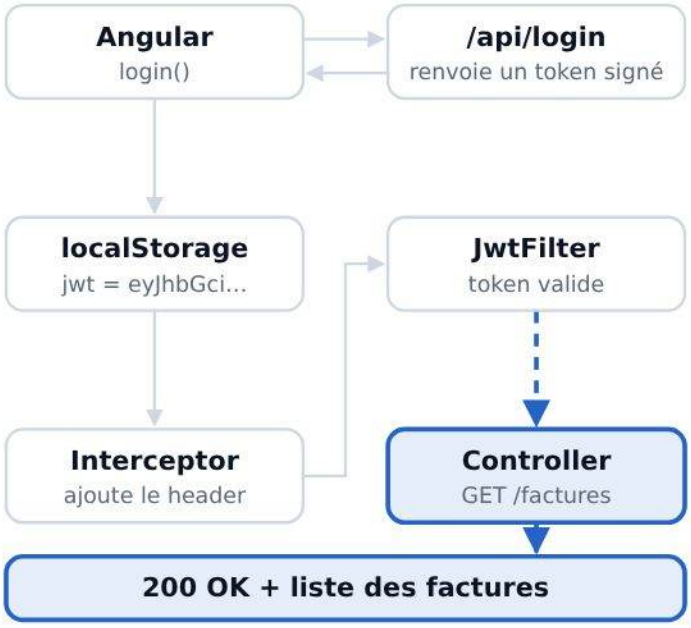


La réponse, étape par étape · *Step-by-step answer*

1. Le front envoie un JSON incorrect.
   The front end sends invalid JSON.
2. @Valid demande à Spring de vérifier les règles du DTO avant d'entrer dans la méthode.
   @Valid asks Spring to check the DTO's rules before entering the method.
3. Deux règles ne sont pas respectées.
   Two rules are not respected.
4. Réponse 400 avec la liste des erreurs. Le code de la méthode n'a jamais été exécuté.
   Response 400 with the list of errors. The method's code was never executed.

JWT

JWT

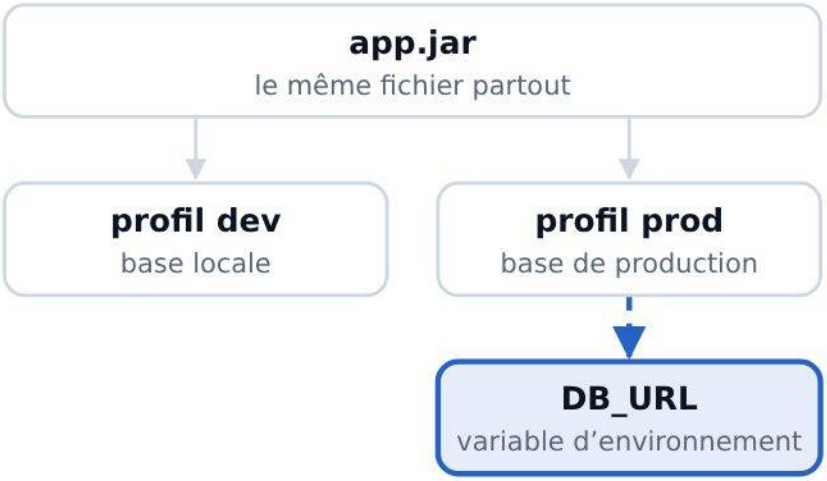


La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur se connecte : Angular envoie son identifiant et son mot de passe au serveur.
   The user logs in: Angular sends their username and password to the server.
2. Le serveur vérifie. Si c'est bon, il renvoie un token : une sorte de badge d'accès signé.
   The server checks. If it is correct, it sends back a token: a kind of signed access badge.
3. Angular garde ce badge de côté, dans localStorage.
   Angular keeps this badge aside, in localStorage.
4. À chaque nouvelle demande, l'interceptor colle automatiquement le badge sur la requête.
   With every new request, the interceptor automatically sticks the badge onto the request.
5. La requête arrive au serveur. Le JwtFilter, comme un vigile, l'arrête à l'entrée.
   The request reaches the server. The JwtFilter, like a security guard, stops it at the door.
6. Le vigile vérifie le badge : est-il vrai ? Est-il encore valable ? Si oui, il laisse entrer.
   The guard checks the badge: is it real? Is it still valid? If so, it lets the request in.
7. Le contrôleur répond avec les factures. Badge faux ou expiré : erreur 401, accès refusé.
   The controller responds with the invoices. Fake or expired badge: error 401, access denied.

Profils

Profiles

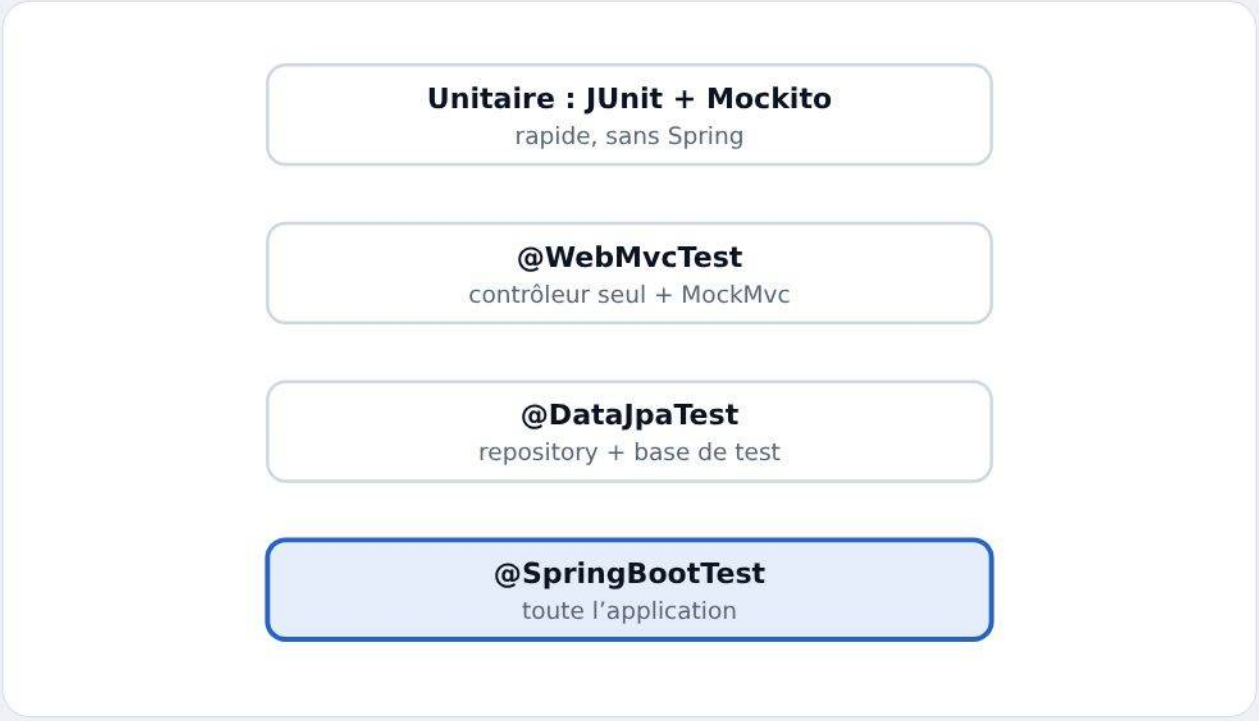


La réponse, étape par étape · *Step-by-step answer*

1. On construit une seule application.
   We build a single application.
2. En développement, le profil dev pointe vers la base locale.
   In development, the dev profile points to the local database.
3. En production, on active le profil prod au lancement.
   In production, we activate the prod profile at launch.
4. L'URL et le mot de passe viennent de variables d'environnement, jamais du code.
   The URL and password come from environment variables, never from the code.

Tests

Tests

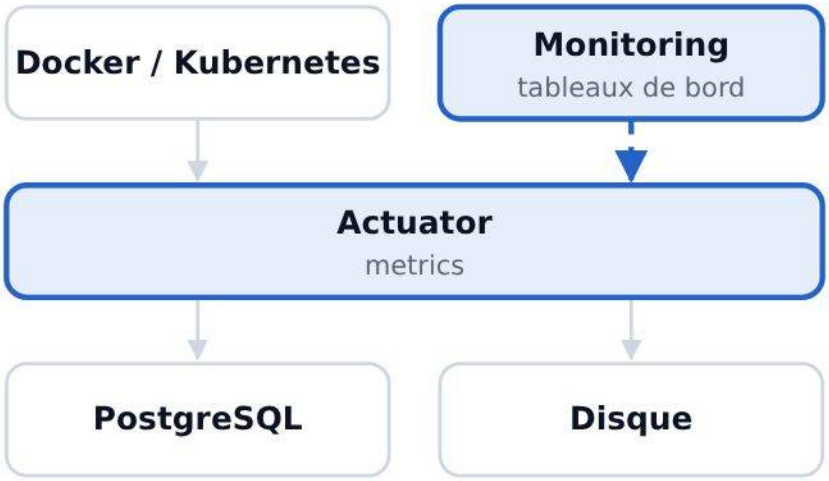


La réponse, étape par étape · *Step-by-step answer*

1. Test unitaire : on teste le service seul, le repository est remplacé par un faux (mock). Très rapide, on en écrit beaucoup.
   Unit test: we test the service alone, the repository is replaced by a fake (mock). Very fast, we write many of them.
2. On teste le contrôleur seul : URL, JSON, codes HTTP. Le service est remplacé par un faux.
   We test the controller alone: URL, JSON, HTTP codes. The service is replaced by a fake.
3. On teste les requêtes du repository sur une vraie base de test.
   We test the repository's queries on a real test database.
4. Test d'intégration : toute l'application démarre, avec un vrai PostgreSQL dans Docker. Plus lent, on en écrit moins.
   Integration test: the whole application starts, with a real PostgreSQL in Docker. Slower, we write fewer of them.

Actuator

Actuator



La réponse, étape par étape · *Step-by-step answer*

1. On ajoute Actuator : l'application expose des adresses de surveillance.
   We add Actuator: the application exposes monitoring endpoints.
2. /health vérifie la base, le disque… et répond UP ou DOWN. Si DOWN, l'app peut être redémarrée automatiquement.
   /health checks the database, the disk… and answers UP or DOWN. If DOWN, the app can be restarted automatically.
3. /metrics donne des chiffres : nombre de requêtes, temps de réponse, mémoire.
   /metrics gives numbers: number of requests, response time, memory.

Champ ou constructeur

Field or constructor

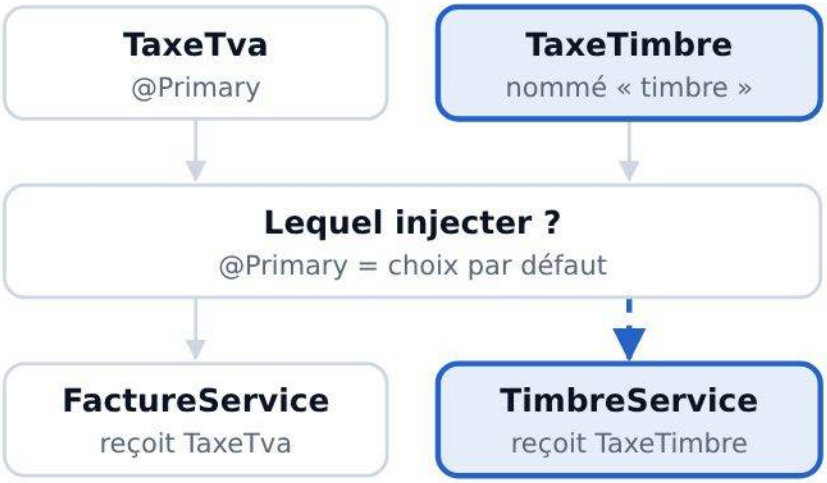


La réponse, étape par étape · *Step-by-step answer*

1. Par champ : Spring remplit le champ en cachette. Impossible de le rendre final.
   By field: Spring fills the field behind the scenes. Impossible to make it final.
2. Par constructeur : la dépendance est obligatoire, final, et visible dans la signature.
   By constructor: the dependency is mandatory, final, and visible in the signature.
3. Pour tester, on passe un faux repository au constructeur. Avec un champ, il faudrait démarrer Spring.
   To test, we pass a fake repository to the constructor. With a field, we would have to start Spring.

@Primary / @Qualifier

@Primary / @Qualifier

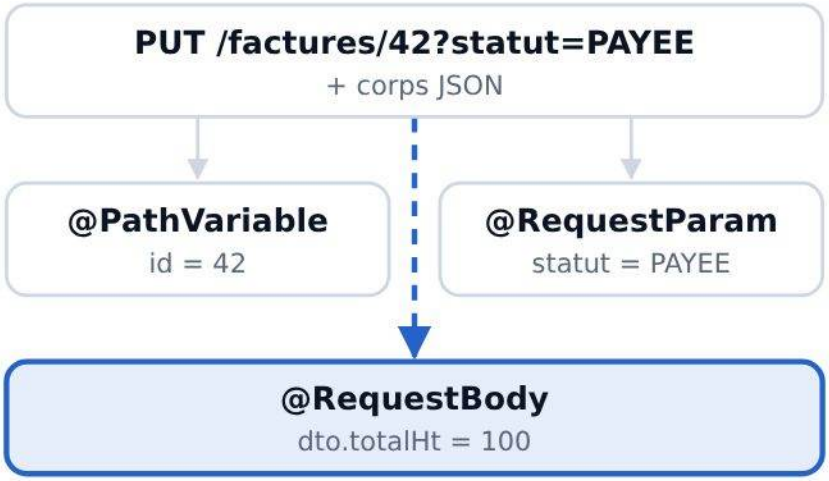


La réponse, étape par étape · *Step-by-step answer*

1. Deux beans implémentent CalculTaxe : sans indication, Spring ne sait pas lequel choisir.
   Two beans implement CalculTaxe: without a hint, Spring does not know which one to choose.
2. @Primary désigne le choix par défaut.
   @Primary marks the default choice.
3. @Qualifier demande un bean précis, par son nom.
   @Qualifier asks for a specific bean, by its name.

Lire la requête

Reading the request

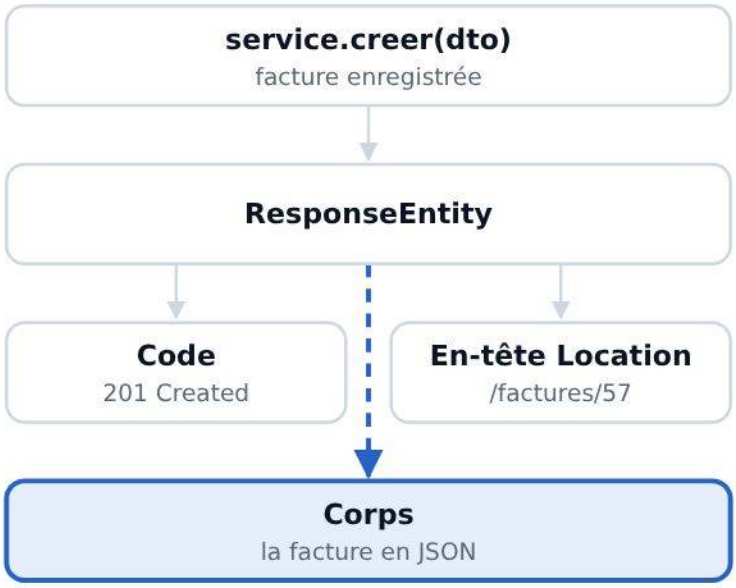


La réponse, étape par étape · *Step-by-step answer*

1. Une requête arrive avec trois informations à trois endroits différents.
   A request arrives with three pieces of information in three different places.
2. @PathVariable lit un morceau de l'adresse : l'identifiant de la ressource.
   @PathVariable reads a piece of the URL: the resource's identifier.
3. @RequestParam lit ce qui suit le « ? » : un filtre ou une option.
   @RequestParam reads what comes after the "?": a filter or an option.
4. @RequestBody transforme le JSON du corps en objet Java.
   @RequestBody turns the JSON body into a Java object.

ResponseEntity

ResponseEntity

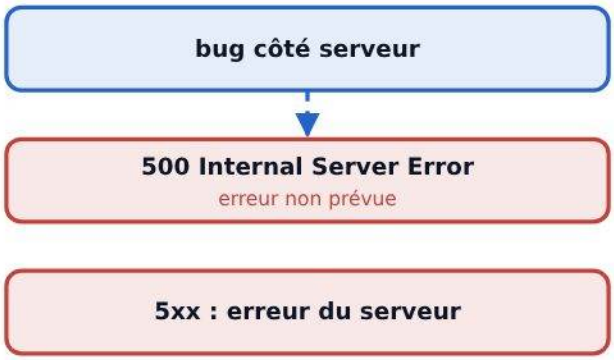


La réponse, étape par étape · *Step-by-step answer*

1. Le service crée la facture.
   The service creates the invoice.
2. ResponseEntity permet de choisir le code : 201 = ressource créée.
   ResponseEntity lets you choose the code: 201 = resource created.
3. On ajoute l'adresse de la nouvelle facture dans l'en-tête Location.
   We add the new invoice's address in the Location header.
4. Et on renvoie la facture créée dans le corps.
   And we return the created invoice in the body.

Codes HTTP

HTTP codes

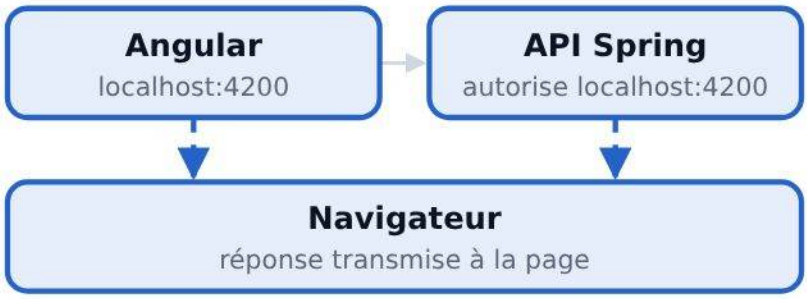


La réponse, étape par étape · *Step-by-step answer*

1. 200 : tout s'est bien passé.
   200: everything went well.
2. 201 : une ressource a été créée.
   201: a resource was created.
3. 204 : action réussie, pas de contenu à renvoyer.
   204: action succeeded, no content to return.
4. 400 : la requête est mal formée ou invalide.
   400: the request is malformed or invalid.
5. 401 : l'utilisateur n'est pas identifié.
   401: the user is not identified.
6. 403 : identifié, mais pas autorisé. Ne pas confondre avec 401.
   403: identified, but not allowed. Do not confuse with 401.
7. 404 : la ressource n'existe pas.
   404: the resource does not exist.
8. 500 : le problème vient du serveur, pas du client.
   500: the problem comes from the server, not the client.

CORS

CORS

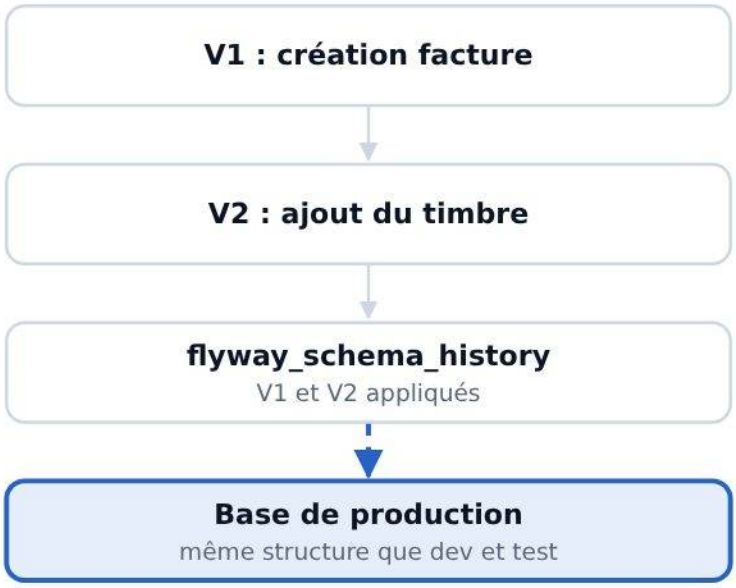


La réponse, étape par étape · *Step-by-step answer*

1. La page Angular appelle une API sur un autre port : c’est une autre « origine ».
   The Angular page calls an API on another port: it is another "origin".
2. Sans autorisation de l’API, le navigateur bloque la réponse. C’est l’erreur CORS dans la console.
   Without permission from the API, the browser blocks the response. That is the CORS error in the console.
3. L’API déclare l’origine autorisée : le navigateur laisse passer la réponse.
   The API declares the allowed origin: the browser lets the response through.

Flyway

Flyway



La réponse, étape par étape · *Step-by-step answer*

1. Chaque changement de base est un script numéroté, rangé dans Git.
   Each database change is a numbered script, stored in Git.
2. Pour ajouter le timbre fiscal, on écrit un nouveau script V2. On ne modifie jamais V1.
   To add the timbre fiscal, we write a new V2 script. We never modify V1.
3. Au démarrage, Flyway regarde l'historique et n'applique que les scripts manquants.
   At startup, Flyway looks at the history and applies only the missing scripts.
4. Résultat : tous les environnements ont exactement la même structure.
   Result: all environments have exactly the same structure.

Logs

Logs

Jamais de mot de passe ni de token

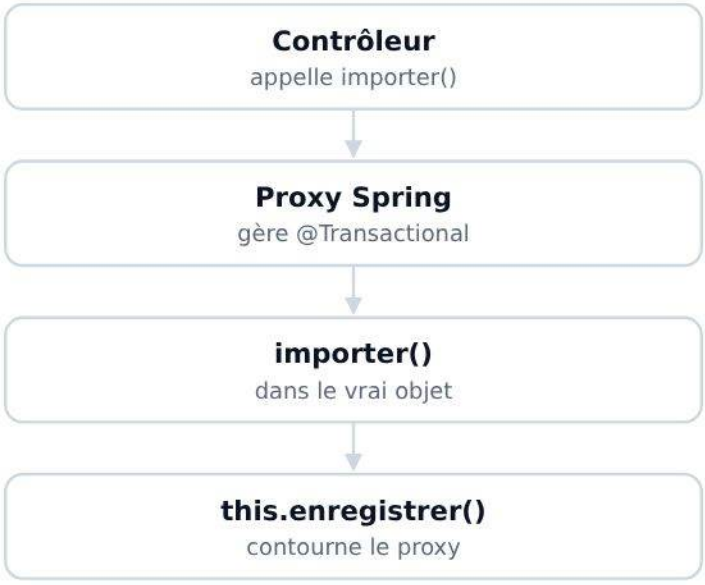
DEBUG pas affiché en prod	**INFO** affiché
WARN affiché	**ERROR** affiché + erreur

La réponse, étape par étape · *Step-by-step answer*

1. DEBUG : détails pour le développement. En production (niveau INFO), ils sont masqués.
   DEBUG: details for development. In production (INFO level), they are hidden.
2. INFO : les événements normaux importants.
   INFO: important normal events.
3. WARN : quelque chose d'anormal, mais l'application continue.
   WARN: something abnormal, but the application keeps running.
4. ERROR : un vrai problème. On ajoute l'exception pour avoir la cause.
   ERROR: a real problem. We add the exception to get the cause.
5. Les logs sont lus par beaucoup de monde : on n'y écrit jamais de données secrètes.
   Logs are read by many people: we never write secret data in them.

Appel interne @Transactional

@Transactional self-call

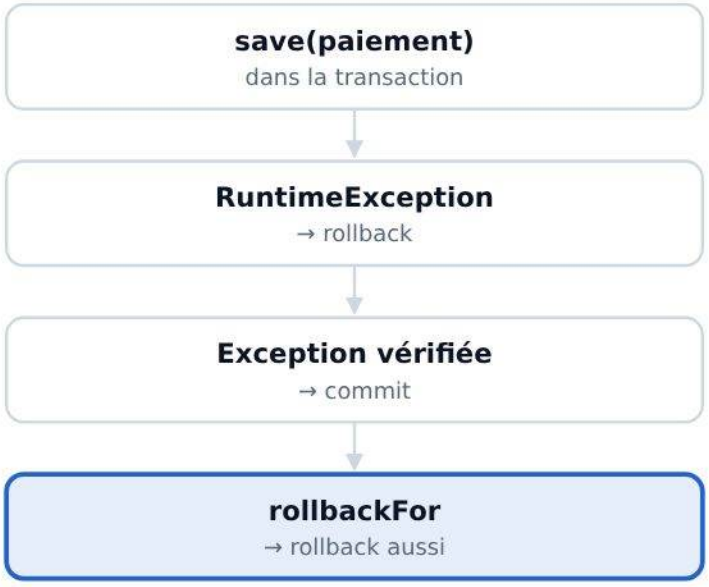


La réponse, étape par étape · *Step-by-step answer*

1. Spring entoure le bean d'un proxy : c'est lui qui ouvre et valide la transaction.
   Spring wraps the bean in a proxy: it opens and commits the transaction.
2. Le contrôleur appelle importer() via le proxy, puis on entre dans le vrai objet.
   The controller calls importer() through the proxy, then enters the real object.
3. importer() appelle this.enregistrer() : l'appel ne repasse pas par le proxy, donc @Transactional est ignoré.
   importer() calls this.enregistrer(): the call does not go back through the proxy, so @Transactional is ignored.
4. Solution : placer la méthode transactionnelle dans un autre bean, ou mettre @Transactional sur la méthode appelée de l'extérieur.
   Fix: move the transactional method to another bean, or put @Transactional on the method called from outside.

Rollback

Rollback

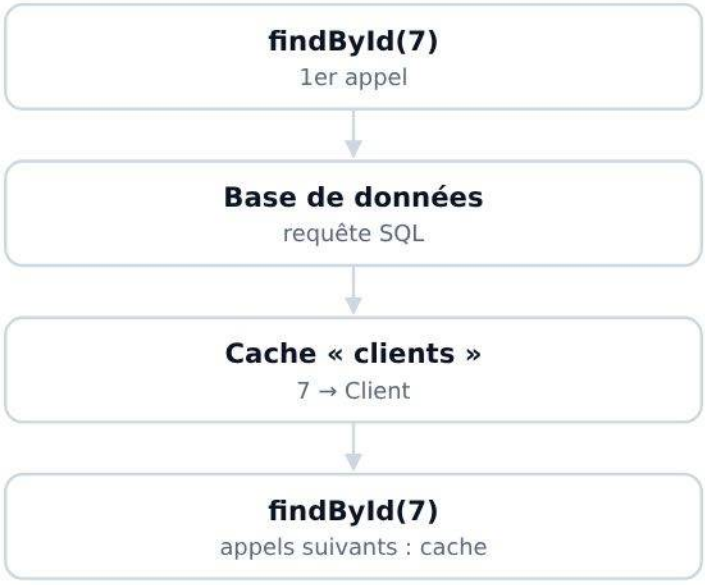


La réponse, étape par étape · *Step-by-step answer*

1. Le paiement est enregistré dans la transaction.
   The payment is saved inside the transaction.
2. Si une RuntimeException (ou une Error) sort de la méthode, Spring annule tout : c'est le comportement par défaut.
   If a RuntimeException (or an Error) leaves the method, Spring rolls everything back: that is the default.
3. Mais une exception vérifiée comme IOException valide la transaction : le paiement reste en base alors que l'envoi a échoué.
   But a checked exception like IOException commits: the payment stays in the database although sending failed.
4. rollbackFor = Exception.class force le rollback pour les exceptions vérifiées aussi.
   rollbackFor = Exception.class forces a rollback for checked exceptions too.

@Cacheable

@Cacheable



La réponse, étape par étape · *Step-by-step answer*

1. Premier appel : la méthode s'exécute et interroge la base.
   First call: the method runs and queries the database.
2. Le résultat est rangé dans le cache « clients » sous la clé 7.
   The result is stored in the "clients" cache under key 7.
3. Appels suivants avec 7 : Spring renvoie la valeur du cache sans exécuter la méthode, donc sans SQL.
   Next calls with 7: Spring returns the cached value without running the method, so no SQL.
4. Quand le client change, @CacheEvict retire l'entrée pour ne pas servir une donnée périmée. Ne pas oublier @EnableCaching.
   When the client changes, @CacheEvict removes the entry so stale data is not served. Don't forget @EnableCaching.

@Async

@Async



La réponse, étape par étape · *Step-by-step answer*

1. Sans @EnableAsync, @Async est ignoré et la méthode s'exécute normalement.
   Without @EnableAsync, @Async is ignored and the method runs normally.
2. L'appel venant d'un autre bean passe par le proxy, qui confie la méthode à un pool de threads.
   The call from another bean goes through the proxy, which hands the method to a thread pool.
3. La requête HTTP se termine tout de suite ; le mail part en arrière-plan.
   The HTTP request ends at once; the mail is sent in the background.
4. Comme @Transactional, un appel interne (this.envoyer) contourne le proxy. On renvoie void ou CompletableFuture pour suivre le résultat.
   Like @Transactional, an internal call (this.envoyer) bypasses the proxy. Return void or CompletableFuture to track the result.

Refresh token

Refresh token

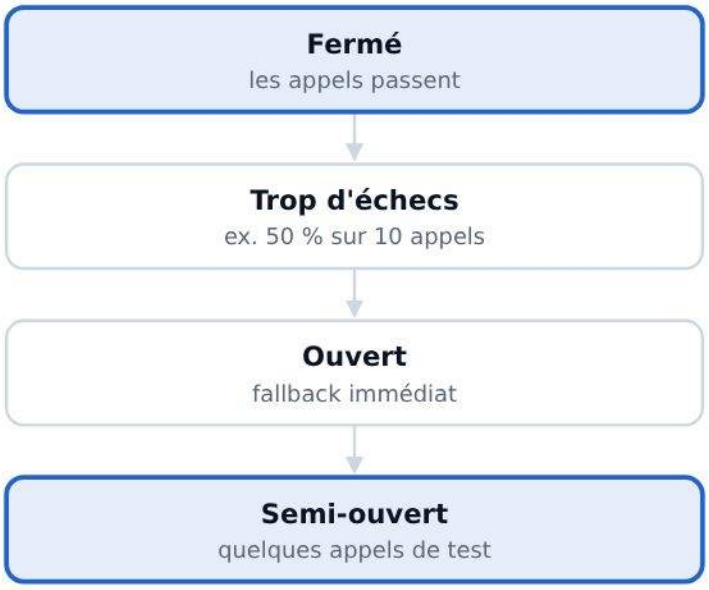


La réponse, étape par étape · *Step-by-step answer*

1. À la connexion, le serveur renvoie deux jetons.
   At login, the server returns two tokens.
2. L'access token JWT accompagne chaque appel. Il n'est pas stocké côté serveur, donc difficile à révoquer : on le garde court.
   The JWT access token goes with every call. It is not stored server-side, so hard to revoke: keep it short.
3. Quand il expire, le client envoie le refresh token et reçoit un nouvel access token, sans redemander le mot de passe.
   When it expires, the client sends the refresh token and gets a new access token, without asking for the password.
4. Le refresh token est enregistré en base : à la déconnexion ou en cas de vol, on le révoque. Un token volé ne sert que 15 minutes.
   The refresh token is stored in the database: on logout or theft, revoke it. A stolen access token only works for 15 minutes.

Circuit breaker

Circuit breaker

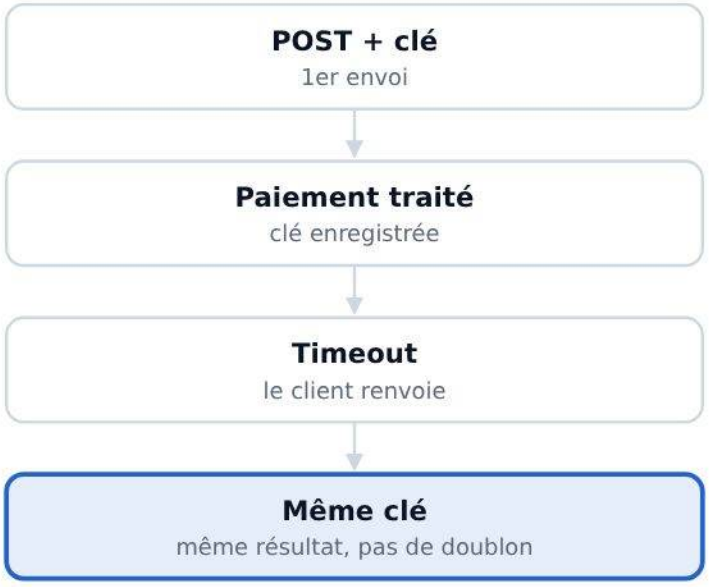


La réponse, étape par étape · *Step-by-step answer*

1. En temps normal, le circuit est fermé : chaque appel part vers le service de stock.
   Normally the circuit is closed: every call goes to the stock service.
2. Le service devient lent ou en erreur : le taux d'échec dépasse le seuil.
   The service becomes slow or failing: the failure rate crosses the threshold.
3. Le circuit s'ouvre : les appels échouent tout de suite et renvoient le fallback, sans attendre ni surcharger le service.
   The circuit opens: calls fail fast and return the fallback, without waiting or overloading the service.
4. Après un délai, quelques appels de test passent. S'ils réussissent, le circuit se referme.
   After a delay, a few test calls go through. If they succeed, the circuit closes again.

Idempotence

Idempotency



La réponse, étape par étape · *Step-by-step answer*

1. Le client génère une clé unique et l'envoie avec le paiement.
   The client generates a unique key and sends it with the payment.
2. Le serveur traite le paiement et mémorise la clé avec le résultat.
   The server processes the payment and stores the key with the result.
3. La réponse se perd (timeout) : le client renvoie exactement la même requête.
   The response gets lost (timeout): the client resends exactly the same request.
4. Le serveur reconnaît la clé et renvoie le résultat déjà produit : un seul débit. C'est l'idempotence.
   The server recognises the key and returns the result already produced: one charge only. That is idempotency.

Pattern outbox

Outbox pattern

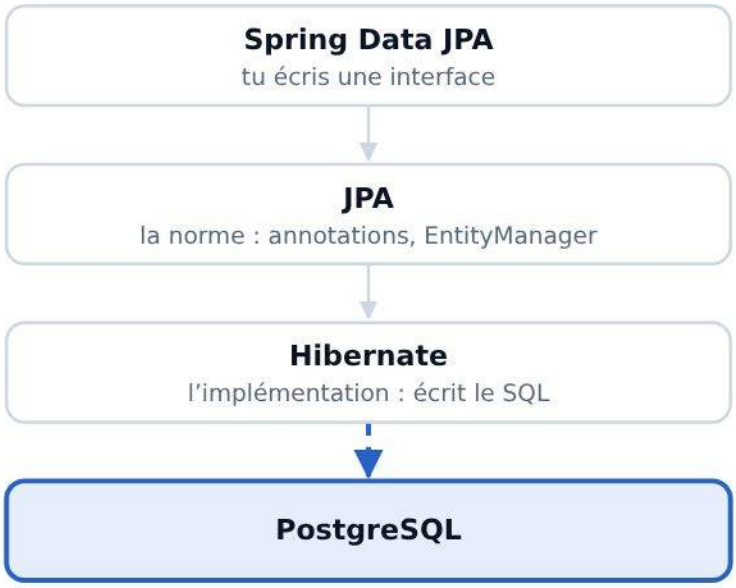


La réponse, étape par étape · *Step-by-step answer*

1. Problème : la base et Kafka n'ont pas de transaction commune. Publier avant ou après le commit peut perdre un message.
   Problem: the database and Kafka share no transaction. Publishing before or after the commit can lose a message.
2. On écrit la commande ET l'événement dans la même base, dans la même transaction.
   Write the order AND the event to the same database, in the same transaction.
3. Un seul commit : soit les deux sont enregistrés, soit aucun.
   One commit: either both are saved or neither.
4. Un relais lit la table outbox et publie sur Kafka, en réessayant si besoin. Les consommateurs doivent tolérer un doublon.
   A relay reads the outbox table and publishes to Kafka, retrying if needed. Consumers must tolerate a duplicate.

JPA, Hibernate, Spring Data

JPA, Hibernate, Spring Data

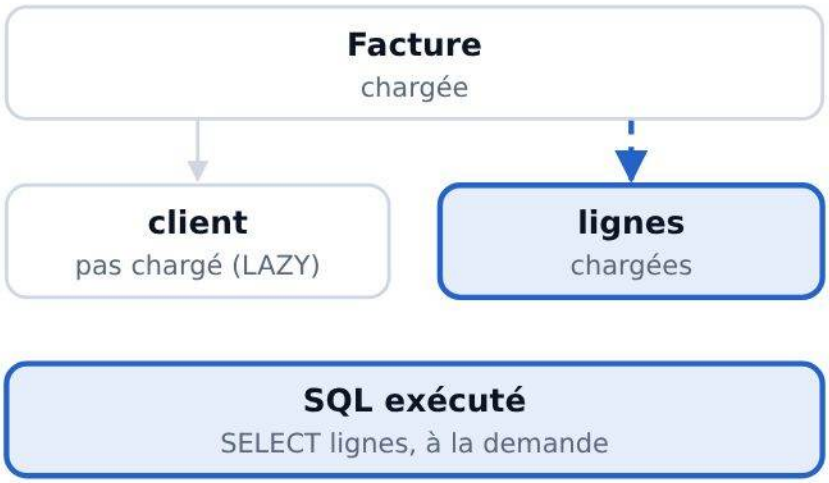


La réponse, étape par étape · *Step-by-step answer*

1. Tu déclares seulement une méthode dans une interface. Spring Data comprend son nom.
   You only declare a method in an interface. Spring Data understands its name.
2. Spring Data passe par JPA, la norme Java.
   Spring Data goes through JPA, the Java standard.
3. Hibernate, qui applique cette norme, écrit la requête SQL.
   Hibernate, which implements this standard, writes the SQL query.
4. Le SQL est exécuté sur PostgreSQL. Tu n'as écrit aucune requête.
   The SQL runs on PostgreSQL. You did not write a single query.

LAZY ou EAGER

LAZY or EAGER

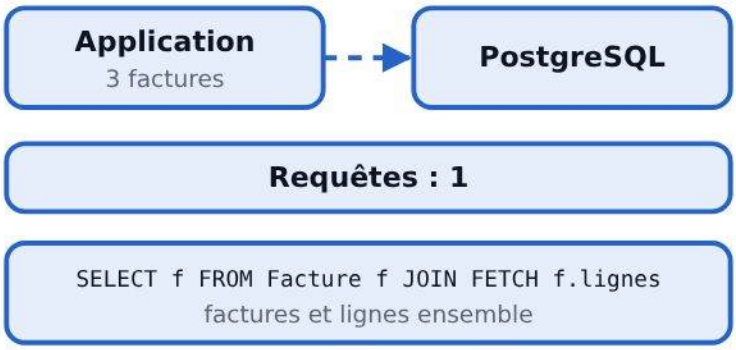


La réponse, étape par étape · *Step-by-step answer*

1. findById charge seulement la facture.
   findById loads only the invoice.
2. Le client et les lignes sont LAZY : Hibernate met juste une « boîte vide » à leur place.
   The customer and the lines are LAZY: Hibernate just puts an "empty box" in their place.
3. Au premier accès, Hibernate va chercher les lignes. En EAGER, elles auraient été chargées dès le début, même inutiles.
   On first access, Hibernate fetches the lines. With EAGER, they would have been loaded from the start, even if useless.

N+1

N+1



La réponse, étape par étape · *Step-by-step answer*

1. On demande toutes les factures : 1 requête. Mais leurs lignes ne sont pas encore chargées.
   We ask for all invoices: 1 query. But their lines are not loaded yet.
2. Dans la boucle, on demande les lignes de la facture 1 : Hibernate fait une nouvelle requête.
   In the loop, we ask for the lines of invoice 1: Hibernate runs a new query.
3. Facture 2 : encore une requête.
   Invoice 2: another query.
4. Facture 3 : encore une. 3 factures = 4 requêtes. 1 000 factures = 1 001 requêtes.
   Invoice 3: another one. 3 invoices = 4 queries. 1,000 invoices = 1,001 queries.
5. C'est le problème N+1 : 1 requête au départ, puis 1 de plus pour chaque facture. C'est très lent.
   This is the N+1 problem: 1 query at the start, then 1 more for each invoice. It is very slow.
6. La solution : JOIN FETCH. On charge factures et lignes en même temps, en 1 seule requête, quel que soit leur nombre.
   The solution: JOIN FETCH. We load invoices and lines together, in just 1 query, however many there are.

LazyInitializationException

LazyInitializationException

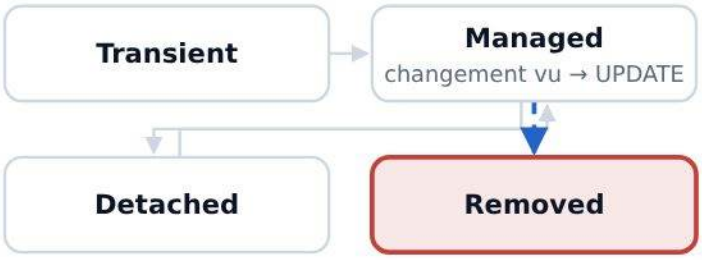


La réponse, étape par étape · *Step-by-step answer*

1. Dans le service, la session Hibernate est ouverte : la facture est chargée, sans ses lignes.
   In the service, the Hibernate session is open: the invoice is loaded, without its lines.
2. La méthode se termine : la session se ferme. La facture arrive dans le contrôleur.
   The method ends: the session closes. The invoice reaches the controller.
3. Le contrôleur demande les lignes, mais il n'y a plus de session pour les charger : erreur.
   The controller asks for the lines, but there is no session left to load them: error.
4. Solution : tout charger pendant que la session est encore ouverte.
   Solution: load everything while the session is still open.

États d’une entité

Entity states



La réponse, étape par étape · *Step-by-step answer*

1. new crée un objet Java normal : Hibernate ne le connaît pas.
   new creates a normal Java object: Hibernate does not know about it.
2. persist : l’entité devient « managed », suivie par Hibernate.
   persist: the entity becomes "managed", tracked by Hibernate.
3. Chaque changement est remarqué et enregistré au commit, sans appeler save().
   Every change is noticed and saved at commit, without calling save().
4. detach (ou fin de session) : l’entité n’est plus suivie. Ses changements sont ignorés.
   detach (or end of session): the entity is no longer tracked. Its changes are ignored.
5. merge : on la rattache, elle est de nouveau suivie.
   merge: we reattach it, it is tracked again.
6. remove : elle sera supprimée au commit.
   remove: it will be deleted at commit.

@Version

@Version

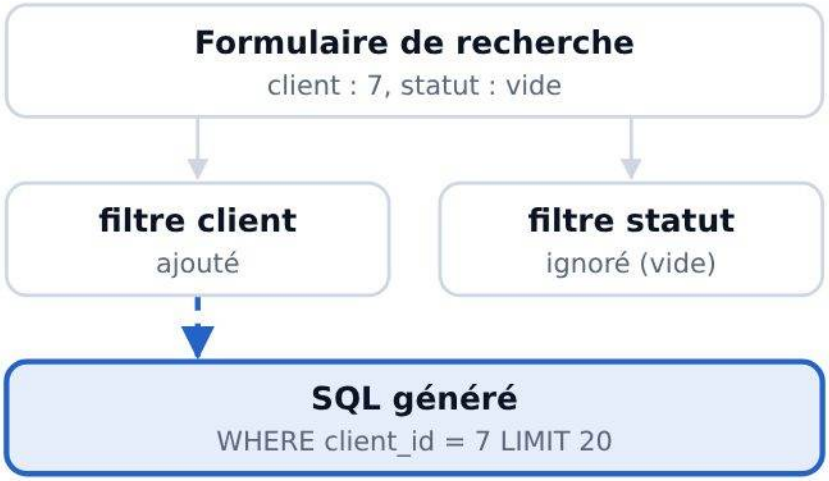


La réponse, étape par étape · *Step-by-step answer*

1. Ali et Sami ouvrent la même facture, en version 1.
   Ali and Sami open the same invoice, at version 1.
2. Ali enregistre en premier : la version passe à 2.
   Ali saves first: the version goes to 2.
3. Sami enregistre avec la version 1, qui n'existe plus : refusé. Il n'écrase pas le travail d'Ali.
   Sami saves with version 1, which no longer exists: rejected. He does not overwrite Ali's work.
4. On lui affiche « modifiée par quelqu'un d'autre » et il recharge la facture.
   He is shown "modified by someone else" and he reloads the invoice.

Filtres dynamiques

Dynamic filters



La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur remplit seulement le client.
   The user fills in only the customer.
2. Le client est renseigné : on ajoute la condition.
   The customer is filled in: we add the condition.
3. Le statut est vide : on n'ajoute rien.
   The status is empty: we add nothing.
4. La requête ne contient que les filtres utiles, avec la pagination.
   The query contains only the useful filters, with pagination.

save ou saveAndFlush

save or saveAndFlush



La réponse, étape par étape · *Step-by-step answer*

1. save garde f1 en mémoire. Le SQL peut attendre.
   save keeps f1 in memory. The SQL can wait.
2. saveAndFlush envoie tout de suite tout ce qui attend. Une erreur (doublon…) apparaît ici.
   saveAndFlush immediately sends everything that is waiting. An error (duplicate…) shows up here.
3. f3 attend à son tour.
   f3 waits in turn.
4. À la fin de la transaction, le reste est envoyé puis validé.
   At the end of the transaction, the rest is sent and then committed.

Relation bidirectionnelle

Bidirectional relationship

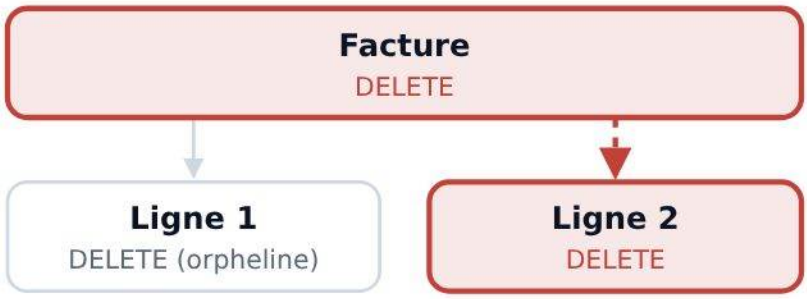


La réponse, étape par étape · *Step-by-step answer*

1. Côté Facture : la liste des lignes. mappedBy veut dire « ce n'est pas moi qui gère la colonne ».
   On the Facture side: the list of lines. mappedBy means "I am not the one managing the column".
2. Côté Ligne : @ManyToOne porte la vraie colonne facture_id. C'est le côté « propriétaire ».
   On the Ligne side: @ManyToOne holds the real facture_id column. It is the "owning" side.
3. ajouterLigne remplit les deux côtés en même temps, pour que Java et la base restent d'accord.
   ajouterLigne fills both sides at the same time, so that Java and the database stay in agreement.

cascade, orphanRemoval

cascade, orphanRemoval



La réponse, étape par étape · *Step-by-step answer*

1. cascade ALL : en enregistrant la facture, ses lignes sont enregistrées aussi.
   cascade ALL: when saving the invoice, its lines are saved too.
2. orphanRemoval : une ligne retirée de la liste est supprimée en base.
   orphanRemoval: a line removed from the list is deleted in the database.
3. Supprimer la facture supprime aussi ses lignes. À réserver aux enfants qui n'existent pas sans leur parent.
   Deleting the invoice also deletes its lines. Reserve this for children that cannot exist without their parent.

Pagination

Pagination



La réponse, étape par étape · *Step-by-step answer*

1. On demande la première page de 20 factures, les plus récentes d'abord.
   We ask for the first page of 20 invoices, most recent first.
2. Hibernate charge seulement 20 lignes, pas les 1 250.
   Hibernate loads only 20 rows, not all 1,250.
3. Une deuxième requête compte le total, pour afficher « page 1 sur 63 ».
   A second query counts the total, to display "page 1 of 63".
4. L'objet Page contient tout ce qu'il faut pour la pagination côté Angular.
   The Page object contains everything needed for pagination on the Angular side.

Projection

Projection

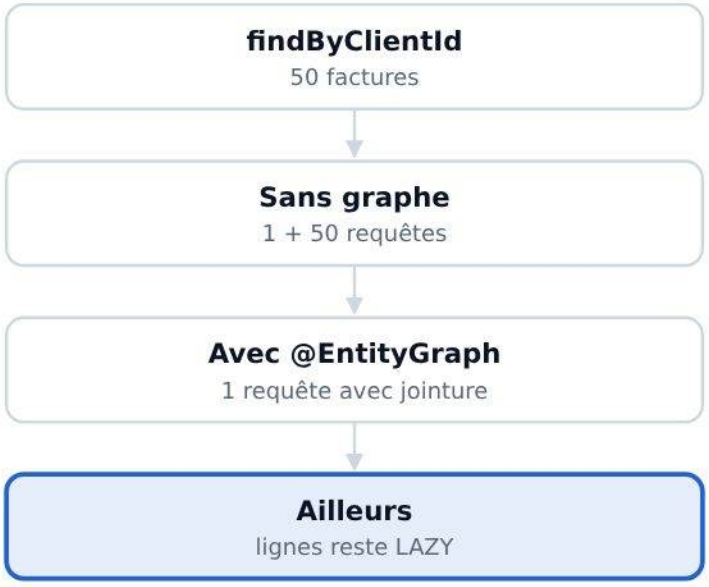


La réponse, étape par étape · *Step-by-step answer*

1. Pour une simple liste, charger l'entité complète ramène beaucoup de colonnes inutiles.
   For a simple list, loading the full entity brings back many useless columns.
2. Avec une projection, on ne lit que les 2 colonnes affichées : plus rapide.
   With a projection, we read only the 2 displayed columns: faster.
3. Un DTO n'a pas de relations LAZY : pas de LazyInitializationException possible.
   A DTO has no LAZY relationships: no LazyInitializationException possible.

@EntityGraph

@EntityGraph

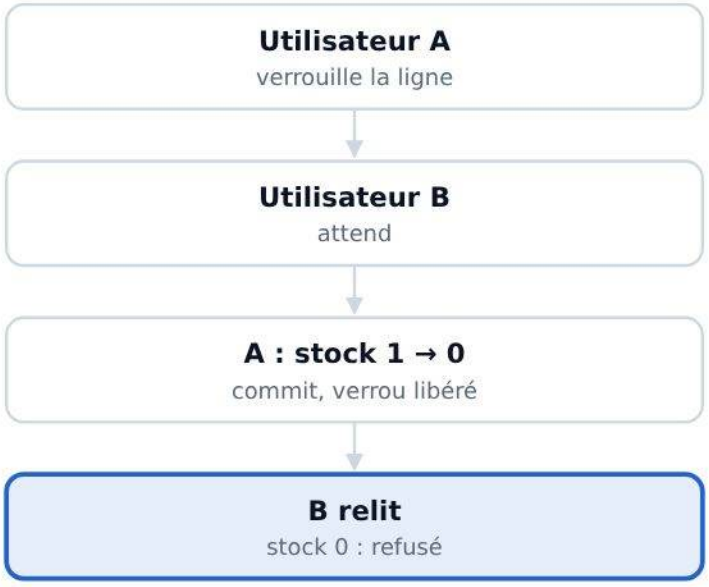


La réponse, étape par étape · *Step-by-step answer*

1. On charge les factures d'un client, puis on affiche leurs lignes.
   We load a client's invoices, then display their lines.
2. Sans rien, chaque accès à facture.getLignes() déclenche une requête : c'est le problème N+1.
   With nothing, each access to invoice.getLines() fires a query: the N+1 problem.
3. @EntityGraph ajoute une jointure pour cette méthode : factures et lignes arrivent en une seule requête.
   @EntityGraph adds a join for this method: invoices and lines arrive in one query.
4. La relation reste LAZY partout ailleurs : on charge le graphe seulement là où on en a besoin.
   The relation stays LAZY everywhere else: the graph is loaded only where needed.

Verrou pessimiste

Pessimistic lock

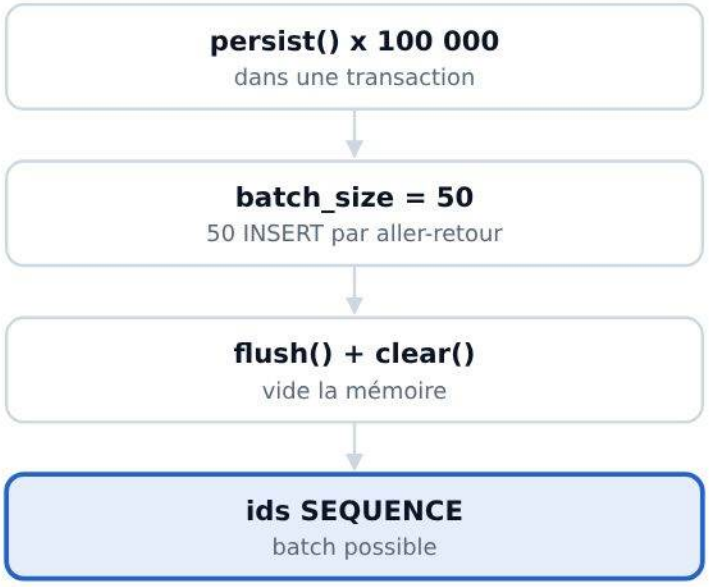


La réponse, étape par étape · *Step-by-step answer*

1. A lit l'article avec SELECT ... FOR UPDATE : la ligne est verrouillée en base.
   A reads the item with SELECT ... FOR UPDATE: the row is locked in the database.
2. B veut le même article : sa requête attend la fin de la transaction de A.
   B wants the same item: its query waits for A's transaction to end.
3. A décrémente le stock et valide : le verrou est libéré.
   A decrements the stock and commits: the lock is released.
4. B lit alors le stock à jour (0) et ne survend pas. Le verrou optimiste (@Version) laisse passer et détecte le conflit au commit, sans attente.
   B then reads the fresh stock (0) and does not oversell. The optimistic lock (@Version) lets it through and detects the conflict at commit, without waiting.

Insertions en masse

Bulk inserts



La réponse, étape par étape · *Step-by-step answer*

1. Sans réglage, Hibernate envoie un INSERT à la fois et garde les 100 000 entités en mémoire.
   Without tuning, Hibernate sends one INSERT at a time and keeps all 100,000 entities in memory.
2. batch_size regroupe les INSERT par paquets de 50 : 50 fois moins d'allers-retours avec la base.
   batch_size groups INSERTs in packs of 50: 50 times fewer round trips to the database.
3. Tous les 50 éléments, flush() envoie le paquet et clear() vide le contexte de persistance : la mémoire reste stable.
   Every 50 items, flush() sends the pack and clear() empties the persistence context: memory stays stable.
4. Avec des id IDENTITY, Hibernate doit faire chaque INSERT pour connaître l'id : il désactive le batch. On utilise SEQUENCE.
   With IDENTITY ids, Hibernate must run each INSERT to learn the id, so it disables batching. Use SEQUENCE.

Cache de second niveau

Second-level cache

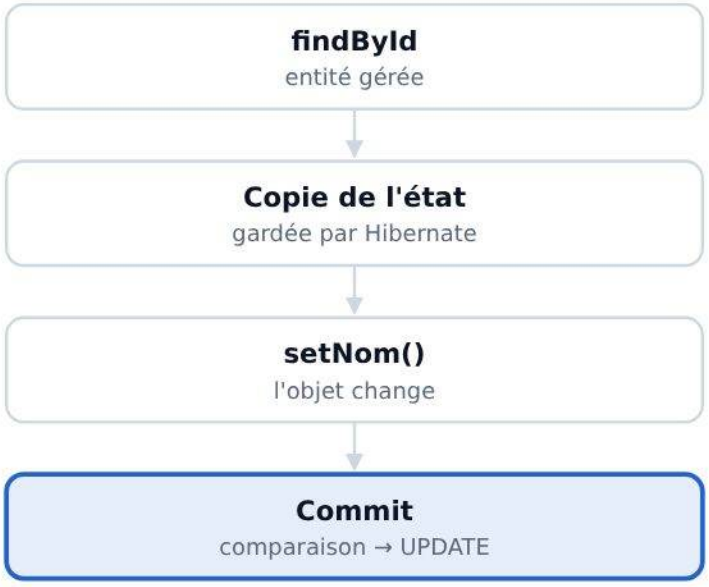


La réponse, étape par étape · *Step-by-step answer*

1. Le cache de 1er niveau est le contexte de persistance : il vit le temps d'une session (d'une transaction).
   The first-level cache is the persistence context: it lives for one session (one transaction).
2. Une autre session ne profite pas de ce cache : elle relit la base.
   Another session does not benefit from it: it reads the database again.
3. Le cache de 2e niveau, optionnel, est partagé par toutes les sessions. On l'active entité par entité.
   The second-level cache, optional, is shared by all sessions. It is enabled entity by entity.
4. Idéal pour des données lues souvent et rarement modifiées (pays, taux de TVA). À éviter pour des données qui changent sans cesse.
   Ideal for data read often and rarely changed (countries, VAT rates). Avoid it for constantly changing data.

Dirty checking

Dirty checking

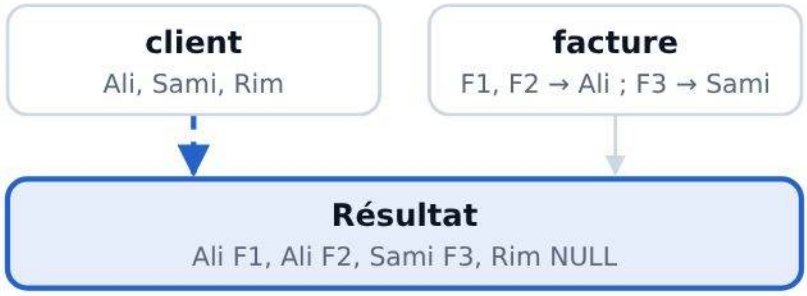


La réponse, étape par étape · *Step-by-step answer*

1. Dans une transaction, findById renvoie une entité « gérée » par le contexte de persistance.
   Inside a transaction, findById returns an entity "managed" by the persistence context.
2. Hibernate garde une copie de son état d'origine.
   Hibernate keeps a copy of its original state.
3. On modifie le nom sans appeler save().
   We change the name without calling save().
4. Au flush (au commit), Hibernate compare l'objet à sa copie, voit la différence et génère l'UPDATE tout seul : c'est le dirty checking.
   At flush (commit), Hibernate compares the object with its copy, sees the change and generates the UPDATE itself: dirty checking.

Jointures

Joins



La réponse, étape par étape · *Step-by-step answer*

1. Deux tables : 3 clients, 3 factures. Rim n'a aucune facture.
   Two tables: 3 customers, 3 invoices. Rim has no invoice.
2. INNER JOIN : seulement les clients qui ont une facture. Rim disparaît.
   INNER JOIN: only customers who have an invoice. Rim disappears.
3. LEFT JOIN : tous les clients. Rim apparaît, avec NULL à la place du numéro.
   LEFT JOIN: all customers. Rim appears, with NULL instead of the number.

WHERE ou HAVING

WHERE or HAVING



La réponse, étape par étape · *Step-by-step answer*

1. On part de toutes les factures.
   We start from all invoices.
2. WHERE filtre les lignes une par une, avant tout regroupement.
   WHERE filters rows one by one, before any grouping.
3. GROUP BY regroupe par client et calcule la somme.
   GROUP BY groups by customer and calculates the sum.
4. HAVING filtre les groupes, avec la somme. WHERE ne peut pas le faire.
   HAVING filters groups, using the sum. WHERE cannot do that.

Index

Indexes

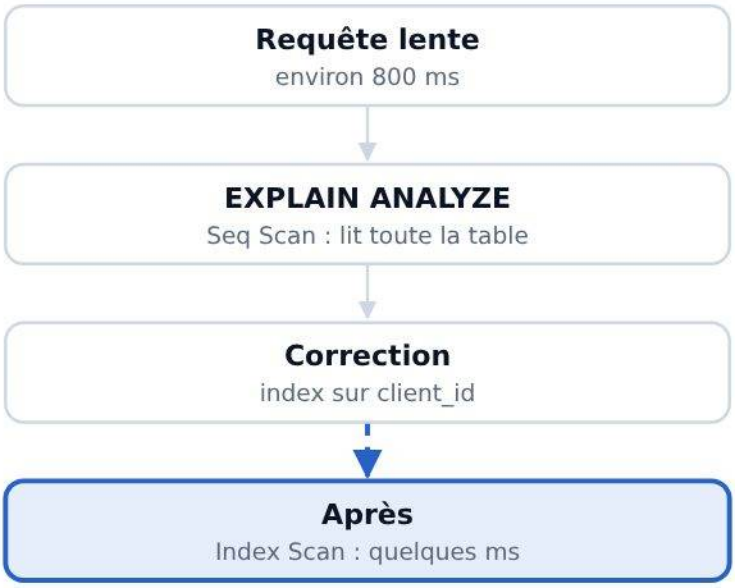


La réponse, étape par étape · *Step-by-step answer*

1. Sans index, PostgreSQL lit toute la table, ligne par ligne.
   Without an index, PostgreSQL reads the whole table, row by row.
2. On crée un index : une sorte de sommaire trié de la colonne.
   We create an index: a kind of sorted table of contents for the column.
3. La recherche va directement à la bonne ligne, comme l'index d'un livre. En échange, les écritures sont un peu plus lentes.
   The search goes straight to the right row, like the index of a book. In exchange, writes are a bit slower.

Requête lente

Slow query



La réponse, étape par étape · *Step-by-step answer*

1. Une requête est lente.
   A query is slow.
2. EXPLAIN ANALYZE montre comment PostgreSQL l'exécute vraiment.
   EXPLAIN ANALYZE shows how PostgreSQL really runs it.
3. On ajoute l'index qui manque.
   We add the missing index.
4. On vérifie à nouveau : la requête est devenue très rapide.
   We check again: the query has become very fast.

ACID

ACID



La réponse, étape par étape · *Step-by-step answer*

1. Atomicité : la facture payée et le paiement sont enregistrés ensemble, ou pas du tout.
   Atomicity: the paid invoice and the payment are saved together, or not at all.
2. Cohérence : les règles (clé étrangère, montant positif…) sont toujours respectées.
   Consistency: the rules (foreign key, positive amount…) are always respected.
3. Isolation : pendant ce temps, les autres utilisateurs ne voient pas le travail à moitié fait.
   Isolation: meanwhile, other users do not see the half-done work.
4. Durabilité : après COMMIT, c’est enregistré, même si le serveur tombe juste après.
   Durability: after COMMIT, it is saved, even if the server crashes right after.

Fonctions de fenêtrage

Window functions



La réponse, étape par étape · *Step-by-step answer*

1. Toutes les factures. Chaque ligne est gardée, sans regroupement.
   All invoices. Each row is kept, with no grouping.
2. PARTITION BY fait un groupe par client. Pour Ali, les factures sont classées de la plus récente à la plus ancienne.
   PARTITION BY makes one group per customer. For Ali, the invoices are ranked from most recent to oldest.
3. Même chose pour Sami. Les lignes restent séparées, contrairement à GROUP BY.
   Same for Sami. Rows stay separate, unlike GROUP BY.
4. On garde rang = 1 : la dernière facture de chaque client.
   We keep rang = 1: each customer's latest invoice.

DELETE, TRUNCATE, DROP

DELETE, TRUNCATE, DROP



La réponse, étape par étape · *Step-by-step answer*

1. DELETE retire les lignes choisies.
   DELETE removes the selected rows.
2. TRUNCATE vide toute la table d’un coup.
   TRUNCATE empties the whole table at once.
3. DROP supprime la table elle-même.
   DROP deletes the table itself.
4. En facturation, on garde l’historique : on change le statut au lieu de supprimer.
   In invoicing, we keep the history: we change the status instead of deleting.

Clés

Keys



La réponse, étape par étape · *Step-by-step answer*

1. La clé primaire identifie chaque facture de façon unique.
   The primary key uniquely identifies each invoice.
2. La clé étrangère relie chaque ligne à une facture qui existe.
   The foreign key links each line to an invoice that exists.
3. PostgreSQL refuse une ligne qui pointe vers une facture inexistante.
   PostgreSQL rejects a line that points to a non-existent invoice.

Normalisation

Normalization

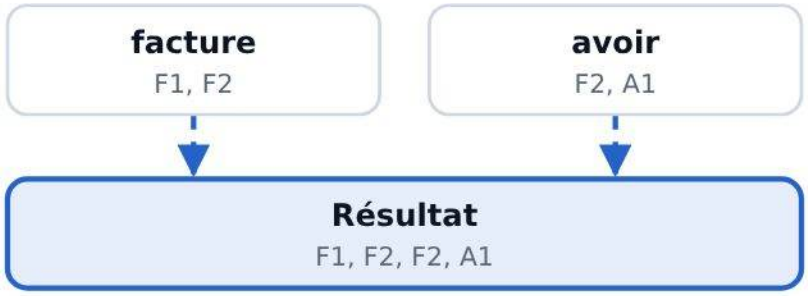


La réponse, étape par étape · *Step-by-step answer*

1. Avant : l'adresse d'Ali est recopiée dans chaque facture.
   Before: Ali's address is copied into every invoice.
2. Ali déménage : il faut modifier toutes les factures. Si on en oublie une, les données se contredisent.
   Ali moves: every invoice must be changed. If we forget one, the data contradicts itself.
3. Après : l'adresse est stockée une seule fois. Une seule modification suffit.
   After: the address is stored only once. A single change is enough.

UNION

UNION



La réponse, étape par étape · *Step-by-step answer*

1. Deux requêtes, avec une valeur en commun : F2.
   Two queries, with one value in common: F2.
2. UNION empile les résultats et retire les doublons : il doit comparer, donc c'est plus lent.
   UNION stacks the results and removes duplicates: it has to compare, so it is slower.
3. UNION ALL empile tout, sans vérifier : plus rapide.
   UNION ALL stacks everything, without checking: faster.

EXISTS

EXISTS



La réponse, étape par étape · *Step-by-step answer*

1. Pour chaque client, une question : a-t-il au moins une facture impayée ?
   For each customer, one question: do they have at least one unpaid invoice?
2. EXISTS parcourt ses factures et s'arrête dès la première impayée.
   EXISTS goes through their invoices and stops at the first unpaid one.
3. Ali est gardé. Pas besoin de compter toutes ses factures : c'est souvent rapide.
   Ali is kept. No need to count all his invoices: it is often fast.

Vues

Views



La réponse, étape par étape · *Step-by-step answer*

1. Une vue est une requête enregistrée sous un nom.
   A view is a query saved under a name.
2. Chaque fois qu'on l'interroge, PostgreSQL relance la requête sur la table.
   Each time we query it, PostgreSQL reruns the query on the table.
3. Une vue matérialisée garde le résultat déjà calculé : lecture très rapide.
   A materialized view keeps the already calculated result: very fast reads.
4. Mais il faut la rafraîchir pour voir les nouvelles factures.
   But it must be refreshed to see new invoices.

EXPLAIN ANALYZE

EXPLAIN ANALYZE

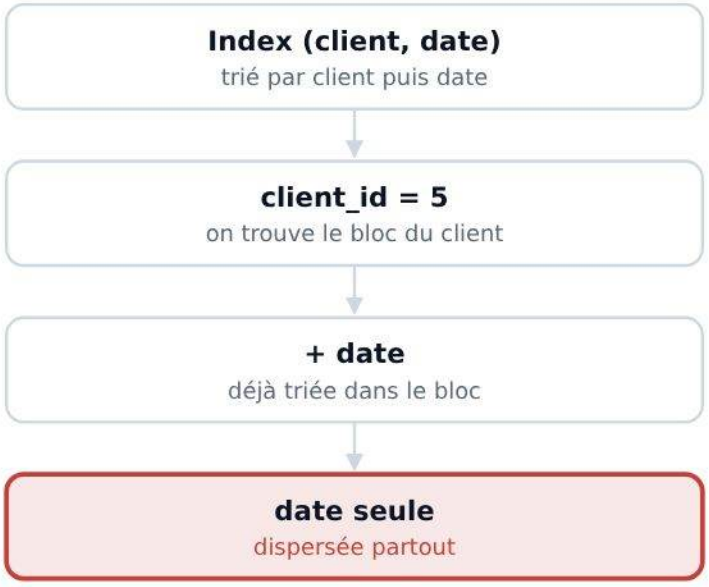


La réponse, étape par étape · *Step-by-step answer*

1. EXPLAIN ANALYZE exécute la requête et montre le plan choisi, avec le temps réel de chaque étape.
   EXPLAIN ANALYZE runs the query and shows the chosen plan, with the real time of each step.
2. Seq Scan : PostgreSQL lit toute la table pour trouver 37 lignes. C'est le signe d'un index manquant.
   Seq Scan: PostgreSQL reads the whole table to find 37 rows. A sign of a missing index.
3. On crée un index sur la colonne filtrée.
   Create an index on the filtered column.
4. Le plan devient un Index Scan : 850 ms → 0,4 ms. Sur une petite table ou un filtre peu sélectif, le Seq Scan peut rester le meilleur choix.
   The plan becomes an Index Scan: 850 ms → 0.4 ms. On a small table or an unselective filter, a Seq Scan can still be the best choice.

Index composite

Composite index

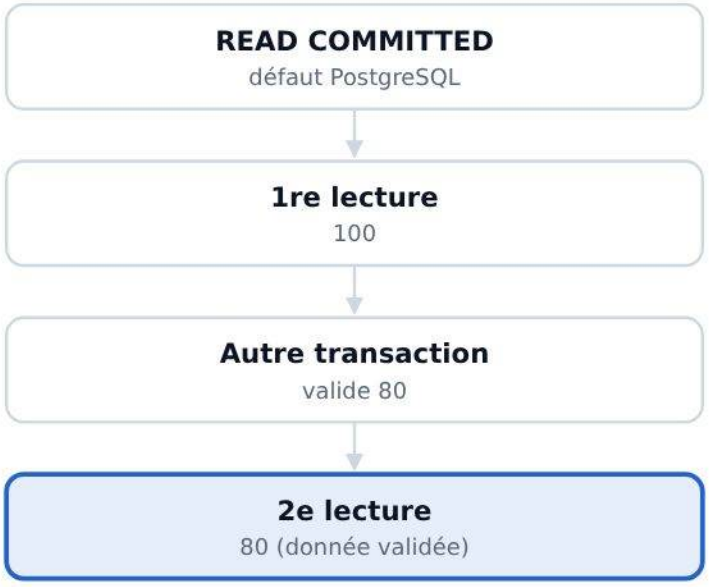


La réponse, étape par étape · *Step-by-step answer*

1. Un index composite est trié comme un annuaire : d'abord par client, puis par date à l'intérieur de chaque client.
   A composite index is sorted like a phone book: first by client, then by date within each client.
2. Filtrer sur client_id : on saute directement au bloc du client.
   Filter on client_id: jump straight to the client's block.
3. Dans ce bloc, les dates sont déjà triées : filtre et ORDER BY sur la date sont rapides.
   Inside that block, dates are already sorted: filtering and ORDER BY on the date are fast.
4. Sans client_id, les dates sont éparpillées dans tous les blocs : l'index ne sert pas. Mettre en premier la colonne la plus utilisée dans les filtres.
   Without client_id, dates are scattered across all blocks: the index is useless. Put the most filtered column first.

Niveaux d'isolation

Isolation levels

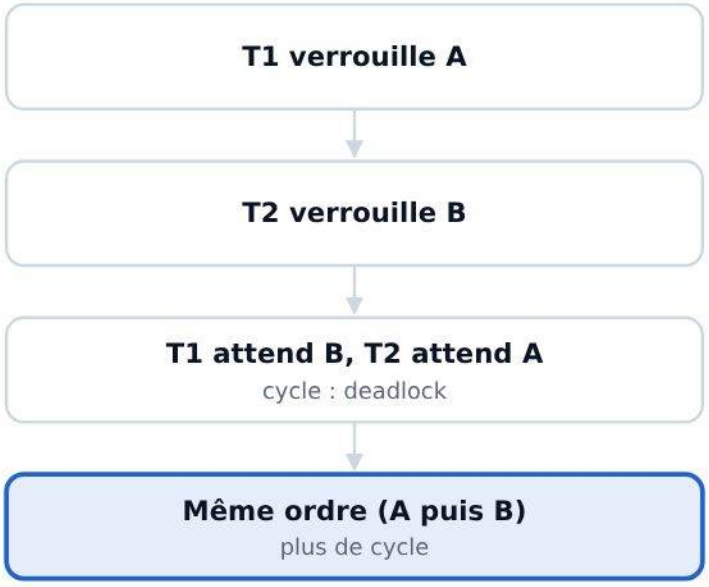


La réponse, étape par étape · *Step-by-step answer*

1. PostgreSQL utilise READ COMMITTED par défaut : on ne voit jamais de données non validées.
   PostgreSQL uses READ COMMITTED by default: you never see uncommitted data.
2. Première lecture du solde dans la transaction : 100.
   First read of the balance inside the transaction: 100.
3. Pendant ce temps, une autre transaction modifie le solde et valide.
   Meanwhile another transaction changes the balance and commits.
4. Deuxième lecture : 80. Chaque requête voit l'état validé à son démarrage. REPEATABLE READ figerait 100 ; SERIALIZABLE empêche aussi les anomalies de concurrence.
   Second read: 80. Each statement sees the committed state when it starts. REPEATABLE READ would freeze 100; SERIALIZABLE also prevents concurrency anomalies.

Deadlock

Deadlock



La réponse, étape par étape · *Step-by-step answer*

1. T1 commence par verrouiller la ligne A.
   T1 starts by locking row A.
2. En même temps, T2 verrouille la ligne B.
   At the same time, T2 locks row B.
3. T1 veut B (tenue par T2) et T2 veut A (tenue par T1) : chacun attend l'autre. PostgreSQL détecte le cycle et annule une des deux.
   T1 wants B (held by T2) and T2 wants A (held by T1): each waits for the other. PostgreSQL detects the cycle and aborts one.
4. Si les deux transactions verrouillent toujours dans le même ordre (par id), le cycle est impossible. Garder les transactions courtes aide aussi.
   If both transactions always lock in the same order (by id), the cycle cannot happen. Keeping transactions short helps too.

MVCC et VACUUM

MVCC and VACUUM

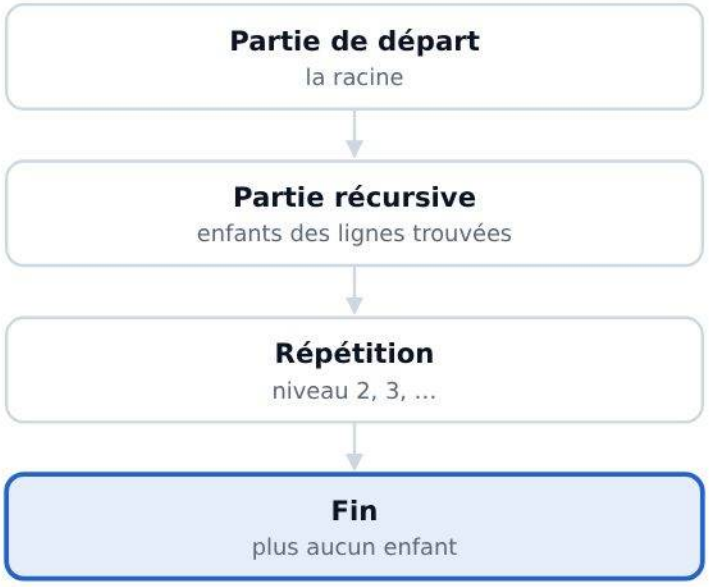


La réponse, étape par étape · *Step-by-step answer*

1. Avec MVCC, un UPDATE ne modifie pas la ligne sur place : il crée une nouvelle version.
   With MVCC, an UPDATE does not change the row in place: it creates a new version.
2. L'ancienne version reste pour les transactions qui l'ont commencée à lire : les lecteurs ne bloquent pas les écrivains.
   The old version stays for transactions already reading it: readers do not block writers.
3. Une fois que plus personne n'en a besoin, v1 devient une ligne morte qui occupe de la place.
   Once nobody needs it, v1 becomes a dead row taking up space.
4. (auto)VACUUM nettoie ces lignes mortes. Sans lui, tables et index gonflent et les requêtes ralentissent.
   (auto)VACUUM cleans these dead rows. Without it, tables and indexes bloat and queries slow down.

CTE récursive

Recursive CTE



La réponse, étape par étape · *Step-by-step answer*

1. La première partie sélectionne le point de départ : les catégories sans parent.
   The first part selects the starting point: categories without a parent.
2. La partie récursive joint la table aux lignes déjà trouvées pour obtenir leurs enfants.
   The recursive part joins the table to rows already found to get their children.
3. PostgreSQL répète l'opération avec les nouveaux enfants : petits-enfants, puis arrière-petits-enfants…
   PostgreSQL repeats with the new children: grandchildren, then great-grandchildren…
4. Quand une étape ne trouve plus personne, la récursion s'arrête. Toute la hiérarchie arrive en une seule requête.
   When a step finds nobody, recursion stops. The whole hierarchy arrives in one query.

Pagination par clé

Keyset pagination

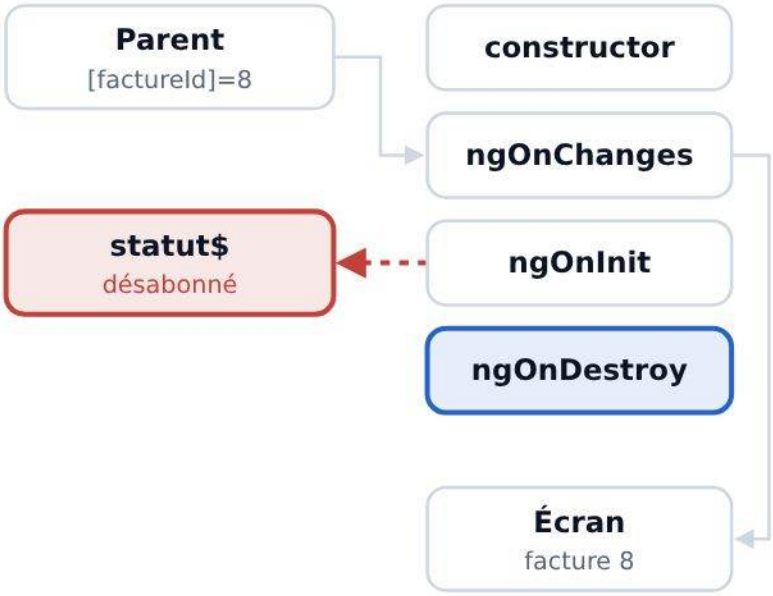


La réponse, étape par étape · *Step-by-step answer*

1. Avec OFFSET, la base doit parcourir toutes les lignes sautées.
   With OFFSET, the database must walk every skipped row.
2. Plus la page est loin, plus c'est lent : page 1 instantanée, page 5001 très lente.
   The further the page, the slower: page 1 is instant, page 5001 very slow.
3. La pagination par clé repart du dernier id affiché : l'index la place directement au bon endroit.
   Keyset pagination resumes from the last id shown: the index jumps straight to the right place.
4. Seules 20 lignes sont lues, quelle que soit la page. Limite : on va de page en page, sans sauter directement à la page 5001.
   Only 20 rows are read, whatever the page. Limit: you move page by page, not jump straight to page 5001.

Cycle de vie

Lifecycle

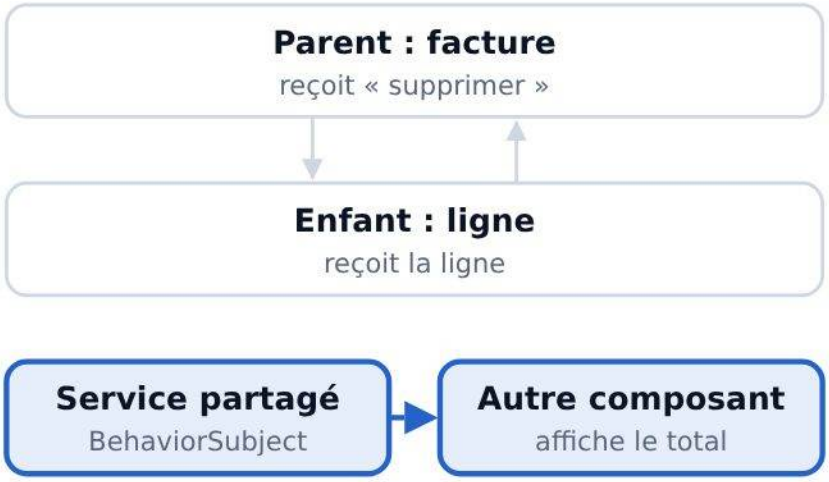


La réponse, étape par étape · *Step-by-step answer*

1. Angular crée le composant. Le constructeur sert seulement à recevoir les services, rien d'autre.
   Angular creates the component. The constructor is only used to receive services, nothing else.
2. Le parent donne factureId = 7. ngOnChanges se déclenche et charge la facture 7.
   The parent passes factureId = 7. ngOnChanges fires and loads invoice 7.
3. ngOnInit se lance une seule fois. On s'abonne aux mises à jour du statut.
   ngOnInit runs only once. We subscribe to status updates.
4. Le parent passe à la facture 8. ngOnChanges se relance et charge la 8. ngOnInit, lui, ne se relance pas.
   The parent switches to invoice 8. ngOnChanges fires again and loads invoice 8. ngOnInit does not run again.
5. On quitte l'écran : ngOnDestroy arrête l'abonnement. Sinon, il continuerait à tourner pour rien (fuite mémoire).
   We leave the screen: ngOnDestroy stops the subscription. Otherwise, it would keep running for nothing (memory leak).

Communication

Communication

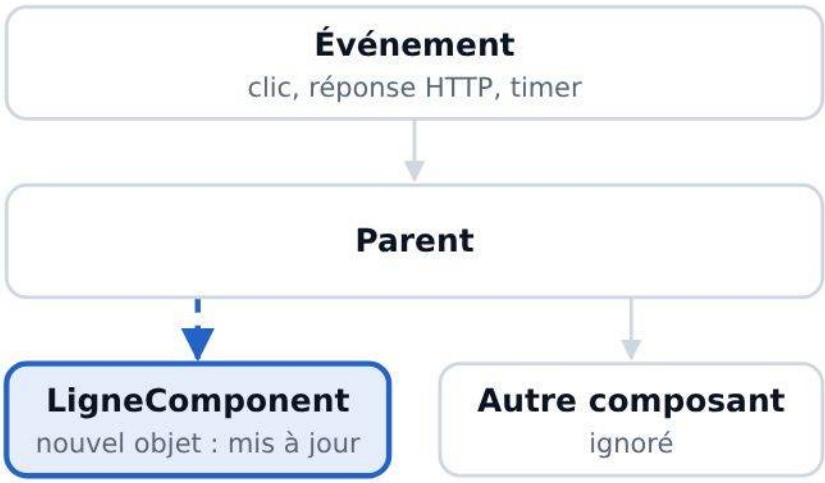


La réponse, étape par étape · *Step-by-step answer*

1. @Input : le parent donne une donnée à l'enfant.
   @Input: the parent gives data to the child.
2. @Output : l'enfant prévient le parent qu'il s'est passé quelque chose.
   @Output: the child tells the parent that something happened.
3. Pour des composants éloignés, on passe par un service partagé : l'un envoie, l'autre écoute.
   For distant components, we go through a shared service: one sends, the other listens.

Détection des changements

Change detection

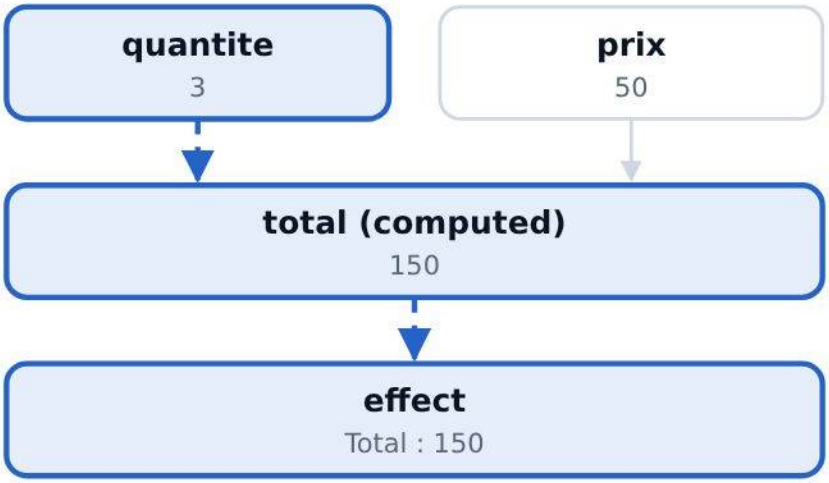


La réponse, étape par étape · *Step-by-step answer*

1. Après chaque événement, Angular vérifie les composants pour mettre l'écran à jour.
   After each event, Angular checks the components to update the screen.
2. Avec OnPush, un composant n'est vérifié que si son @Input change de référence (ou s'il a son propre événement).
   With OnPush, a component is checked only if its @Input changes reference (or if it has its own event).
3. Modifier l'objet existant ne change pas sa référence : l'écran n'est pas mis à jour.
   Modifying the existing object does not change its reference: the screen is not updated.
4. On crée un nouvel objet : Angular voit le changement. D'où l'habitude de ne pas modifier les objets.
   We create a new object: Angular sees the change. Hence the habit of not modifying objects.

Signals

Signals



La réponse, étape par étape · *Step-by-step answer*

1. Deux signals : des valeurs qu'Angular surveille.
   Two signals: values that Angular watches.
2. computed calcule le total à partir d'eux, et sait de qui il dépend.
   computed calculates the total from them, and knows what it depends on.
3. effect s'exécute à chaque changement du total.
   effect runs every time the total changes.
4. quantite passe à 3 : seul ce qui en dépend est recalculé, automatiquement.
   quantite goes to 3: only what depends on it is recalculated, automatically.

Standalone

Standalone

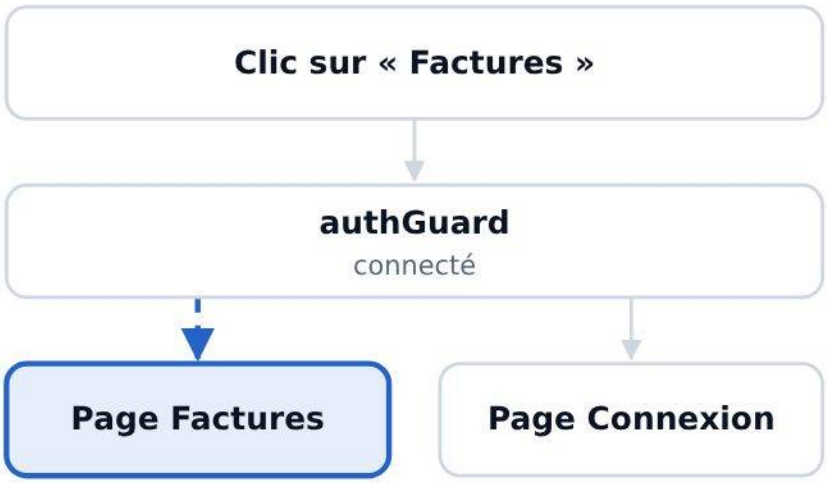


La réponse, étape par étape · *Step-by-step answer*

1. Avant, chaque composant devait être déclaré dans un NgModule.
   Before, each component had to be declared in an NgModule.
2. Un composant standalone importe directement ce dont il a besoin.
   A standalone component directly imports what it needs.
3. Il peut être chargé à la demande depuis une route, sans module intermédiaire.
   It can be loaded on demand from a route, without an intermediate module.

Guards

Guards

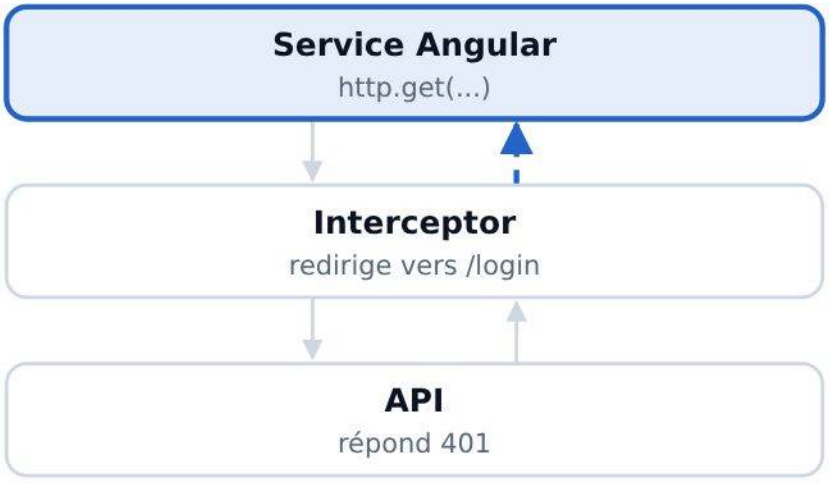


La réponse, étape par étape · *Step-by-step answer*

1. Avant d’ouvrir la page, Angular exécute le guard.
   Before opening the page, Angular runs the guard.
2. Utilisateur non connecté : le guard l’envoie vers la page de connexion.
   User not logged in: the guard sends them to the login page.
3. Utilisateur connecté : la page s’ouvre normalement.
   User logged in: the page opens normally.

Interceptor

Interceptor

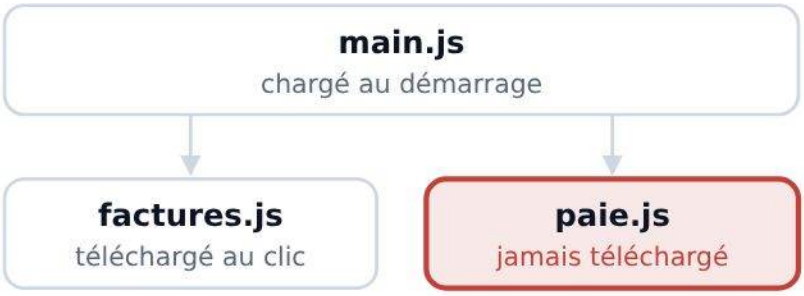


La réponse, étape par étape · *Step-by-step answer*

1. Un service lance une requête, sans s'occuper du token.
   A service sends a request, without worrying about the token.
2. L'interceptor ajoute le token à chaque requête, à un seul endroit.
   The interceptor adds the token to every request, in a single place.
3. La réponse revient en 401 : l'interceptor redirige vers la connexion, pour tous les écrans.
   The response comes back as 401: the interceptor redirects to login, for all screens.
4. L'erreur continue jusqu'au service, qui peut afficher un message.
   The error continues up to the service, which can display a message.

Lazy loading

Lazy loading

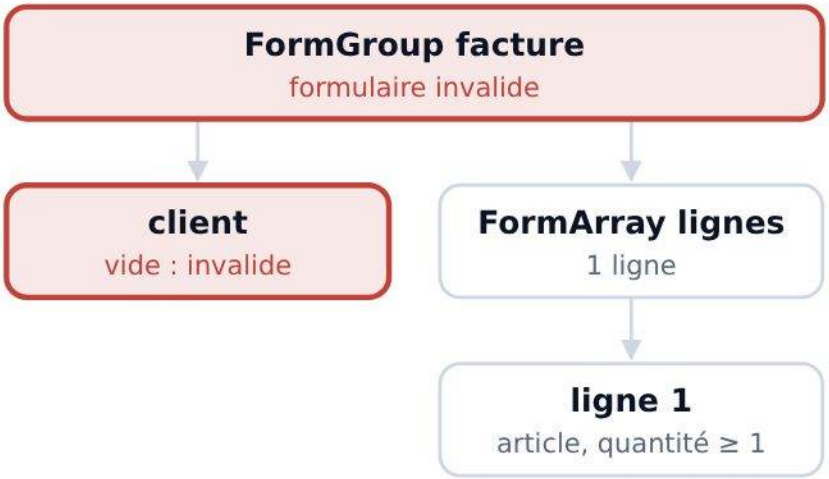


La réponse, étape par étape · *Step-by-step answer*

1. Au démarrage, seul le code de base est téléchargé : premier affichage rapide.
   At startup, only the base code is downloaded: fast first display.
2. L'utilisateur va sur /factures : ce code est téléchargé à ce moment-là.
   The user goes to /factures: that code is downloaded at that moment.
3. S'il ne va jamais sur la paie, ce code n'est jamais téléchargé.
   If they never go to payroll, that code is never downloaded.

Reactive forms

Reactive forms



La réponse, étape par étape · *Step-by-step answer*

1. Le formulaire est décrit en TypeScript : un client et une liste de lignes.
   The form is described in TypeScript: a customer and a list of lines.
2. Un clic sur « Ajouter » crée une ligne, avec ses propres règles.
   A click on "Add" creates a line, with its own rules.
3. Si le client est vide, tout le formulaire est invalide : on désactive le bouton Enregistrer.
   If the customer is empty, the whole form is invalid: we disable the Save button.

Longues listes

Long lists



La réponse, étape par étape · *Step-by-step answer*

1. track dit comment reconnaître chaque élément. Après un rechargement, Angular ne recrée que ce qui a changé.
   track says how to recognize each item. After a reload, Angular recreates only what changed.
2. Afficher 1 000 lignes d’un coup alourdit fortement la page.
   Displaying 1,000 rows at once makes the page much heavier.
3. Le virtual scroll n’affiche que les lignes visibles et les remplace pendant le défilement.
   Virtual scroll displays only the visible rows and swaps them while scrolling.

Pipes

Pipes

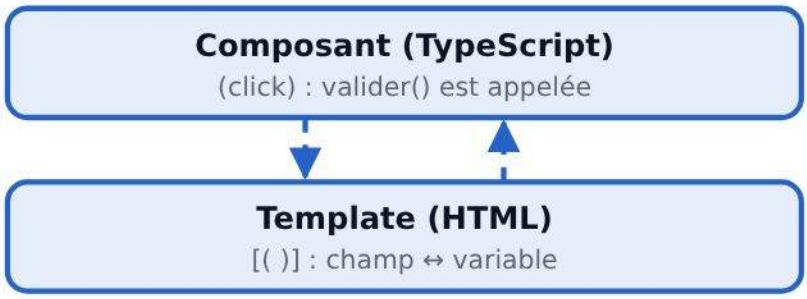


La réponse, étape par étape · *Step-by-step answer*

1. Premier affichage : les deux pipes calculent « 100.000 TND ».
   First display: both pipes calculate "100.000 TND".
2. Nouvelle vérification, total inchangé : le pipe pur ne refait rien, l'impur recalcule.
   New check, total unchanged: the pure pipe does nothing again, the impure one recalculates.
3. La valeur change : le pipe pur recalcule. Pur = calcul seulement quand c'est nécessaire.
   The value changes: the pure pipe recalculates. Pure = calculate only when needed.

Data binding

Data binding

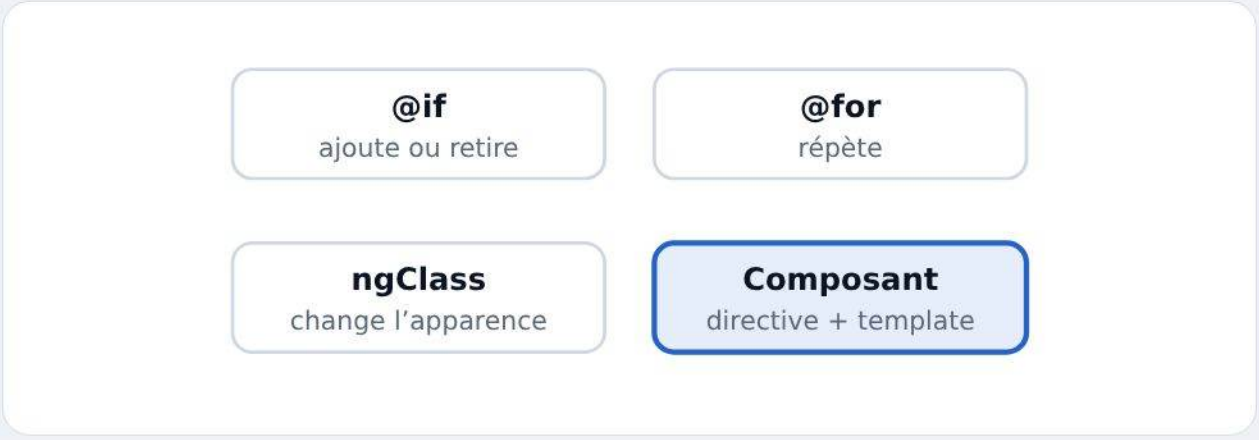


La réponse, étape par étape · *Step-by-step answer*

1. Interpolation : la valeur du code s'affiche dans la page.
   Interpolation: the value from the code is displayed on the page.
2. Property binding : le code contrôle une propriété HTML.
   Property binding: the code controls an HTML property.
3. Event binding : la page prévient le code qu'un événement a eu lieu.
   Event binding: the page tells the code that an event happened.
4. Two-way binding : le champ et la variable restent synchronisés dans les deux sens.
   Two-way binding: the field and the variable stay in sync in both directions.

Directives

Directives

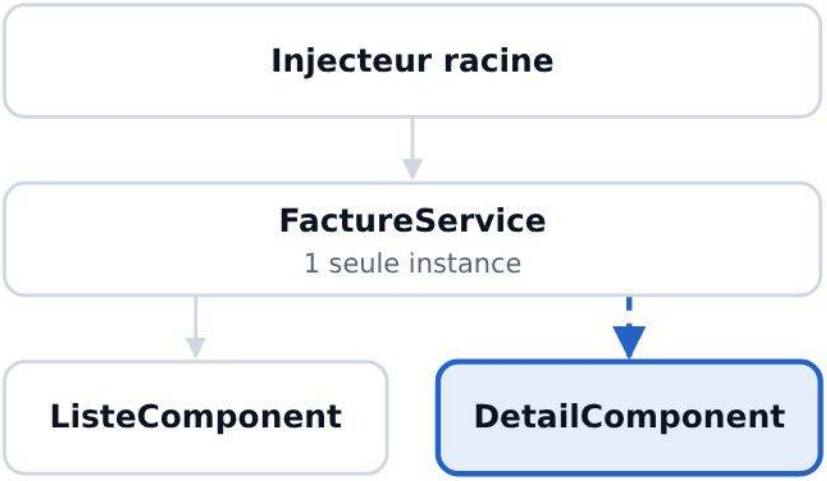


La réponse, étape par étape · *Step-by-step answer*

1. Directive structurelle : @if ajoute ou retire un bloc selon une condition.
   Structural directive: @if adds or removes a block based on a condition.
2. @for répète un bloc pour chaque élément de la liste.
   @for repeats a block for each item in the list.
3. Directive d'attribut : ngClass change le style, sans ajouter ni retirer d'élément.
   Attribute directive: ngClass changes the style, without adding or removing any element.
4. Un composant est une directive avec son propre template.
   A component is a directive with its own template.

Injection Angular

Angular injection

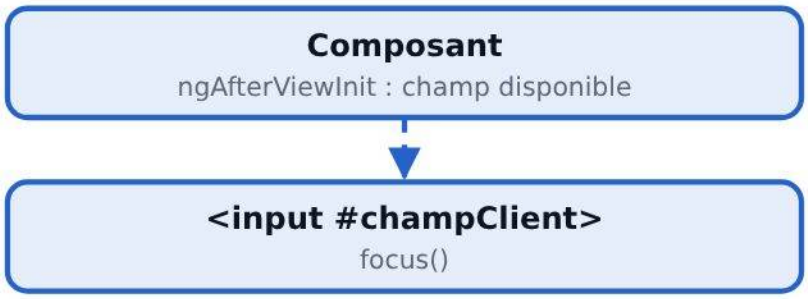


La réponse, étape par étape · *Step-by-step answer*

1. providedIn root : Angular crée un seul FactureService pour toute l'application.
   providedIn root: Angular creates a single FactureService for the whole application.
2. Un composant le demande avec inject()…
   One component asks for it with inject()…
3. …un autre par son constructeur. Les deux reçoivent la même instance, comme un singleton Spring.
   …another through its constructor. Both receive the same instance, like a Spring singleton.

@ViewChild

@ViewChild



La réponse, étape par étape · *Step-by-step answer*

1. #champClient pose une étiquette sur l'élément du template.
   #champClient puts a label on the template element.
2. Dans le constructeur, la vue n'existe pas encore : impossible d'y accéder.
   In the constructor, the view does not exist yet: impossible to access it.
3. Dans ngAfterViewInit, la vue est prête : on peut placer le curseur dans le champ.
   In ngAfterViewInit, the view is ready: we can put the cursor in the field.

Erreurs HTTP

HTTP errors

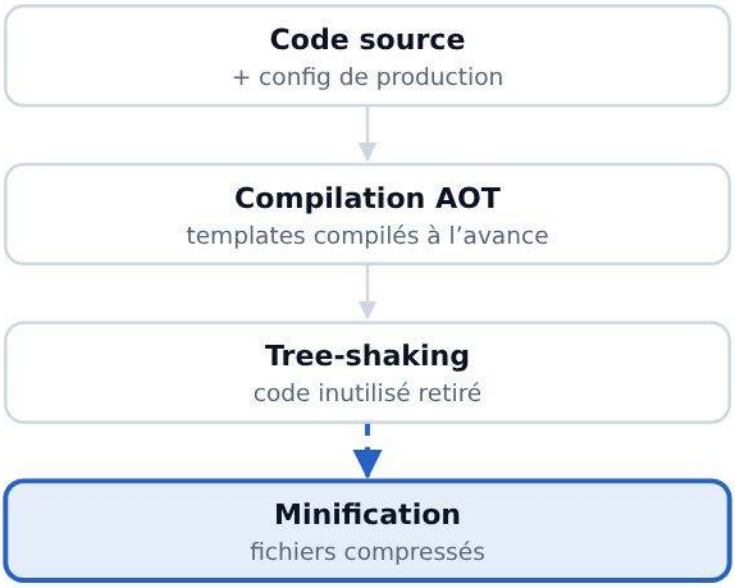


La réponse, étape par étape · *Step-by-step answer*

1. On demande la facture au serveur.
   We ask the server for the invoice.
2. Le serveur répond par une erreur.
   The server responds with an error.
3. catchError attrape l’erreur : on prépare un message et une valeur de repli.
   catchError catches the error: we prepare a message and a fallback value.
4. L’écran affiche un message clair au lieu de planter.
   The screen shows a clear message instead of crashing.

Build de production

Production build

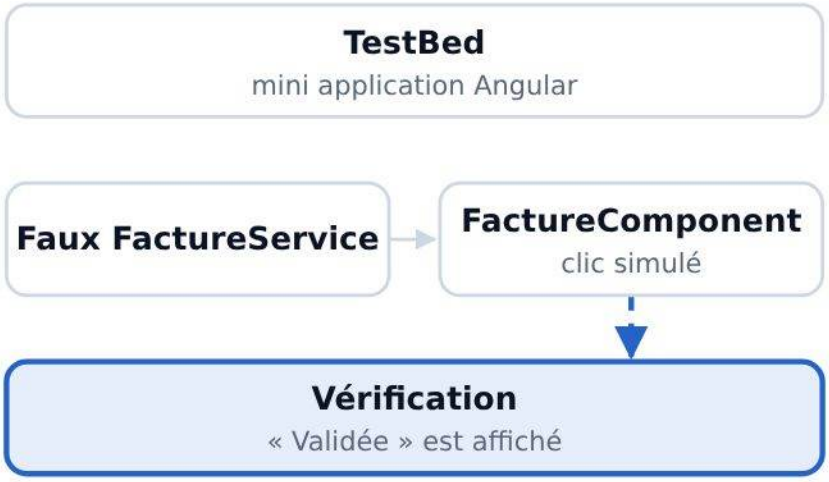


La réponse, étape par étape · *Step-by-step answer*

1. Le code de l'application, avec la config de production (adresse de l'API).
   The application code, with the production config (the API address).
2. Les templates sont compilés à l'avance : le navigateur n'a plus ce travail à faire.
   Templates are compiled ahead of time: the browser no longer has to do this work.
3. Le code jamais utilisé est supprimé.
   Code that is never used is removed.
4. Les noms sont raccourcis et les espaces retirés : des fichiers beaucoup plus légers.
   Names are shortened and spaces removed: much lighter files.

Tester un composant

Testing a component



La réponse, étape par étape · *Step-by-step answer*

1. TestBed prépare un mini environnement Angular juste pour ce composant.
   TestBed sets up a mini Angular environment just for this component.
2. On remplace le vrai service par un faux : pas d'appel HTTP réel.
   We replace the real service with a fake one: no real HTTP call.
3. On crée le composant, on simule un clic et on met l'affichage à jour.
   We create the component, simulate a click and update the display.
4. On vérifie que le texte attendu est bien affiché.
   We check that the expected text is actually displayed.

OnPush et immutabilité

OnPush and immutability

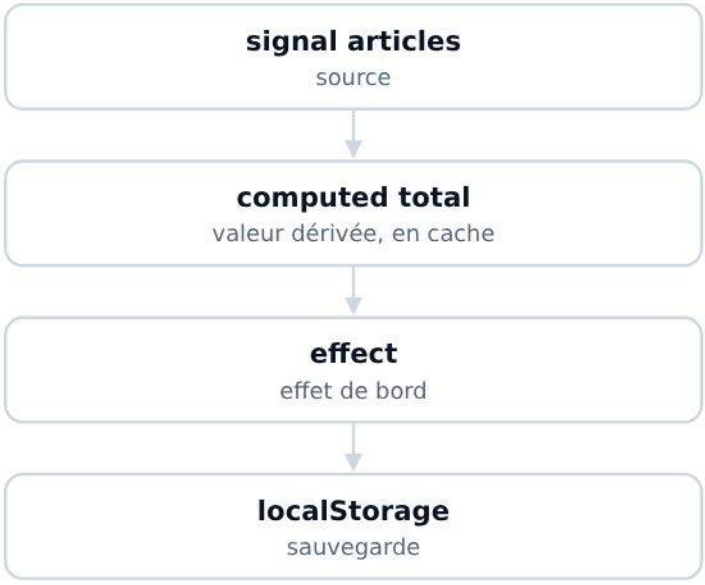


La réponse, étape par étape · *Step-by-step answer*

1. Avec OnPush, Angular ne revérifie le composant que dans certains cas, ce qui économise beaucoup de travail.
   With OnPush, Angular only re-checks the component in specific cases, which saves a lot of work.
2. Modifier le tableau sur place (push) garde la même référence : Angular ne voit pas le changement.
   Changing the array in place (push) keeps the same reference: Angular does not see the change.
3. Créer un nouveau tableau change la référence de l'@Input : le composant est revérifié.
   Creating a new array changes the @Input reference: the component is re-checked.
4. Autres déclencheurs : un événement dans le composant, l'async pipe, ou un signal lu dans le template qui change.
   Other triggers: an event inside the component, the async pipe, or a signal read in the template that changes.

computed et effect

computed and effect

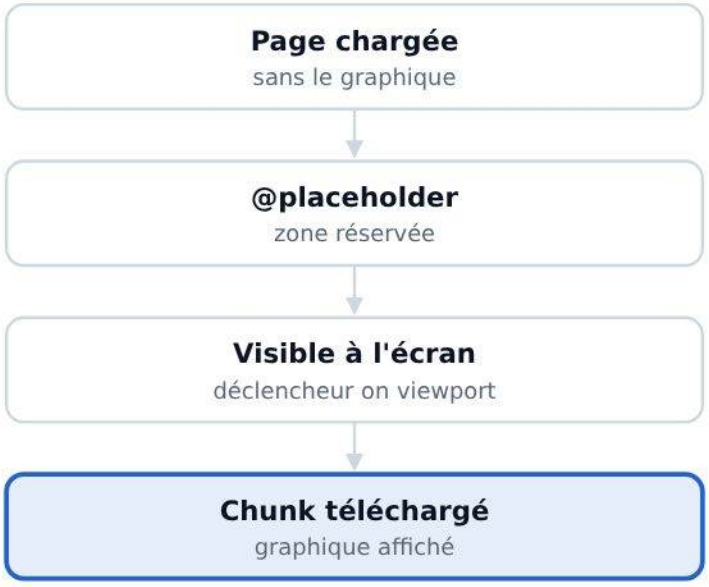


La réponse, étape par étape · *Step-by-step answer*

1. articles est un signal : une valeur qui prévient quand elle change.
   articles is a signal: a value that notifies when it changes.
2. computed calcule total à partir d'articles. Il est mis en cache et recalculé seulement si articles change.
   computed derives total from articles. It is cached and recalculated only when articles changes.
3. effect sert aux effets de bord : ici, sauvegarder le panier à chaque modification.
   effect is for side effects: here, saving the cart on every change.
4. Règle : pour obtenir une valeur → computed (lecture seule). Pour agir sur l'extérieur → effect. Éviter de modifier d'autres signals dans un effect.
   Rule: to get a value → computed (read-only). To act on the outside world → effect. Avoid setting other signals in an effect.

@defer

@defer

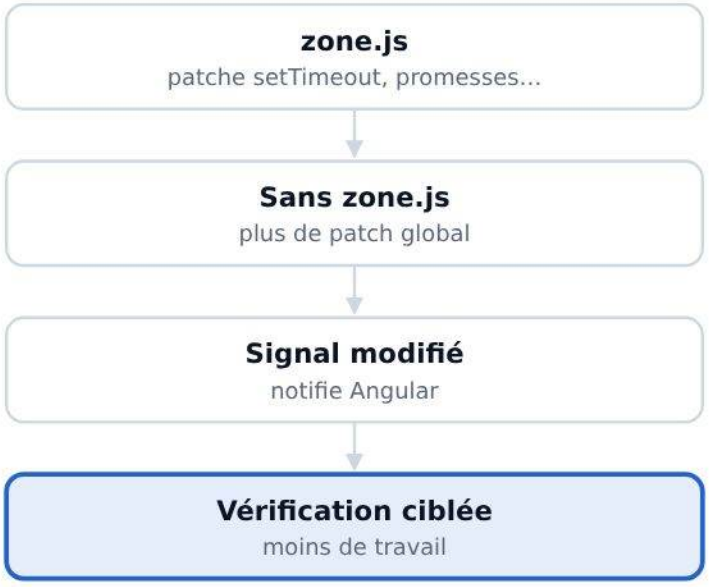


La réponse, étape par étape · *Step-by-step answer*

1. La page s'affiche d'abord avec son contenu léger.
   The page first renders with its light content.
2. À la place du graphique, Angular montre le @placeholder.
   Instead of the chart, Angular shows the @placeholder.
3. Quand la zone arrive à l'écran, le déclencheur on viewport télécharge le code du graphique (un fichier séparé), avec @loading pendant l'attente.
   When the area scrolls into view, the on viewport trigger downloads the chart's code (a separate file), with @loading meanwhile.
4. Le bundle initial est plus léger : la page s'ouvre plus vite. Autres déclencheurs : on interaction, on idle, on timer.
   The initial bundle is lighter: the page opens faster. Other triggers: on interaction, on idle, on timer.

Sans zone.js

Zoneless

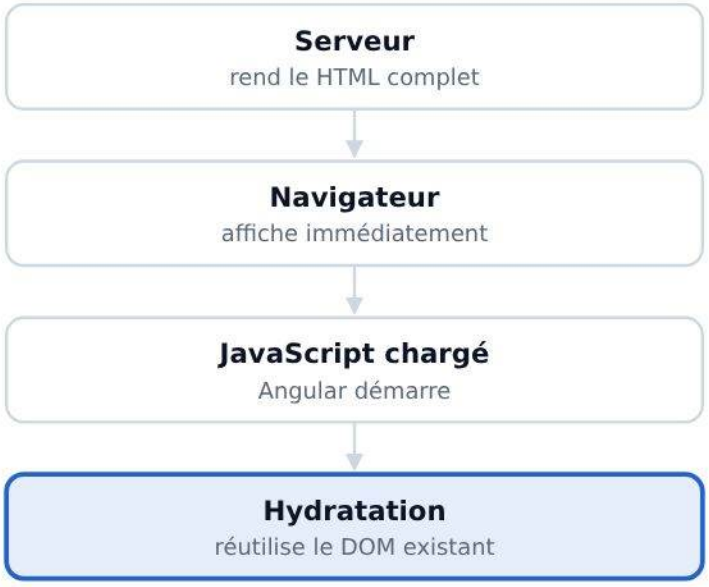


La réponse, étape par étape · *Step-by-step answer*

1. Avec zone.js, Angular est prévenu après chaque setTimeout, promesse ou événement, et vérifie souvent toute l'application.
   With zone.js, Angular is notified after every setTimeout, promise or event, and often checks the whole app.
2. En mode zoneless, ce patch global disparaît : moins de code chargé et des traces d'erreur plus lisibles.
   In zoneless mode, this global patching disappears: less code loaded and clearer stack traces.
3. Angular réagit aux notifications explicites : un signal qui change, un événement du template, l'async pipe.
   Angular reacts to explicit notifications: a changing signal, a template event, the async pipe.
4. Résultat : moins de vérifications inutiles. Condition : écrire l'état avec des signals (ou markForCheck) plutôt que muter des variables en silence.
   Result: fewer useless checks. Requirement: keep state in signals (or call markForCheck) instead of silently mutating fields.

SSR et hydratation

SSR and hydration

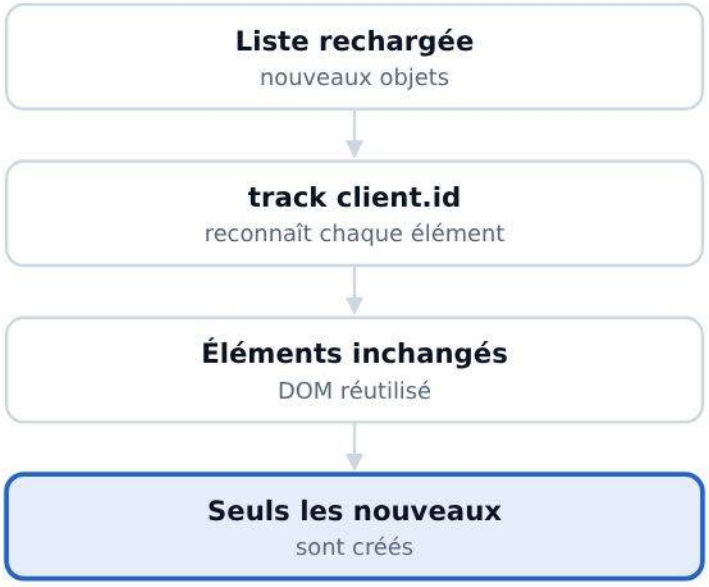


La réponse, étape par étape · *Step-by-step answer*

1. Avec le SSR, le serveur exécute Angular et envoie une page HTML déjà remplie.
   With SSR, the server runs Angular and sends an already filled HTML page.
2. Le visiteur voit le contenu tout de suite, et les moteurs de recherche peuvent le lire (SEO).
   The visitor sees the content immediately, and search engines can read it (SEO).
3. Ensuite, le JavaScript arrive et Angular démarre dans le navigateur.
   Then the JavaScript arrives and Angular starts in the browser.
4. L'hydratation rattache les composants au HTML existant au lieu de tout effacer et redessiner : pas de clignotement, page interactive plus vite.
   Hydration attaches components to the existing HTML instead of wiping and redrawing it: no flicker, interactive sooner.

track dans @for

track in @for

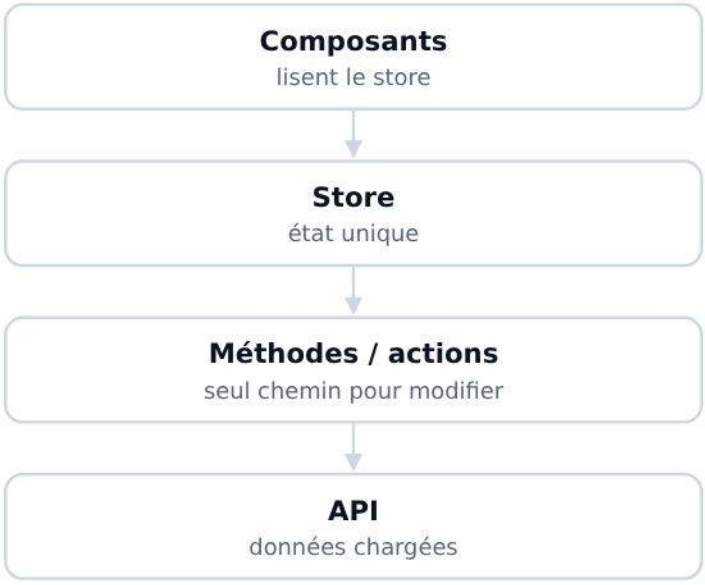


La réponse, étape par étape · *Step-by-step answer*

1. Après un rechargement, la liste contient de nouveaux objets, même si les clients sont les mêmes.
   After a reload, the list holds new objects, even if the clients are the same.
2. track client.id dit à Angular comment reconnaître un élément d'une liste à l'autre.
   track client.id tells Angular how to recognise an item from one list to the next.
3. Les clients déjà affichés gardent leur DOM et leur état (champ saisi, animation).
   Clients already shown keep their DOM and state (typed input, animation).
4. Seuls les éléments ajoutés ou retirés touchent le DOM : la liste reste fluide, même longue. @empty gère la liste vide.
   Only added or removed items touch the DOM: the list stays smooth, even when long. @empty handles the empty list.

Gestion d'état

State management



La réponse, étape par étape · *Step-by-step answer*

1. L'état partagé (clients, chargement) vit à un seul endroit : le store.
   Shared state (clients, loading) lives in one place: the store.
2. Les composants lisent cet état ; plusieurs écrans voient toujours les mêmes données.
   Components read this state; several screens always see the same data.
3. Pour le modifier, on passe par les méthodes (ou les actions avec NgRx) : on sait toujours qui change quoi.
   To change it, go through the methods (or actions with NgRx): you always know who changes what.
4. Plus facile à déboguer et à tester dans une grosse application. Pour une petite, un service avec des signals suffit.
   Easier to debug and test in a large application. For a small one, a service with signals is enough.

Observable ou Promise

Observable or Promise

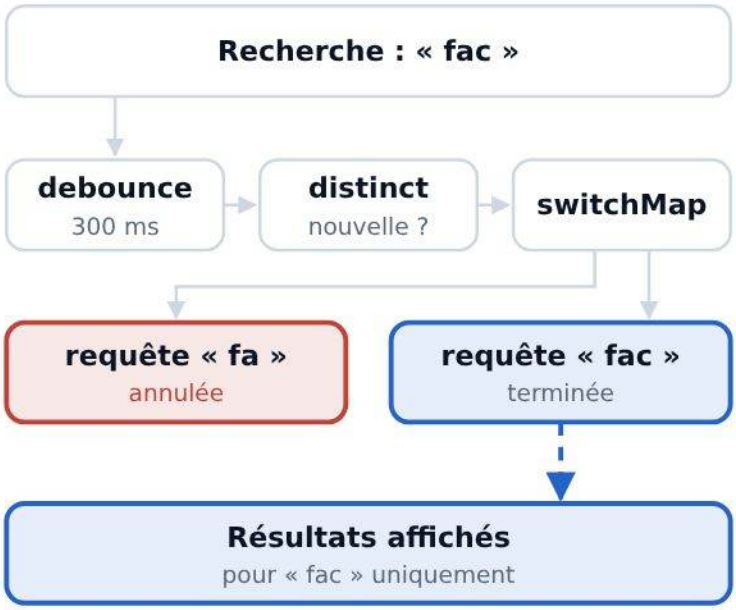


La réponse, étape par étape · *Step-by-step answer*

1. Une Promise démarre dès sa création.
   A Promise starts as soon as it is created.
2. Un Observable ne fait rien tant que personne ne s'abonne.
   An Observable does nothing as long as nobody subscribes.
3. subscribe le démarre.
   subscribe starts it.
4. Un Observable peut envoyer plusieurs valeurs dans le temps. Une Promise, une seule.
   An Observable can send several values over time. A Promise, only one.
5. Et on peut l'annuler avec unsubscribe. Une Promise, non.
   And it can be cancelled with unsubscribe. A Promise cannot.

switchMap

switchMap

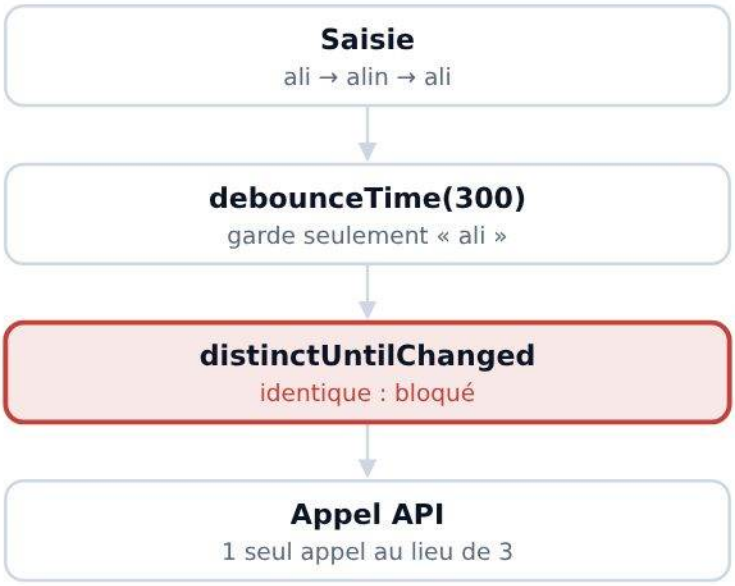


La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur tape « f » dans la barre de recherche.
   The user types "f" in the search bar.
2. Il tape vite « fa ». debounceTime attend une petite pause (300 ms) avant d'agir. « f » n'est donc jamais envoyé.
   They quickly type "fa". debounceTime waits for a short pause (300 ms) before acting. So "f" is never sent.
3. Pause terminée : la recherche « fa » est lancée.
   Pause over: the search for "fa" starts.
4. Pendant que « fa » cherche encore, l'utilisateur tape « fac ».
   While "fa" is still searching, the user types "fac".
5. switchMap annule l'ancienne recherche « fa » et lance « fac ». On garde seulement la plus récente.
   switchMap cancels the old "fa" search and starts "fac". We keep only the most recent one.
6. Seuls les résultats de « fac » s'affichent : c'est exactement ce que l'utilisateur veut voir.
   Only the "fac" results are shown: exactly what the user wants to see.

Autocomplétion

Autocomplete



La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur tape « ali » rapidement : 3 valeurs.
   The user quickly types "ali": 3 values.
2. debounceTime attend 300 ms de calme : seule la dernière valeur passe.
   debounceTime waits for 300 ms of calm: only the last value goes through.
3. distinctUntilChanged bloque une valeur identique à la précédente.
   distinctUntilChanged blocks a value identical to the previous one.
4. switchMap lance l'appel. Résultat : 1 requête au lieu de 3.
   switchMap makes the call. Result: 1 request instead of 3.
5. S'il tape une lettre puis l'efface, la valeur redevient « ali » : aucun appel inutile.
   If they type a letter then delete it, the value is "ali" again: no useless call.

Les Subjects

Subjects



La réponse, étape par étape · *Step-by-step answer*

1. Subject n'a pas de valeur au départ. BehaviorSubject en a toujours une.
   Subject has no value at the start. BehaviorSubject always has one.
2. Les deux envoient « Ali ».
   Both send "Ali".
3. Un abonné arrivé après coup ne reçoit rien du Subject.
   A subscriber that arrives later receives nothing from the Subject.
4. Avec BehaviorSubject, il reçoit tout de suite la dernière valeur : idéal pour l'utilisateur connecté.
   With BehaviorSubject, it immediately receives the last value: ideal for the logged-in user.
5. ReplaySubject rejoue les N dernières valeurs à chaque nouvel abonné.
   ReplaySubject replays the last N values to each new subscriber.

Fuites mémoire

Memory leaks

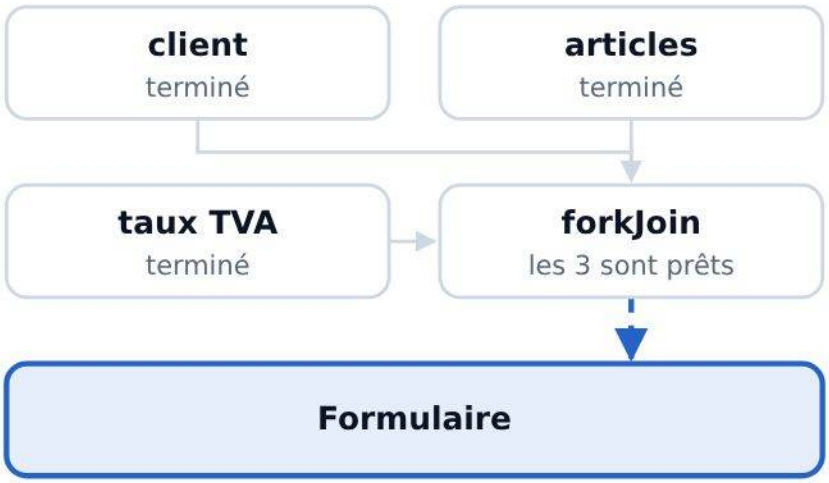


La réponse, étape par étape · *Step-by-step answer*

1. Deux abonnements, toutes les secondes.
   Two subscriptions, every second.
2. L'utilisateur quitte l'écran : le composant est détruit.
   The user leaves the screen: the component is destroyed.
3. Le premier continue pour rien. À chaque visite, un nouveau s'ajoute : fuite mémoire.
   The first one keeps running for nothing. On each visit, a new one is added: memory leak.
4. takeUntilDestroyed (ou le pipe async) arrête l'abonnement tout seul.
   takeUntilDestroyed (or the async pipe) stops the subscription on its own.

forkJoin

forkJoin



La réponse, étape par étape · *Step-by-step answer*

1. Trois appels partent en même temps.
   Three calls go out at the same time.
2. Deux sont terminés. forkJoin attend encore le dernier.
   Two are finished. forkJoin is still waiting for the last one.
3. Le dernier arrive : forkJoin envoie les 3 résultats ensemble.
   The last one arrives: forkJoin sends the 3 results together.
4. On ouvre le formulaire une seule fois, avec toutes les données.
   We open the form only once, with all the data.

catchError et retry

catchError and retry



La réponse, étape par étape · *Step-by-step answer*

1. Premier essai : échec.
   First attempt: failure.
2. retry(2) réessaie automatiquement…
   retry(2) automatically tries again…
3. …une deuxième fois. Toujours en échec.
   …a second time. Still failing.
4. catchError prend le relais avec une valeur de repli : l'écran continue de fonctionner.
   catchError takes over with a fallback value: the screen keeps working.

Froid ou chaud

Cold or hot

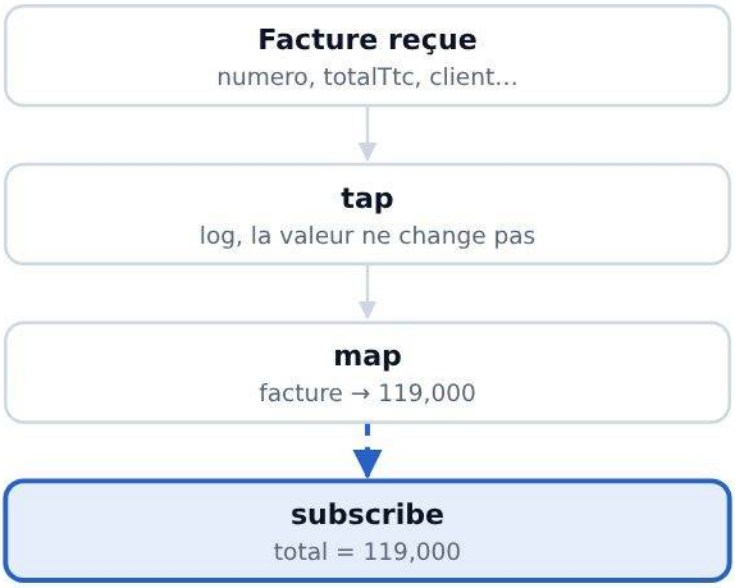


La réponse, étape par étape · *Step-by-step answer*

1. Observable froid : le premier abonné déclenche une requête.
   Cold Observable: the first subscriber triggers a request.
2. Le deuxième abonné déclenche une autre requête, identique. Gaspillage.
   The second subscriber triggers another, identical request. Waste.
3. Avec shareReplay(1), la requête est partagée.
   With shareReplay(1), the request is shared.
4. Le deuxième abonné reçoit le résultat déjà obtenu, sans nouvel appel.
   The second subscriber receives the result already obtained, with no new call.

map ou tap

map or tap



La réponse, étape par étape · *Step-by-step answer*

1. Le serveur renvoie une facture complète.
   The server returns a full invoice.
2. tap fait une action à côté (un log) et laisse passer la facture telle quelle.
   tap does a side action (a log) and lets the invoice through as is.
3. map transforme : on ne garde que le total.
   map transforms: we keep only the total.
4. L'abonné reçoit directement le total.
   The subscriber receives the total directly.

subscribe imbriqués

Nested subscribes

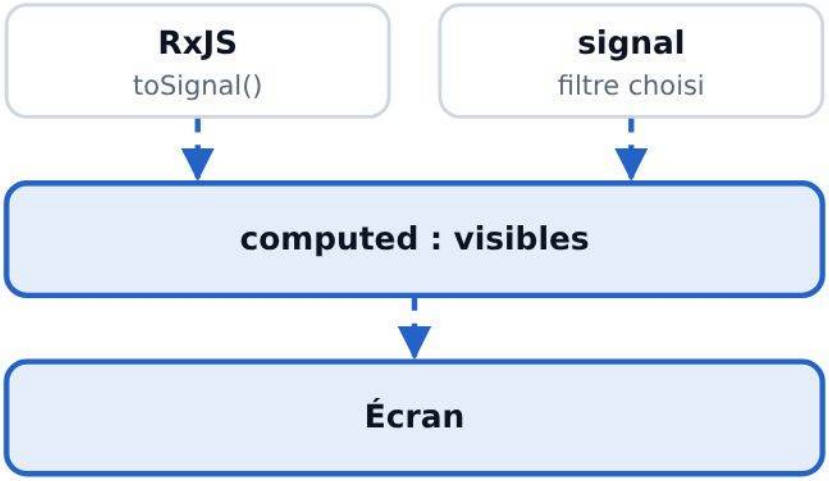


La réponse, étape par étape · *Step-by-step answer*

1. Un subscribe dans un subscribe : difficile à lire, et celui de l'intérieur n'est jamais annulé.
   A subscribe inside a subscribe: hard to read, and the inner one is never cancelled.
2. switchMap enchaîne les deux appels dans un seul flux.
   switchMap chains the two calls into a single stream.
3. Un seul abonnement, géré par le pipe async : rien à fermer à la main.
   A single subscription, managed by the async pipe: nothing to close by hand.

Signals ou RxJS

Signals or RxJS

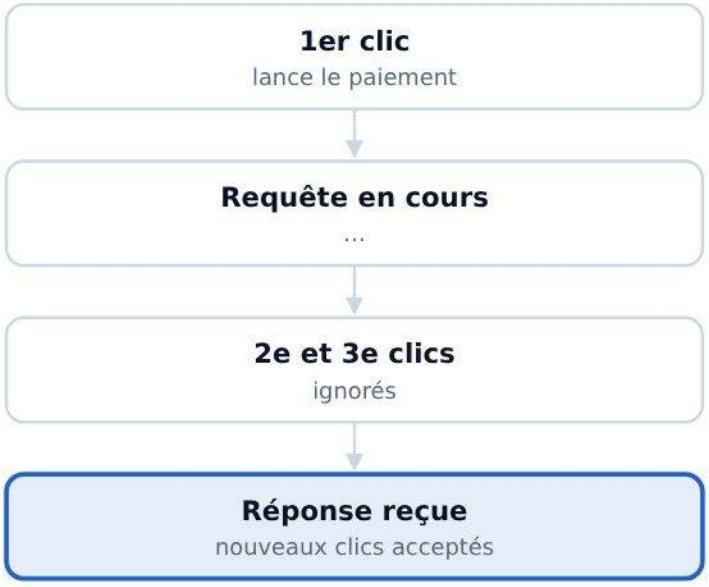


La réponse, étape par étape · *Step-by-step answer*

1. Les données arrivent du serveur : c'est un flux asynchrone, le domaine de RxJS.
   Data arrives from the server: it is an asynchronous stream, the domain of RxJS.
2. toSignal transforme ce flux en signal.
   toSignal turns this stream into a signal.
3. Le filtre choisi par l'utilisateur est un simple état : un signal suffit.
   The filter chosen by the user is simple state: a signal is enough.
4. computed combine les deux et l'écran se met à jour tout seul.
   computed combines the two and the screen updates on its own.

exhaustMap

exhaustMap

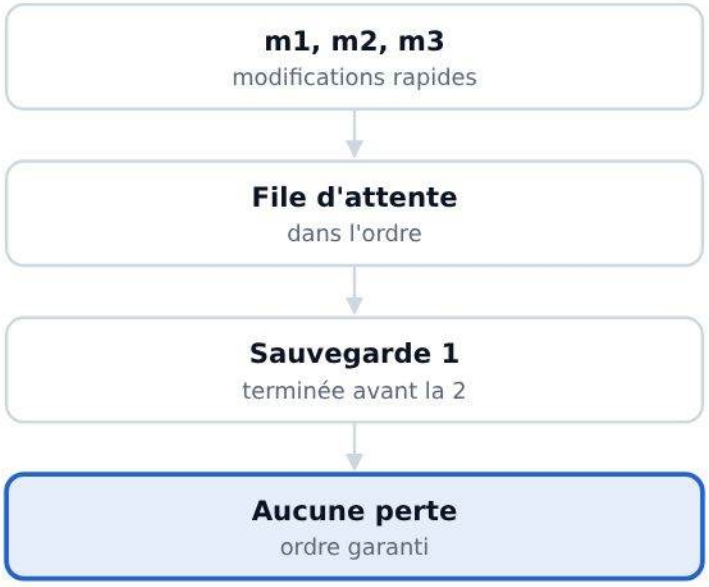


La réponse, étape par étape · *Step-by-step answer*

1. Premier clic : exhaustMap lance la requête de paiement.
   First click: exhaustMap starts the payment request.
2. Tant que la requête n'est pas terminée, exhaustMap est « occupé ».
   While the request is running, exhaustMap is "busy".
3. Les clics suivants sont simplement ignorés : pas de double paiement. switchMap, lui, annulerait la requête en cours.
   The next clicks are simply ignored: no double payment. switchMap would cancel the running request.
4. Une fois la réponse reçue, un nouveau clic est de nouveau pris en compte. Idéal pour connexion, paiement, envoi de formulaire.
   Once the response arrives, a new click is accepted again. Ideal for login, payment, form submission.

concatMap

concatMap

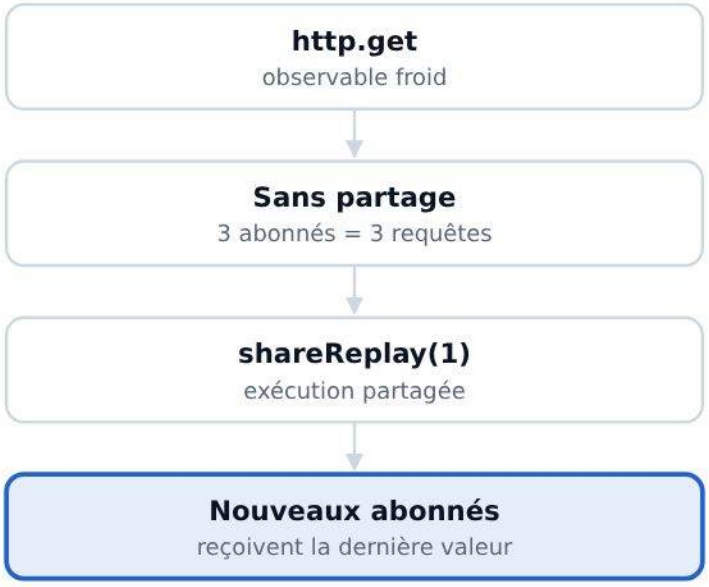


La réponse, étape par étape · *Step-by-step answer*

1. L'utilisateur fait trois modifications coup sur coup.
   The user makes three edits in a row.
2. concatMap les met en file : il attend la fin d'une sauvegarde avant de lancer la suivante.
   concatMap queues them: it waits for one save to finish before starting the next.
3. La sauvegarde de m2 ne peut donc jamais arriver avant celle de m1.
   So m2's save can never arrive before m1's.
4. Rien n'est perdu ni annulé. switchMap annulerait, exhaustMap ignorerait, mergeMap lancerait tout en parallèle sans ordre.
   Nothing is lost or cancelled. switchMap would cancel, exhaustMap would ignore, mergeMap would run everything in parallel without order.

shareReplay

shareReplay



La réponse, étape par étape · *Step-by-step answer*

1. Un observable HTTP est froid : chaque abonnement relance la requête.
   An HTTP observable is cold: each subscription restarts the request.
2. Trois composants qui s'abonnent provoquent donc trois appels identiques.
   Three subscribing components therefore trigger three identical calls.
3. shareReplay partage une seule exécution entre tous les abonnés.
   shareReplay shares a single execution among all subscribers.
4. Il garde la dernière valeur et la rejoue aux abonnés suivants. refCount: true libère l'abonnement quand plus personne n'écoute.
   It keeps the last value and replays it to later subscribers. refCount: true releases the subscription when nobody listens anymore.

combineLatest

combineLatest

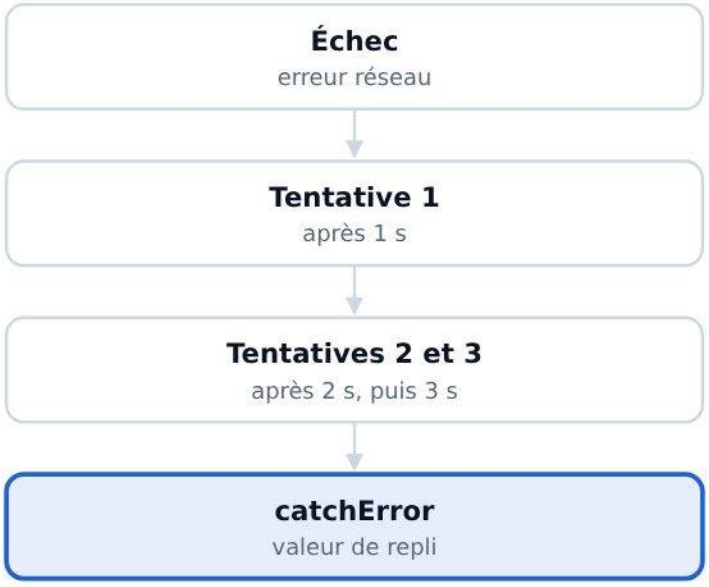


La réponse, étape par étape · *Step-by-step answer*

1. filtre$ émet une première valeur, mais combineLatest attend encore.
   filter$ emits a first value, but combineLatest still waits.
2. Dès que page$ a aussi émis, combineLatest émet [filtre, page].
   As soon as page$ has also emitted, combineLatest emits [filter, page].
3. Ensuite, chaque nouvelle valeur de l'un ou de l'autre réémet la combinaison avec la dernière valeur de l'autre.
   Then every new value from either one re-emits the combination with the other's latest value.
4. Si une source n'émet jamais, rien ne sort : donner une valeur de départ (startWith). forkJoin, lui, attend que toutes les sources soient terminées.
   If a source never emits, nothing comes out: give it a start value (startWith). forkJoin waits for all sources to complete.

Retry avec délai

Retry with backoff



La réponse, étape par étape · *Step-by-step answer*

1. La requête échoue (réseau instable, serveur surchargé).
   The request fails (unstable network, overloaded server).
2. retry relance la requête après un délai calculé : 1 seconde pour la 1re tentative.
   retry re-runs the request after a computed delay: 1 second for the first attempt.
3. Le délai grandit à chaque essai : on laisse au serveur le temps de récupérer au lieu de le bombarder.
   The delay grows with each try: the server gets time to recover instead of being hammered.
4. Après 3 échecs, catchError fournit une valeur de repli. Ne réessayer que les erreurs temporaires et les requêtes idempotentes (GET).
   After 3 failures, catchError provides a fallback value. Only retry transient errors and idempotent requests (GET).

API REST

REST API



La réponse, étape par étape · *Step-by-step answer*

1. Chaque ressource a une adresse. GET /api/factures renvoie la liste en JSON.
   Each resource has an address. GET /api/factures returns the list as JSON.
2. Le verbe dit l'action, l'adresse dit sur quoi.
   The verb says the action, the address says on what.
3. Sans état : chaque requête contient tout ce qu'il faut (dont le token). Le serveur ne se souvient de rien entre deux appels.
   Stateless: each request contains everything needed (including the token). The server remembers nothing between two calls.

Verbes HTTP

HTTP verbs

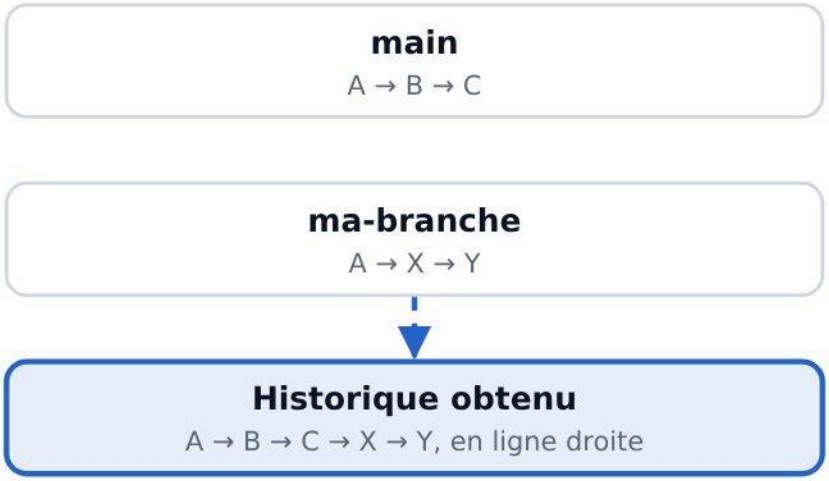


La réponse, étape par étape · *Step-by-step answer*

1. PUT est idempotent : le répéter donne le même état.
   PUT is idempotent: repeating it gives the same state.
2. DELETE aussi : la facture reste supprimée.
   So is DELETE: the invoice stays deleted.
3. POST n'est pas idempotent : un double clic crée deux factures. D'où l'intérêt de désactiver le bouton.
   POST is not idempotent: a double click creates two invoices. Hence the point of disabling the button.
4. PATCH modifie une partie. PUT remplace la ressource entière.
   PATCH modifies a part. PUT replaces the entire resource.

merge ou rebase

merge or rebase



La réponse, étape par étape · *Step-by-step answer*

1. main a avancé (B, C) pendant que tu travaillais (X, Y).
   main moved forward (B, C) while you were working (X, Y).
2. merge réunit les deux avec un commit de fusion : l'historique réel est conservé.
   merge joins the two with a merge commit: the real history is kept.
3. rebase rejoue tes commits après C : historique en ligne droite. À éviter sur une branche déjà partagée.
   rebase replays your commits after C: a straight-line history. Avoid it on a branch that is already shared.

Conflit Git

Git conflict

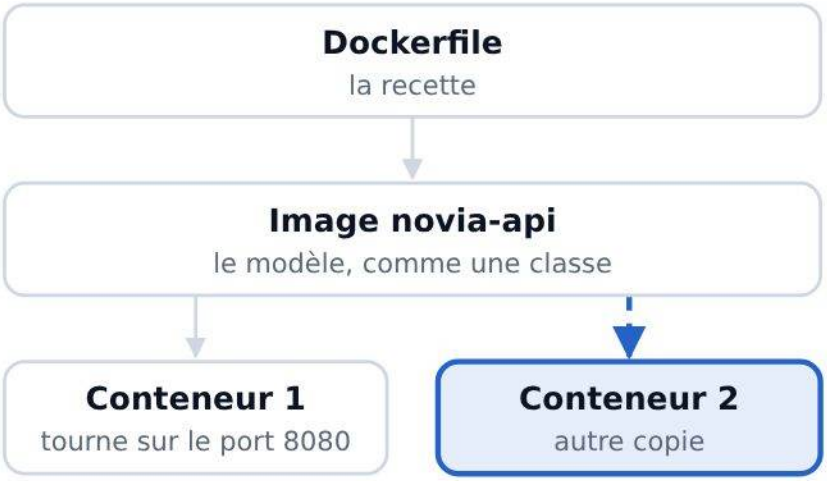


La réponse, étape par étape · *Step-by-step answer*

1. Deux personnes ont modifié la même ligne : Git ne peut pas choisir seul.
   Two people changed the same line: Git cannot choose on its own.
2. On compare les deux versions, et on vérifie avec le collègue ou le besoin laquelle est juste.
   We compare the two versions, and check with the colleague or the requirement which one is right.
3. On garde la bonne version et on supprime les lignes <<<<, ==== et >>>>.
   We keep the right version and delete the <<<<, ==== and >>>> lines.
4. On teste, puis git add et commit pour terminer.
   We test, then git add and commit to finish.

Image et conteneur

Image and container

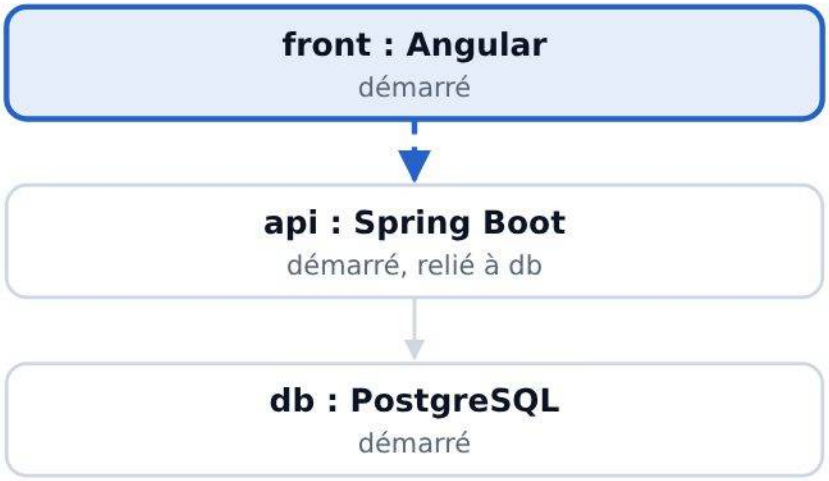


La réponse, étape par étape · *Step-by-step answer*

1. Le Dockerfile est la recette : partir de Java 21, copier le jar, le lancer.
   The Dockerfile is the recipe: start from Java 21, copy the jar, run it.
2. docker build fabrique l'image : un paquet figé qui contient tout.
   docker build makes the image: a frozen package that contains everything.
3. docker run crée un conteneur : l'image qui tourne, comme un objet créé depuis une classe.
   docker run creates a container: the running image, like an object created from a class.
4. On peut lancer plusieurs conteneurs à partir de la même image.
   We can start several containers from the same image.

Docker Compose

Docker Compose



La réponse, étape par étape · *Step-by-step answer*

1. Un seul fichier décrit les trois services.
   A single file describes the three services.
2. docker compose up démarre d'abord la base…
   docker compose up first starts the database…
3. …puis l'API, qui dépend de la base…
   …then the API, which depends on the database…
4. …puis le front. Tout l'environnement tourne avec une seule commande.
   …then the front end. The whole environment runs with a single command.

Monolithe ou microservices

Monolith or microservices

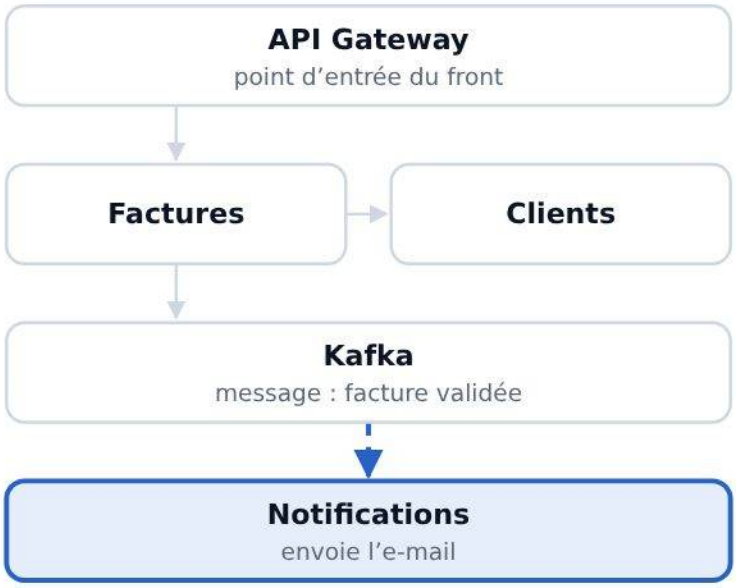


La réponse, étape par étape · *Step-by-step answer*

1. Monolithe : tout est dans une seule application. Simple à développer, tester et déployer.
   Monolith: everything is in a single application. Simple to develop, test and deploy.
2. Microservices : chaque partie est une application séparée.
   Microservices: each part is a separate application.
3. Avantage : on peut modifier et déployer la paie sans toucher aux factures.
   Advantage: we can change and deploy payroll without touching invoices.
4. Inconvénient : réseau, plusieurs bases, surveillance… beaucoup plus complexe. On commence souvent par un monolithe bien organisé.
   Drawback: network, several databases, monitoring… much more complex. We often start with a well-organized monolith.

Communication

Communication

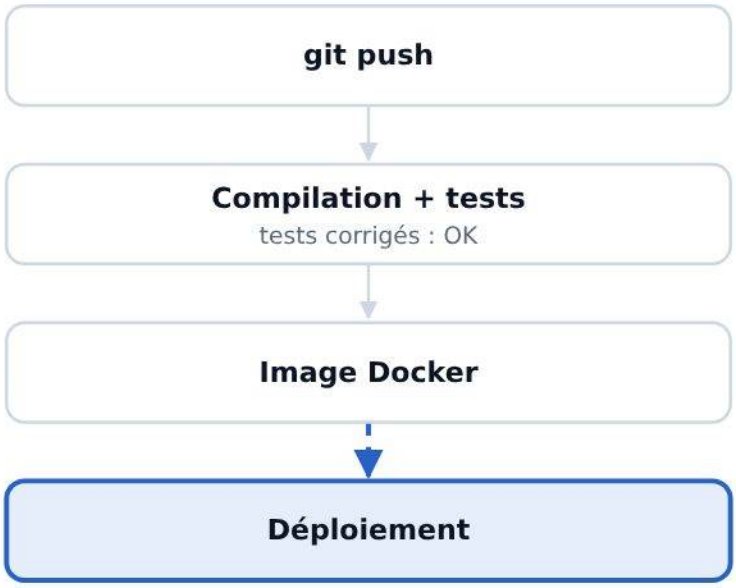


La réponse, étape par étape · *Step-by-step answer*

1. Le front passe par la Gateway, qui envoie vers le bon service.
   The front end goes through the Gateway, which routes to the right service.
2. REST : Factures demande le client et attend la réponse.
   REST: Factures asks for the customer and waits for the answer.
3. Message : Factures publie « facture validée » et continue sans attendre.
   Message: Factures publishes "invoice validated" and continues without waiting.
4. Notifications lit le message quand il peut. Si ce service est en panne, le message attend.
   Notifications reads the message when it can. If this service is down, the message waits.

CI/CD

CI/CD

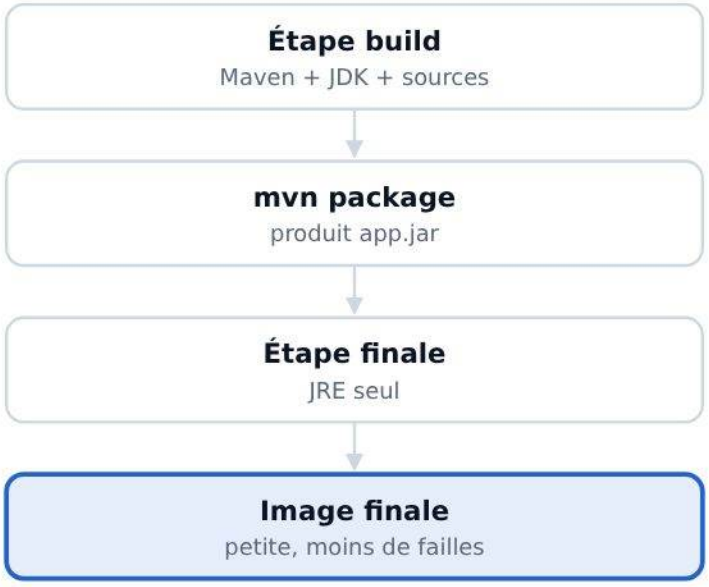


La réponse, étape par étape · *Step-by-step answer*

1. Tu envoies ton code.
   You push your code.
2. CI : le serveur compile et lance tous les tests automatiquement.
   CI: the server compiles and runs all the tests automatically.
3. Si un test échoue, tout s'arrête et tu es prévenu. Rien de cassé n'arrive en production.
   If a test fails, everything stops and you are notified. Nothing broken reaches production.
4. Une fois les tests verts, l'image Docker est construite.
   Once the tests are green, the Docker image is built.
5. CD : la nouvelle version est déployée automatiquement.
   CD: the new version is deployed automatically.

Build multi-étapes

Multi-stage build

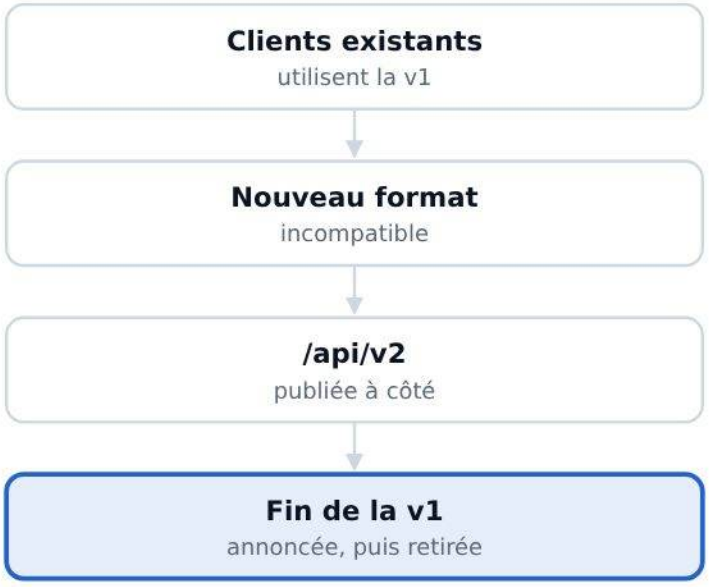


La réponse, étape par étape · *Step-by-step answer*

1. La première étape utilise une image Maven avec le JDK et copie les sources.
   The first stage uses a Maven image with the JDK and copies the sources.
2. Elle compile et produit le JAR.
   It compiles and produces the JAR.
3. La deuxième étape part d'une image JRE légère et ne copie que le JAR.
   The second stage starts from a light JRE image and copies only the JAR.
4. Maven, le JDK et les sources ne sont pas dans l'image finale : elle est plusieurs fois plus petite et expose moins de failles.
   Maven, the JDK and the sources are not in the final image: it is several times smaller and exposes fewer vulnerabilities.

Versionner une API

API versioning



La réponse, étape par étape · *Step-by-step answer*

1. Des applications utilisent déjà /api/v1 avec un champ nom unique.
   Applications already use /api/v1 with a single name field.
2. Séparer prénom et nom casserait ces applications si on modifiait la v1.
   Splitting first and last name would break them if v1 were changed.
3. On publie une v2 à côté : les nouveaux clients l'utilisent, les anciens continuent avec la v1.
   Publish a v2 alongside: new clients use it, old ones keep v1.
4. On annonce une date de fin pour la v1, puis on la retire. Ajouter un champ optionnel, lui, ne demande pas de nouvelle version.
   Announce an end date for v1, then remove it. Adding an optional field does not need a new version.

Déploiement blue-green

Blue-green deployment

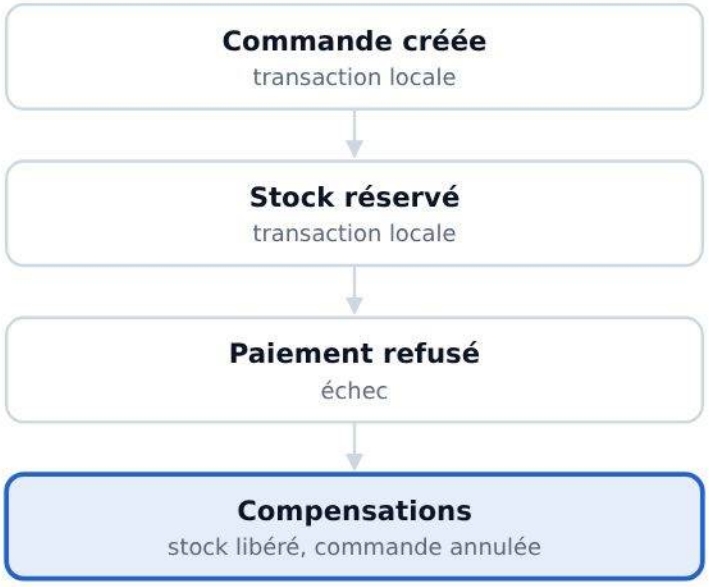


La réponse, étape par étape · *Step-by-step answer*

1. L'environnement blue sert tous les utilisateurs avec la version actuelle.
   The blue environment serves all users with the current version.
2. On déploie la nouvelle version sur green, identique, et on la teste sans impacter personne.
   Deploy the new version to green, an identical environment, and test it without affecting anyone.
3. Le load balancer bascule tout le trafic vers green d'un coup : pas d'interruption.
   The load balancer switches all traffic to green at once: no downtime.
4. En cas de problème, on rebascule vers blue en quelques secondes. Point d'attention : la base de données doit rester compatible avec les deux versions.
   If something breaks, switch back to blue in seconds. Watch out: the database must stay compatible with both versions.

Saga

Saga



La réponse, étape par étape · *Step-by-step answer*

1. Chaque service a sa propre base : une transaction unique sur tous n'est pas possible.
   Each service has its own database: one transaction across all of them is not possible.
2. La saga enchaîne des transactions locales : le service commande crée la commande, puis le service stock réserve les articles.
   The saga chains local transactions: the order service creates the order, then the stock service reserves the items.
3. Le paiement est refusé : les étapes déjà validées ne peuvent pas être « rollbackées ».
   The payment is refused: the steps already committed cannot be rolled back.
4. On exécute des actions de compensation dans l'ordre inverse : libérer le stock, annuler la commande. Orchestrée par un coordinateur ou par événements.
   Compensating actions run in reverse order: release the stock, cancel the order. Driven by an orchestrator or by events.

cherry-pick

cherry-pick



La réponse, étape par étape · *Step-by-step answer*

1. develop contient le correctif, mais aussi des fonctionnalités pas encore prêtes.
   develop holds the fix, but also features that are not ready yet.
2. On repère l'identifiant du commit du correctif.
   Find the fix commit's identifier.
3. Sur main, git cherry-pick applique uniquement ce commit (avec un nouvel identifiant).
   On main, git cherry-pick applies only that commit (with a new identifier).
4. La production reçoit le correctif sans le reste de develop. Un merge aurait tout apporté.
   Production gets the fix without the rest of develop. A merge would have brought everything.

Probes et healthcheck

Probes and healthcheck



La réponse, étape par étape · *Step-by-step answer*

1. Spring Boot expose deux contrôles via Actuator : readiness et liveness.
   Spring Boot exposes two checks through Actuator: readiness and liveness.
2. Au démarrage (ou si la base est indisponible), readiness échoue : Kubernetes n'envoie pas de trafic à ce pod.
   At startup (or if the database is down), readiness fails: Kubernetes sends no traffic to this pod.
3. Dès que readiness répond OK, le pod reçoit les requêtes.
   As soon as readiness reports OK, the pod receives requests.
4. Si l'application est bloquée, liveness échoue : Kubernetes redémarre le conteneur. Ne pas tester la base dans liveness, sinon une panne de base redémarre tout.
   If the app is stuck, liveness fails: Kubernetes restarts the container. Don't check the database in liveness, or a database outage restarts everything.

Limitation de débit

Rate limiting



La réponse, étape par étape · *Step-by-step answer*

1. Chaque client dispose d'un quota, par exemple 100 requêtes par minute (algorithme du seau à jetons).
   Each client has a quota, e.g. 100 requests per minute (token bucket algorithm).
2. S'il reste un jeton, la requête est traitée.
   If a token is left, the request is processed.
3. Quota dépassé : on répond 429 Too Many Requests, le code prévu pour ce cas.
   Quota exceeded: reply 429 Too Many Requests, the status made for this case.
4. Retry-After indique quand réessayer. 503 signifie que le serveur est indisponible, 403 un accès interdit : ce n'est pas la même chose.
   Retry-After says when to retry. 503 means the server is unavailable, 403 means access denied: not the same thing.